



华南理工大学

South China University of Technology

本科毕业设计（论文）

无人驾驶赛车行车路径规划研究

学 院	机械与汽车工程学院
--------	-----------

专 业	车辆工程
--------	------

学生姓名	龚国铮
------	-----

学生学号	201630053460
------	--------------

指导教师	李巍华
------	-----

提交日期	2020 年 5 月 25 日
------	-----------------

摘 要

在中国大学生无人驾驶方程式大赛规则的限定下，研究了无人驾驶赛车路径规划问题，整体目标是设计出高效和稳定的规划层算法，主要开展了如下工作：

应用 Delaunay 三角剖分把桩桶数据组成的空间点集进行离散化，并且通过合理设计代价函数，运用贪婪算法实现了边界检测和中心线提取，成功规划出一条粗糙的路径。利用一组 U 型弯道数据对算法进行仿真验证，结果显示算法对桩桶颜色数据的缺失有一定的容许度，展现了算法的鲁棒性。

应用最优化理论，对粗糙的路径进行优化，使赛车可以充分利用赛道的宽度。设计了集规划控制一体化的局部控制器，综合车辆动力学模型与车辆运动学模型的优势，建立了车辆线性时变预测模型。根据控制和规划的需要合理地定义了输出量，并进行了线性化处理。利用预测模型，合理设计目标函数和约束，以寻找使车辆保持在边界内，本身足够光滑，并且相对中心线预测弧长最大的路径，从而获得更快的圈速。阐述了把优化问题转化标准二次规划问题的方法，以方便计算机的求解。为了提升算法的效率，进一步优化了算法，使算法的求解速度快了大约 10 倍，同时对算法进行了失效处理，以增强其鲁棒性。在局部控制器的算法框架下，又设计了全局路径生成算法，以使用更加简单有效的控制算法或者车辆运动控制领域更加成熟实用的控制算法进行轨迹跟踪，从而弥补局部控制器的不足。

通过四阶龙格-库塔法对算法进行了仿真，对比了未优化算法和优化算法的性能，结果表明对算法进行优化可以大幅度提升性能。在 CarSim/Simulink 联合仿真平台上验证了局部规划算法和全局路径规划算法在两个地图的性能，分析整个过程中车辆的运行轨迹、平均求解时间、平均圈速等数据，结果表明两种算法都具有高效性和稳定性。

关键词：无人驾驶车辆；路径规划；车辆模型；贪婪算法；快速模型预测控制；仿真分析

Abstract

This article studies the path planning problem related to the driverless racing car under the restriction of the rules of the Formula Student Auto Competition. The overall goal is to design an efficient and stable planning algorithm. The main works about path planning are as follows,

Using Delaunay triangulation to discretize the spatial point set composed of pile data. Designing the cost function reasonably and using greedy algorithm to detect the boundary and get the centerline successfully, which is the rough path used later. There is a certain tolerance for the lack of color data in the pile, which shows the robustness of the algorithm.

Applying the optimization theory to optimize the rough path, so that the car can make full use of the width of the track. A local controller that integrates path planning and car control is designed. Establishing the linear time-varying prediction model with the combine of the vehicle dynamics model and the vehicle kinematics model. According to the requirements of control and planning, the output variables is reasonably defined and linearized. Using the prediction model, the objective function and constraints are reasonably designed to find the path that keeps the vehicle within the boundary and is sufficiently smooth to maximize progress on the track, so as to reduce the driven lap time. The method of transforming the optimization problem into the standard quadratic programming problem is expounded to facilitate the computer solution. In order to improve the efficiency, the algorithm was further optimized and the solution time is reduced by 10 times. At the same time, the algorithm processes the failure of solution to enhance its robustness. Under the algorithm framework of the local controller, a global path generation algorithm is designed so that we can use simpler and more effective control algorithms or more mature and practical control algorithms which is in the field of vehicle control for the trajectory tracking, thereby making up for the defects of the local controller.

One simulation experiment is carried out using the fourth-order Runge-Kutta method. Comparing the performance of the unoptimized algorithm and the optimized algorithm. The results show that optimizing the algorithm can greatly improve performance. The other simulation experiment is carried out on the CarSim/Simulink joint simulation platform. Comparing the performance of local planner and global path planning algorithm under two

maps. Analyzing the data such as the vehicle's running trajectory, average solution time, average lap-time, etc. The results show that both algorithms have high efficiency and stability.

Keywords: Unmanned vehicle; Path planning; Vehicle model; Greedy algorithm; Fast model predictive control; Simulation analysis

目 录

摘 要	I
Abstract.....	II
目 录	IV
第一章 绪论	1
1.1 课题背景及意义	1
1.2 路径规划问题的一般性描述	1
1.3 国内外研究现状	2
1.4 课题研究内容及方法	4
第二章 车辆模型与轮胎模型	6
2.1 车辆动力学模型	6
2.2 轮胎模型	9
2.3 车辆运动学模型	12
2.4 小结	13
第三章 基于贪婪策略的中心线提取算法	14
3.1 问题概述	14
3.2 状态空间的离散化	14
3.3 基于贪婪策略的中心线搜索算法	15
3.4 算法仿真结果分析	19
3.5 小结	21
第四章 基于预测模型与边界约束的规划算法	22
4.1 问题概述	22
4.2 非线性模型预测控制	22
4.3 输出量的定义	24
4.4 线性时变预测模型	27
4.5 线性时变模型预测局部优化路径生成器	36

4.6 算法优化	44
4.7 全局路径生成算法	48
4.8 小结	49
第五章 仿真结果分析	50
5.1 四阶龙格-库塔法仿真器仿真分析	50
5.2 CarSim/Simulink 联合仿真结果分析	58
5.3 小结	69
总结与展望	70
1. 论文工作总结	70
2. 工作展望	71
参考文献	73
附录	76
致谢	78

第一章 绪论

1.1 课题背景及意义

智能交通系统的兴起为汽车工业的发展带来了巨大的改变，促使国内外车企投入更多资金研究智能车技术。同时，国家为了鼓励大学生学习掌握最新的技术，发挥自己的创新能力和良好的团队协作能力，于 2017 年开始创办中国大学生无人驾驶方程式大赛（FSAC），该大赛允许各大学车队的本科生或研究生参加，目的是构想、设计、制造、开发并完成一辆具有无人驾驶功能，并能够以自动驾驶模式完成对应动态赛事的纯电动方程式赛车并参加比赛^[1]。该赛事吸引了广大大学生方程式赛车队的参与，上一年，华南理工大学首次参加 FSAC 比赛，但由于无人系统路径规划层的相关算法还不够完善，动态赛事的成绩不尽人意。因此，研究出一套通用、高效、稳定的路径规划算法显得十分必要。

无人驾驶系统由感知层、定位层、规划层和控制层组成，规划层作为上接感知定位，下接智能控制的关键桥梁，是实现无人驾驶的基础。规划层的主体是路径规划算法，其任务是根据感知层提供的精确的传感器数据和 GPS-INS (Global Positioning System - Inertial Navigation System) 组合导航系统提供的定位数据，按照一定的标准，搜索出一条可行驶的最优路径，并且将路径数据传递给控制层进行轨迹跟随。路径规划算法不仅广泛用于无人驾驶，而且广泛用于智能移动机器人等人工智能领域，是相关领域研究的热点，具有良好的研究前景。

1.2 路径规划问题的一般性描述

作为课题研究的基础，首先对路径规划问题进行一般性描述。无人驾驶路径规划的任务是在已知无人车辆的状态与几何形状，还有车载传感器提供的道路障碍物、边界等信息的前提下，计算出一条从源点到目标点的最优路径，而对于路径最优性的评估，可以按照是否避免碰撞、是否满足动力学约束、路径是否足够平滑等准则进行。在获取最优路径后，将结果送至运动控制层，控制层根据轨迹与定位信息实时计算出控制量，以实现轨迹跟随。

车辆的所有状态集合组成构形空间，可用 X 表示，其中 $X \subseteq \mathbb{R}^n$ ，于是 X 就作为规划问题的状态空间。设 $X_{\text{obs}} \subseteq X$ 是与障碍物碰撞的状态空间， $X_{\text{free}} = X \setminus X_{\text{obs}}$ 为允许状态的结果集。令 $\mathbf{x}_{\text{start}} \in X_{\text{free}}$ 为初始状态， $\mathbf{x}_{\text{goal}} \in X_{\text{free}}$ 为期望的最终状态。连续映射 $\sigma: [0,1] \mapsto X$ 定义为构形空间的一条路径， Σ 是所有非平凡路径的集合。

然后正式定义最优路径规划问题的搜索路径 σ^* ，该路径使给定代价函数 $c: \Sigma \mapsto \mathbb{R}_{\geq 0}$ 最小化，且会连接通过 $\mathbf{x}_{\text{start}}$ 和 $\mathbf{x}_{\text{goal}}$ 之间的自由空间，

$$\sigma^* = \arg \min_{\sigma \in \Sigma} \{c(\sigma) \mid \sigma(0) = \mathbf{x}_{\text{start}}, \sigma(1) = \mathbf{x}_{\text{goal}}\}, \quad (1-1)$$

$$\forall s \in [0,1], \sigma(s) \in X_{\text{free}} \quad ,$$

其中 $\mathbb{R}_{\geq 0}$ 为非负实数集。

所以，无人驾驶车辆的规划问题就是定义一个合适的代价函数，搜索整个允许状态的结果集，找到从源点到目标点中满足代价函数最小化的路径，并输出给控制层进行控制。

1.3 国内外研究现状

当今，无人驾驶领域具有重要实用意义的有 4 类路径规划算法：传统算法、智能优化算法、基于强化学习的算法以及混合算法^[2]。

传统的路径规划算法可大体分为两大类，一类是以 A* 算法为代表的基于图搜索的路径规划算法，另一类是以快速扩展树（rapidly exploring random tree, RRT）为代表的基于采样的路径规划算法。基于图搜索的路径规划算法的研究最早可以追溯到 20 世纪 50 年代 Dijkstra 提出的基于贪婪策略的 Dijkstra 算法^[3]，但是 Dijkstra 算法由于其广度优先的性质会导致搜索太多无关节点，求解效率并不高。20 世纪 60 年代 Hart 等人^[4]提出了启发式的 A* 搜索算法，通过设计代价函数，并逐一搜索代价函数最小的点，进而大大提升了搜索效率。虽然 A* 能有效解决最短路问题，但是其规划出来的路径折点多，而且容易陷入死循环，所以随后相关研究人员对 A* 算法进行了一系列改进，发展了双重 A*、DMHA* 等算法^[2]。与基于图搜索的算法不同，基于采样的路径规划算法通过对空

间进行均匀随机采样，从而得以探索整个空间。基于采样的算法优点在于不需要对空间进行离散化处理，也不需要预先对空间中的可行空间进行建模，缺陷在于不能处理非完整约束动力学问题。基于采样的算法中最有代表性的就是 RRT 算法^[5]，RRT 算法通过树生长的方式随机探索可行空间，具有概率完备性，也即随着时间的增加，搜索树会充满整个状态空间，找到可行解。但是 RRT 算法有盲目搜索的缺点，为了解决这个问题，Kuffner 等人提出了 RRT-connect 算法^[6]，从起始和目标点分别建立搜索树，引导树向目标状态扩展，加快搜索速度，提高搜索质量。RRT 与 RRT-connect 算法均不能评估路径的好坏，只能找到可行解，Karaman 等人借助 A* 的思想提出了 RRT* 算法^[7]，将距离定义为代价函数，通过重新选择父节点和重新布线，找到逐次优化的路径。文中还证明了 RRT 返回的路径是次优路径的概率为 1，说明所有基于 RRT 的方法几乎肯定是次优的，而 RRT* 算法被证明是渐近最优的，也就是当迭代次数接近无穷大时，找到最优解的概率接近于 1。Gammell 等人在 RRT* 的基础上又进行了改进^[8]，他们称这种方法为 Informed RRT*，由于 RRT* 的搜索范围如同 RRT 一样，在整个可行空间进行随机采样，难免复杂度高，所以 Gammell 等人把 RRT* 的搜索范围缩小为一个超椭圆，并且给出了相关证明，最后用实验验证了 Informed RRT* 的性能的确优于 RRT*。但 Informed RRT* 方法可能会在起始点与终点间有较长横跨障碍物时失效。

除了以上两大类传统方法，比较常用的传统方法还有人工势场算法。算法的基本原理是把空间当作是一个力场，障碍物会对车辆产生斥力，目标点会产生引力，然后通过计算合力来获取加速度，并更新车辆的状态，如此反复，直到车辆走到终点。杨一波等人^[9]通过在斥力势场函数中加入目标点与被控对象之间的距离作为引入调节因子来解决普通人工势场算法目标不可达问题，通过使被控对象在向目标点运动的同时所受斥力相应减小，避免了目标不可达的问题。安林芳等人^[10]提出一种按照正弦避障换道方式以最小安全距离作为对称距离构建障碍点模型的方法，保证局部目标点处于道路的对称轴线上，并通过智能车辆行驶速度和转弯半径对转向角进行约束控制。

近年来，智能优化算法不断发展，其灵感来源于自然界生物的行为规律，优点是具有自学习、自决定的功能。典型的智能优化算法包括蚁群算法、触须算法和智能水滴算法等。张明环等人^[11]研究了一种基于触须算法的智能车避障方法，介绍了如何建立障碍物地图，以及不同速度值下触须和可行区域的计算方法以及决策机制的实现，该文献

给出了三个触须的挑选原则，并且进行了算法仿真及结果分析。史恩秀等人^[12]把蚁群算法用到了全局路径规划中，并且分析了蚁群算法中不同参数对结果路径长度与迭代次数的影响。

经典的路径规划算法和智能算法或者基于强化学习的算法都可以实现无人驾驶车辆的路径规划，但是每种算法都有其局限性，在不同情境下，算法可能会体现出不同的性能。所以，将多种算法相结合也成为相关领域人员研究的热点，比如 A*与蚁群相结合的算法^[13]，可以充分发挥 A*与蚁群算法的优点。冯来春等人^[14]提出了用 A*创建一个 RRT 的引导域，同时在进行最近邻搜索时考虑了车辆的转向角约束，相比于 connect-RRT，其搜索空间减小、效率上升。

除了[2]中提到的四大类方法，基于最优化理论的方法也有许多人研究，Liniger 等人^[15]提出了一种规划控制一体化的方法，基于车辆的动力学模型，通过设计目标函数和约束把规划控制问题转化为最优化问题并求解，找出一条满足车辆动力学约束的最优路径。文中还提出了一种基于车辆运动学模型的规划方法，根据运动学模型预先计算出一系列路径，再在实时规划过程中挑选出一条最优的路径，该方法规划得到的路径没有考虑多重转弯等复杂弯道的影响，具有一定局限性，但是其解算效率高。Christ 等人^[16]根据非线性规划理论提出了寻找最小圈速时间路径的方法，虽然产生了较完美的路径，但是求解时间较长，不适用于实时规划。Heilmeier 等人^[17]根据二次规划理论提出了寻找最小曲率路径的方法，每个采样时刻的平均迭代求解时间为 0.85s，实时性仍然不够高。

1.4 课题研究内容及方法

论文主要研究了无人驾驶路径规划层算法的总体设计思路，研究对象是无人赛车，使用的工具有 C++、MATLAB、CarSim 和 Simulink，其中 C++和 MATLAB 用于算法的实现，CarSim/Simulink 用于算法的联合仿真。研究内容包括三大主要部分：

(1) 基于桩桶数据的中心线提取算法，主要内容是根据赛道的桩桶数据进行边界检测和提取赛道的中心线，其中桩桶数据主要由感知层与定位层提供，以桩桶在世界坐标系下的 x 、 y 坐标和对应的颜色表示，运用 Delaunay 三角剖分算法与贪婪算法得到赛道的中心线，对比颜色信息完整时和有缺失时的差异，验证算法是否具有足够的鲁棒性。得到赛道中心线数据后，可以根据赛道宽度和赛车宽度对中心线进行左右的平移，产生

虚拟的左右边界，并作为第二部分的输入进行进一步优化。

(2) 运用最优化理论，对中心线进行进一步优化。设计了集规划控制一体化的局部控制器的算法框架，首先，建立车辆的运动学与动力学模型，定义相应的状态量、输入量与输出量，并对模型进行线性化和离散化处理，得到车辆的线性时变预测模型；随后，设计相关的目标函数与约束，把路径规划问题转化优化问题求解；最后，为了方便计算机的求解，设计优化变量，推导如何将优化问题转化为标准二次规划问题。为了提高算法的效率，满足实时性需求，要对算法进行优化；同时为了确保算法具有较高的鲁棒性，还要求算法能够处理求解失败的情况。基于以上算法框架，进一步设计了全局路径生成算法，以便使用更加简单有效的控制算法或者车辆运动控制领域比较成熟实用的控制算法进行轨迹跟踪，从而弥补局部控制器的不足。

(3) 仿真及结果分析。算法验证将在两个仿真器进行，一个是应用四阶龙格-库塔法的仿真器，另一个是 CarSim/Simulink 联合仿真器。使用了两个求解器求解第二部分的标准二次规划问题，一个是 MATLAB 自带的 quadprog 求解器，另一个是 hpipm 求解器，由于 hpipm 仅仅支持 linux 系统，所以不能在 CarSim/Simulink 联合仿真器上进行仿真，只能运用四阶龙格-库塔法通过车辆动力学模型进行仿真，在这个仿真环境下验证了算法优化后模型的性能和 hpipm 求解器的性能。在 CarSim/Simulink 联合仿真器中使用 quadprog 求解，验证了局部规划算法和全局路径规划算法在两个地图的性能，分析整个过程中车辆的运行轨迹、平均求解时间、平均圈速等数据，验证算法效果。

尽管所提供的方法研究的是无人赛车，并且假设赛道上没有动/静态障碍物，但是只要给出赛道边界及中心线，所提方法完全可拓展到有障碍物的乘用车场景，具有实用价值。

第二章 车辆模型与轮胎模型

对于模型预测控制来说，建立合适的模型是十分重要的，车辆模型可以分为运动学模型和动力学模型两大类，运动学模型在低速下可以很好地预测车辆的行为，动力学模型则在中高速情况下工作良好，为了能够更加准确地预测车辆的行为，同时建立了车辆运动学与车辆动力学模型，在低速情况下，采用运动学模型预测车辆行为，在中高速情况下采用动力学模型，这样就可以综合两者的优点。此外，过于复杂的运动学模型或者动力学模型会增加计算负担，使得算法不能很好地满足实时性要求，因此选择合适的运动学或动力学模型并对它们进行适当的简化是十分必要的。

对于车辆动力学模型来说，轮胎模型是很重要的一环，因此完成动力学模型的建立后，需要重点研究轮胎的特性，从而更加准确地描述车辆的受力情况。

2.1 车辆动力学模型

动力学主要研究作用于物体的力与物体运动的关系，因为车辆大部分情况都处于中高速状态，所以动力学模型是最重要的模型，本章也最先研究车辆的动力学模型，以用于后续的模型预测和优化。此外，也可以在动力学模型中发现其缺陷，为后续运动学模型的建立给出理由。

本节将赛车建模为具有非线性轮胎力的动力学自行车模型，该模型可以很好地预测车辆行为，并且模型足够简单，可以满足后续模型预测及优化问题的实时求解需求，模型基于如下假设：

- (1) 假设车辆只做二维平面运动，即车辆沿 z 轴的平动、绕 y 轴的俯仰以及绕 x 轴的侧倾均忽略不计；
- (2) 只考虑纯侧偏轮胎特性，忽略轮胎力的纵横向耦合关系；
- (3) 不考虑载荷的左右转移；
- (4) 车辆为前轮转向，后轮转角恒为 0；
- (5) 假设悬架系统和车辆是刚性的，忽略车辆的悬架特性；
- (6) 车辆没有主动制动，纵向驱动力集中作用于质心。

具体模型如图 2-1 所示。其中绿色向量指代质心的位置向量，蓝色向量指代各个速

度的方向，红色向量指代车的受力情况。

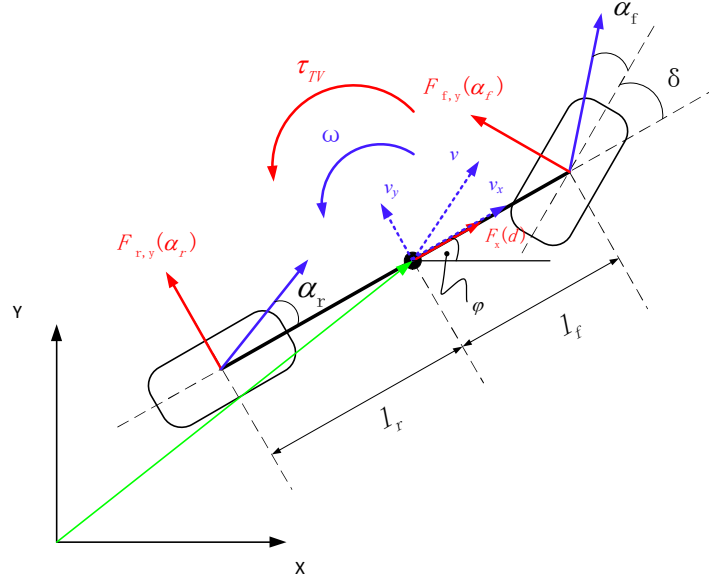


图 2-1 车辆的动力学自行车模型

惯性坐标系为固定在地面的坐标系，原点根据实际定义。车辆坐标系为固定在车辆上的坐标系，原点在车辆质心，其 x 、 y 方向分别对应图 2-1 中 v_x 、 v_y 的方向， z 轴与 x 、 y 形成右手系。

选取的状态量为惯性坐标系下质心的坐标值 X 、坐标值 Y 、质心航偏角 φ 、质心纵向速度 v_x 、质心横向速度 v_y 和车辆的角速度 ω 。选取的控制量为驱动电机的 PWM 占空比 d 和前轮转角 δ ，这里假设电机可以正转和反转，正转对应的是驱动，反转对应的是制动， d 的范围为 $[-1,1]$ ，1 代表最大驱动力，-1 代表最大制动力。根据图 2-1 所示模型，可以列出如下的动力学方程，

$$\dot{X} = v_x \cos(\varphi) - v_y \sin(\varphi) \quad (2-1a)$$

$$\dot{Y} = v_x \sin(\varphi) + v_y \cos(\varphi) \quad (2-1b)$$

$$\dot{\varphi} = \omega \quad (2-1c)$$

$$\dot{v}_x = \frac{1}{m} (F_x - F_{f,y} \sin \delta + m v_y \omega) \quad (2-1d)$$

$$\dot{v}_y = \frac{1}{m} (F_{r,y} + F_{f,y} \cos \delta - m v_x \omega) \quad (2-1e)$$

$$\dot{\omega} = \frac{1}{I_z} (F_{f,y} l_f \cos \delta - F_{r,y} l_r + \tau_{TV}) \quad (2-1f)$$

上述方程中, m 代表车辆的质量, I_z 代表车辆绕 z 轴的转动惯量。 l_f 和 l_r 分别是质心到前轴和后轴的距离。 $F_{f,y}$ 和 $F_{r,y}$ 分别是前/后轮的侧向力, 前面的字母 f/r 分别代表前胎(front tyre)/后轮(rear tyre), 后面的字母 y 代表车辆坐标系中 y 方向的受力。 F_x 是车辆受到的总纵向力, 使用了直流电机模型、滚动摩擦力模型与空气阻力模型来计算^[15], 具体公式如 (2-2) 所示。

$$F_x = (C_{m1} - C_{m2}v_x)d - C_r - C_d v_x^2 \quad (2-2)$$

式中: C_{m1} 、 C_{m2} 是与直流电机模型相关的参数, C_r 是车辆所受的滚动阻力, $C_d v_x^2$ 是车辆所受的空气阻力。所有的参数 C_α , $\alpha \in \{m1, m2, r, d\}$, 都要通过实验进行标定。

自行车模型是不考虑左右轮单独作用的, 但在实际运用中, 由于底层控制的逻辑, 双电机后轮驱动系统或者四电机四轮驱动系统会在某些情况下给予左右轮不同的力矩, 从而使得车辆产生附加的扭矩, 以促进车辆的转向。因此在动力学自行车模型中加入了附加力矩项 τ_{TV} , 这一项是由底层控制器决定的, 这里将其建模为一个比例控制器, 由运动学的理想目标角速度 ω_{target} 与当前角速度 ω 的差值决定, τ_{TV} 的具体计算方式如下,

$$\omega_{\text{target}} = \delta \frac{v_x}{l_f + l_r} \quad (2-3)$$

$$\tau_{TV} = (\omega_{\text{target}} - \omega) P_{TV} \quad (2-4)$$

其中计算 ω_{target} 运用了关于 δ 的小角度假设, 也即 $\tan(\delta) \approx \delta$ 。 P_{TV} 是比例控制系数, 与具体的底层控制器有关。

进一步地, 令 $\tilde{x} = [X, Y, \varphi, v_x, v_y, \omega]^T$, $\tilde{u} = [d, \delta]^T$, f_{dynamic} 代表 (2-1) 中的关系, 有

$$\dot{\tilde{x}} = f_{\text{dynamic}}(\tilde{x}, \tilde{u}) \quad (2-5)$$

这样, 就可以通过计算 $\frac{\partial f_{\text{dynamic}}}{\partial \tilde{x}}$ 与 $\frac{\partial f_{\text{dynamic}}}{\partial \tilde{u}}$ 对模型进行线性化处理, 其中式 (2-1d) 与

(2-1e) 需要求出 $\frac{\partial F_{f,y}}{\partial \tilde{x}}$ 、 $\frac{\partial F_{f,y}}{\partial \tilde{u}}$ 、 $\frac{\partial F_{r,y}}{\partial \tilde{x}}$ 和 $\frac{\partial F_{r,y}}{\partial \tilde{u}}$, 这部分的计算将在轮胎模型中介绍。

2.2 轮胎模型

车辆动力学模型需要对轮胎侧向力进行建模，要想知道地面对车辆的侧向力大小与哪些因素有关，需要研究轮胎与地面的相互作用，轮胎模型正是研究这个问题的。轮胎模型的构造一般分为两种^[18]，一种是理论模型（物理模型），这种模型从轮胎的物理结构出发，从形变机理角度建立了轮胎的数学模型，较有影响的是 Gim 模型、Fiala 模型等；另一种是经验公式或半经验公式模型，其本质是对轮胎数据进行回归分析，给出轮胎力的经验公式并且拟合相应的参数，较有影响的是我国郭孔辉院士建立的幂指数公式半经验模型以及 H. B. Pacejka 教授给出的魔术公式轮胎模型。

2.2.1 魔术公式轮胎模型的建立

魔术公式因为其健壮性和高拟合精度，已经被轮胎制造商广泛使用，且正在成为工业标准。本文选用魔术公式轮胎模型 Pacejka 模型，该模型是 H. B. Pacejka 教授于 1987 年提出的^[19]，对于侧向力的计算如下，

$$F_{f,y} = D_f \sin\left(C_f \arctan\left(B_f \alpha_f\right)\right) \quad (2-6)$$

$$F_{r,y} = D_r \sin\left(C_r \arctan\left(B_r \alpha_r\right)\right) \quad (2-7)$$

α_f 和 α_r 分别是前后轮侧偏角，计算如下，

$$\alpha_f = -\arctan\left(\frac{\dot{\phi}l_f + v_y}{v_x}\right) + \delta \quad (2-8)$$

$$\alpha_r = \arctan\left(\frac{\dot{\phi}l_r - v_y}{v_x}\right) \quad (2-9)$$

式中， $B_\bullet$ 、 $C_\bullet$ ， $\bullet \in \{f, r\}$ 是与具体轮胎相关的参数， $D_\bullet$ ， $\bullet \in \{f, r\}$ 是与载荷和轮胎相关的参数，考虑理想情况， $D_\bullet$ 取为

$$D_\bullet = 1.2m_\bullet g \quad (\bullet \in \{f, r\}) \quad (2-10)$$

m_f 、 m_r 分别为前后轴等效质量。

根据上面的公式，可以求出 $\frac{\partial F_{f,y}}{\partial \tilde{x}}$ 、 $\frac{\partial F_{f,y}}{\partial \tilde{u}}$ 、 $\frac{\partial F_{r,y}}{\partial \tilde{x}}$ 和 $\frac{\partial F_{r,y}}{\partial \tilde{u}}$ ，具体计算公式如下，

$$\frac{\partial F_{f,y}}{\partial v_x} = D_f \cos\left(C_f \arctan(B_f \alpha_f)\right) \cdot \frac{B_f C_f}{1 + (B_f C_f)^2} \cdot \frac{\dot{\phi} l_f + v_y}{v_x^2 + (\dot{\phi} l_f + v_y)^2} \quad (2-11)$$

$$\frac{\partial F_{f,y}}{\partial v_y} = D_f \cos\left(C_f \arctan(B_f \alpha_f)\right) \cdot \frac{B_f C_f}{1 + (B_f C_f)^2} \cdot \frac{-v_x}{v_x^2 + (\dot{\phi} l_f + v_y)^2} \quad (2-12)$$

$$\frac{\partial F_{f,y}}{\partial \omega} = D_f \cos\left(C_f \arctan(B_f \alpha_f)\right) \cdot \frac{B_f C_f}{1 + (B_f C_f)^2} \cdot \frac{-l_f v_x}{v_x^2 + (\dot{\phi} l_f + v_y)^2} \quad (2-13)$$

$$\frac{\partial F_{f,y}}{\partial \delta} = D_f \cos\left(C_f \arctan(B_f \alpha_f)\right) \cdot \frac{B_f C_f}{1 + (B_f C_f)^2} \quad (2-14)$$

$$\frac{\partial F_{r,y}}{\partial v_x} = D_r \cos\left(C_r \arctan(B_r \alpha_r)\right) \cdot \frac{B_r C_r}{1 + (B_r C_r)^2} \cdot \frac{-(l_r \dot{\phi} - v_y)}{(l_r \dot{\phi} - v_y)^2 + v_x^2} \quad (2-15)$$

$$\frac{\partial F_{r,y}}{\partial v_y} = D_r \cos\left(C_r \arctan(B_r \alpha_r)\right) \cdot \frac{B_r C_r}{1 + (B_r C_r)^2} \cdot \frac{-v_x}{(l_r \dot{\phi} - v_y)^2 + v_x^2} \quad (2-16)$$

$$\frac{\partial F_{r,y}}{\partial \omega} = D_r \cos\left(C_r \arctan(B_r \alpha_r)\right) \cdot \frac{B_r C_r}{1 + (B_r C_r)^2} \cdot \frac{l_r v_x}{(l_r \dot{\phi} - v_y)^2 + v_x^2} \quad (2-17)$$

除了式 (2-11) ~ (2-17) 外， $F_{r,y}$ 和 $F_{f,y}$ 对其他状态量和控制量的偏导数均为 0。

2.2.2 参数对侧向力曲线的影响

为了准确描述轮胎的侧向力，需要选择合适的参数以符合实际轮胎数据，图 2-2 展示了取 $m_f = 1167.66\text{kg}$ ， $m_r = 782.34\text{kg}$ 时，车辆侧向力随各参数变化的曲线。

从图中可以看出，当 $B_{\bullet}$ 越大时，轮胎的线性区越窄，同时在大侧偏角时，侧向力将大幅度减小， $C_{\bullet}$ 对于曲线的影响也类似，只不过 $C_{\bullet}$ 稍微增大一点，就会使曲线产生较大的改变。通过分析市场中常用轮胎的数据，可以得出 $B_{\bullet}$ 一般在 [10, 16] 区间内， $C_{\bullet}$ 一般在 [1.4, 2] 区间内。（ $\bullet \in \{f, r\}$ ）

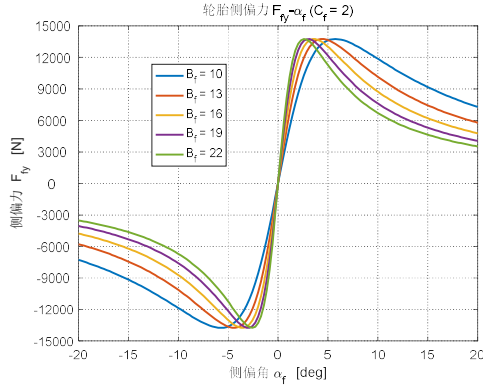
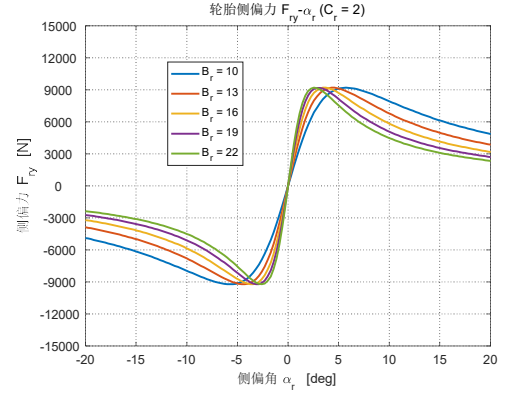
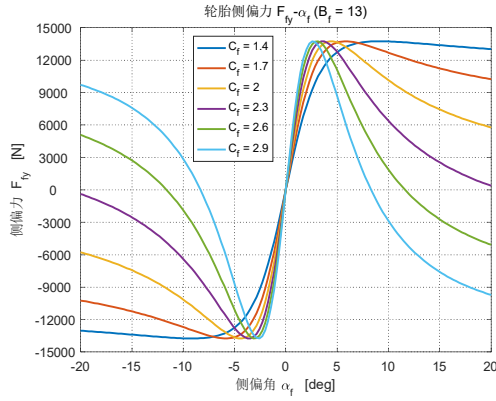
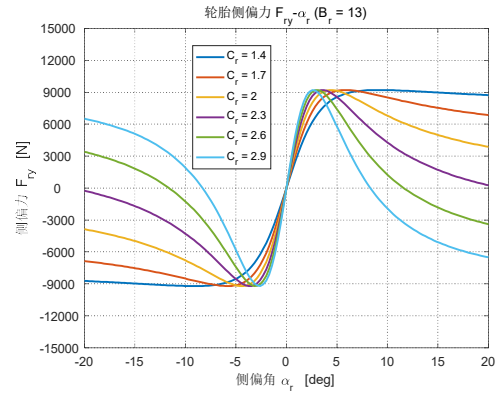
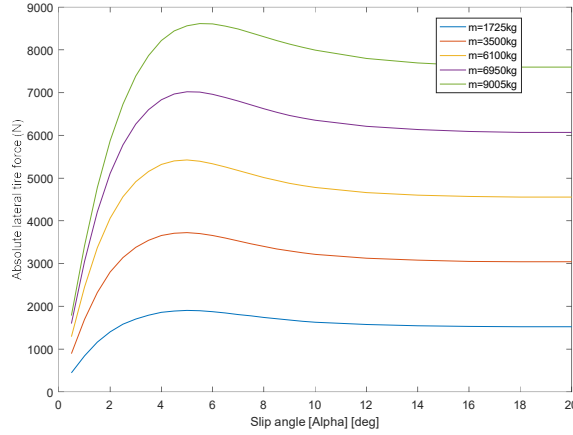

 a) $C_f=2$ 时 B_f 对前轮侧向力曲线的影响

 b) $C_r=2$ 时 B_r 对后轮侧向力曲线的影响

 c) $B_f=13$ 时 C_f 对前轮侧向力曲线的影响

 d) $B_r=13$ 时 C_r 对后轮侧向力曲线的影响

图 2-2 各参数对于前后轮侧向力曲线的影响

2.2.3 参数拟合

如果已经测得车辆的侧向力与侧偏角的数据（如图 2-3 所示为 CarSim 上使用的轮胎数据），那么可以利用 MATLAB 自带的 cftool 工具对参数进行拟合。假设前后轮用的是同一种轮胎，即 $B_r = B_f$ ， $C_r = C_f$ ，选取 CarSim 上 $m=9005\text{kg}$ 时的曲线数据，用 cftool 工具进行拟合可以得到 $B_{\bullet}=16.34$ ， $C_{\bullet}=1.5$ 。（ $\bullet \in \{f, r\}$ ）


 图 2-3 CarSim 中不同载荷下的 F_y - α 曲线

2.3 车辆运动学模型

上面小节介绍的车辆动力学模型固然已经能够很好地描述车辆的运动，可是仍然存在缺陷，低速情况下通过式（2-8）和（2-9）计算得到的侧偏角会较大，由于轮胎模型的限制，会导致车辆的侧向力计算不准确，而低速情况在比赛开始阶段和急转弯阶段是十分重要的，为了解决这个问题，在低速情况下选用不依赖侧偏角的车辆运动学模型。

运动学模型的推导主要用了低速下车辆的稳态响应，

$$\tan(\delta) = \frac{l_f + l_r}{R} \quad (2-18)$$

其中 R 是车辆的转弯半径，再通过角速度与转弯半径的关系和 δ 的小角度假设，可得，

$$\omega = \frac{v_x}{R} = \frac{v_x \delta}{l_f + l_r} \quad (2-19)$$

v_y 可由 $v_y = \omega l_r$ 得到，忽略轮胎的侧向力，车辆运动学模型可描述如下，

$$\dot{X} = v_x \cos(\varphi) - v_y \sin(\varphi) \quad (2-20a)$$

$$\dot{Y} = v_x \sin(\varphi) + v_y \cos(\varphi) \quad (2-20b)$$

$$\dot{\varphi} = \omega \quad (2-20c)$$

$$\dot{v}_x = \frac{F_x}{m} \quad (2-20d)$$

$$\dot{v}_y = (\dot{\delta}v_x + \delta\dot{v}_x) \frac{l_r}{l_r + l_f} \quad (2-20e)$$

$$\dot{\omega} = (\dot{\delta}v_x + \delta\dot{v}_x) \frac{1}{l_r + l_f} \quad (2-20f)$$

其中 F_x 的定义跟动力学的一致。还要注意式 (2-20e) 和 (2-20f)，因为需要已知 δ 的一阶导数，可是动力学模型中定义的状态量与控制量均不涉及 $\dot{\delta}$ ，所以需要定义新的状态量与控制量，文中重新定义为 $\tilde{x} = [X, Y, \varphi, v_x, v_y, d, \delta]^T$, $\tilde{u} = \Delta\tilde{u} = [\Delta d, \Delta\delta]^T$ ，通过把控制量定义为增量的形式，可以利用方程 (2-20a) ~ (2-20f) 求取状态方程对控制量和状态量的偏导，也即令

$$\dot{\tilde{x}} = f_{kinetic}(\tilde{x}, \tilde{u}) \quad (2-21)$$

根据上述定义容易求得 $\frac{\partial f_{kinetic}}{\partial \tilde{x}}$ 与 $\frac{\partial f_{kinetic}}{\partial \tilde{u}}$ 。

关于运动学与动力学模型的具体选择准则，以及对两个模型的进一步处理将在第四章中进行介绍。

2.4 小结

本章首先通过一些合理的假设，建立了车辆的动力学自行车模型，动力学模型在高速情况下比运动学模型能够更加描述车辆的运动，因此是后续章节优化问题的主要模型。该模型不仅足够简单，而且能够准确描述车辆的运动，对于有实时性要求的优化路径生成算法来说是很好的选择。另外，定义了用于模型预测的状态量与控制量，并且介绍了状态方程对于状态量与控制量偏导数的计算，这是第四章求取 **Jacobi** 矩阵的基础。

动力学模型中轮胎侧向力的计算，需要研究轮胎模型，本章建立了简单的 **Pacejka** 魔术轮胎模型。随后研究了轮胎模型中各个参数对侧向力拟合曲线的影响，得出了参数的大体区间。最后展示了在有轮胎实验数据的情况下，可以利用 **MATLAB** 的 **cftool** 工具对数据进行曲线拟合，从而得到准确的拟合参数。

由于在低速情况下，动力学自行车模型存在轮胎侧向力计算不准确的缺陷，所以本章又建立了车辆运动学模型，并且为了能够对状态方程求取偏导数，重新定义了状态量和控制量，这是后续章节的研究基础。

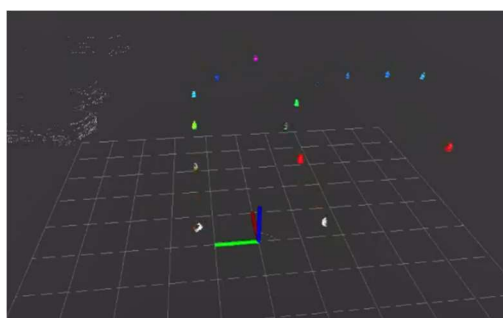
第三章 基于贪婪策略的中心线提取算法

3.1 问题概述

无论是对于赛车还是乘用车来说，获取赛道的边界和中心线是至关重要的，它们能够指示车辆的大体行驶方向与可行驶的区域。FSAC 比赛中，赛道的边界是以桩桶限定的，其中左边界用红色桩桶限定，右边界用蓝色桩桶限定^[1]。桩桶的数据可以通过感知层获取，以桩桶在车辆坐标系下的 x 、 y 坐标和相应的颜色信息表示，图 3-1 显示了感知层的相关数据，其中摄像头可以给出颜色信息和位置信息，激光雷达可以得到更加准确的位置信息，通过激光雷达与摄像头的数据融合，可以得到桩桶在车辆坐标系下的坐标值和颜色信息。



a) 摄像头的感知数据



b) 激光雷达的感知数据投影到世界坐标系

图 3-1 华南理工大学无人赛车的桩桶感知数据

然而，感知层给出的桩桶信息对于实际应用来说仍然是凌乱的，因为它们不是有序的，而轨迹跟踪需要的是有序轨迹向量，因此要解决的问题就是如何把这些凌乱的桩桶数据组织成一个有序向量，实现赛道中心线与赛道边界的提取，此外，考虑到摄像头给出的颜色信息有可能错误或者缺失，算法还应该能够有效处理这类错误。

3.2 状态空间的离散化

作为一个路径规划问题，对状态空间进行离散化处理是很有利的，有助于计算机在有限时间内找到问题的解。本文使用 Delaunay 三角剖分对桩桶数据组成的空间点集进行离散化处理。

Delaunay 三角剖分具有两个非常重要的性质，一是外接圆性质，即任意三角形的外

接圆内不包含其他点，二是最大化最小角的性质^[20]。由于这两个重要的性质，Delaunay 三角网既具有极大的应用价值，又是二维平面三角网中唯一最好的，它可以避免“极瘦”三角形的出现。图 3-2 显示了不同三角剖分的区别。



图 3-2 非 Delaunay 三角剖分（图 a），Delaunay 三角剖分(图 b)

使用 C++ 的 opencv 库实现平面点集的三角剖分，搜索空间定义为所有 Delaunay 三角形的中点，对于每个中点，定义其邻近点为该点所在边对应三角形的另外两边的中点。其中每个搜索点都对应一组左右桩桶，为了更全面地进行搜索，对每个中点，在整个搜索空间中增加另外一组与该点对应左右桩桶数据相反（也即左桩桶变为右桩桶，右桩桶变为左桩桶）的点作为共轭搜索点，如此便形成了整个离散的状态搜索空间。图 3-3 是苏黎世理工大学的 AMZRacing 车队中心线提取的结果^[21]，其中黑色线为桩桶数据的 Delaunay 三角剖分，绿色线为整个搜索空间。

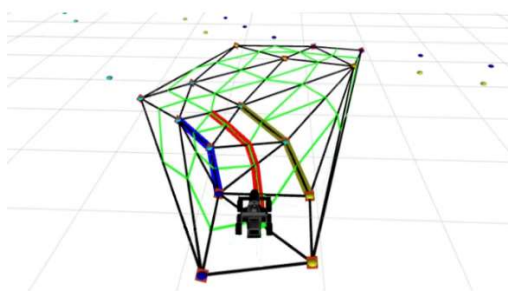


图 3-3 AMZRacing 车队中心线提取的结果

3.3 基于贪婪策略的中心线搜索算法

3.3.1 代价函数的设计

整个状态搜索空间可以产生许多不同的路径，定义赛道中心线为这些路径中的最优路径，为了在整个状态空间中搜索到最优路径，首先需要明确评价最优性的准则，在不

同路径间，通过定义代价函数来量化它们的好坏，从一个点到另一个点，会产生相应的代价，因此不同路径间的代价一般是不一样的，搜索过程分为两部分，首先是搜索从当前点出发的所有潜在路径，然后是从所有潜在路径中挑选最优路径，两部分所用到的代价函数有区别，表 3-1 列出了用来定义相应代价函数的量和相关的描述，其中每个代价量都有相应的权重，代价函数就定义为所有代价量的加权求和。

表中最后两行的代价量是两部分搜索中代价函数的差异，关于路径长度定义差异，因为在搜索所有潜在路径的过程中，如果仍将其定义为道路总长度与传感器范围的差值的绝对值，会导致搜索偏向于较长的路径，从而忽略了潜在的解；在挑选最优路径时，则应偏向于挑选长的路径，所以两部分搜索过程中路径长度的定义不同。另外，在搜索所有潜在路径的过程中，为了能够搜索更多的路径，没有用到左右边界桩桶数量的比这个代价量。

表 3-1 代价函数的设计¹

代价量	描述
*最大偏转角 $\beta_{\max} / \text{rad}$	在一条路径中，路径的最大偏转角不宜过大，否则不满足车辆的运动学和动力学约束
*左(右)边界邻近桩桶的距离与标准距离的标准差 l_{std} / m	由于桩桶的摆放是均匀的，左（右）边界上相邻的两个桩桶的距离应该是接近的，且会接近一个标准值，不会出现差别很大的情况
*左右配对桩桶的距离与道路宽度的标准差 w_{std} / m	由于道路宽度是赛规定义的，因此左右配对桩桶的距离与道路宽度的差别不会太大
*左右桩桶颜色 color	左右配对桩桶一般是左红右蓝，如果左右桩桶颜色相同，color=1，如果其中有一个桩桶的颜色不确定，color=0.5，否则 cross=0
*左右边界是否相交 cross	最优路径内的左右边界不会出现相交的情况，如果出现 cross=1，否则 cross=0
路径长度 s / m	1.搜索所有潜在路径：定义为从一个搜索点到另一个搜索点的距离与道路宽度差值的绝对值 2.挑选最优路径：定义为路径总长度与传感器感知距离差值的绝对值
左右边界桩桶数量比值 punish_ratio_of_left_right_cone	（只用于挑选最优路径）一个桩桶最多与三个桩桶配对，如果多于三个，punish_ratio_of_left_right_cone=1，否则 punish_ratio_of_left_right_cone=0

¹ 前面有*号的量代表在搜索所有潜在路径和挑选最优路径过程中都使用了

3.3.2 贪婪算法

有了代价函数，就可以以此为准则进行搜索，首先介绍搜索所有潜在路径的实现过程，对于每个结点，分别定义一个指向它父结点的 `parent` 指针，和一个指向它所有子结点的 `next_nodes` 数组，更新完所有结点的 `parent` 指针和 `next_nodes` 数组后就可以形成一棵全局搜索树，因此搜索所有潜在路径的任务就是完成这些结点的拓扑连接。

利用小顶堆实现对整个状态空间 X_{mid_set} 的贪婪搜索，通过 `visited` 表进行搜索过程中的剪枝，具体算法的流程见算法 1。

算法 1： `searchAllPaths( $X_{mid\_set}$ )`

```

1.  for node in  $X_{mid\_set}$  do
2.      cost_by_now[node.id] =  $\infty$ 
3.      visited[node.id] = false
4.  end
5.  cost_by_now[start_node.id] = 0
6.   $H_{min\_heap}$ .push(start_node, cost_by_now[start_node.id])
7.  iter_num = 0
8.  while ! $H_{min\_heap}$ .is_empty() and iter_num < MAX_ITER_NUM do
9.      cur_node =  $H_{min\_heap}$ .pop()
10.     if visited[cur_node.id] then
11.         continue
12.     visited[cur_node.id] = true
13.     for neibornode in neighbors(cur_node) do
14.         if visited[neibornode.id] then
15.             continue
16.         cost_neighbor = cur_node.calculateCostNext(neibornode)
17.         if cost_neighbor < cost_by_now[neibornode.id] then
18.             old_parent = neibornode.parent
19.             old_parent.next_nodes.delete(neibornode)

```

```

20.         neibornode.parent = cur_node
21.         cur_node.next_nodes.push(neibornode)
22.         cost_by_now[neibornode.id] = cost_neighbor
23.          $H_{min\_heap}$ .push(neibornode,cost_by_now[neibornode.id])
24.     end
25.     iter_num = iter_num+1
26. end
27. return getAllPathFrom(start_node)

```

首先，算法将车辆当前点放入小顶堆，然后进入循环，利用贪婪策略，每次都从小顶堆中取出代价函数最小的结点，并标记为 **visited**，表明已经是最优结点，以后也不必访问，从而实现对搜索空间的剪枝。随后搜索该结点的邻近结点，并计算相应的代价函数，如果该代价函数比曾经计算得到的代价更小，就更新相关的数据结构。

算法 **while** 循环执行完毕后，基本完成了全局搜索树的构建，最后的 **getAllPathFrom** 函数功能是从当前点出发，把所有潜在路径都放入 X_{paths} 集合中，具体算法流程可见算法 2。

算法通过不停搜索子结点，直到搜索至树叶，也即结点的 **next_nodes** 数组为空，然后利用该结点的 **parent** 指针一直回到起始点，便得到了一条潜在路径，将该路径放入 X_{pat} 集合内为后续挑选最优路径做准备。

算法 2： **getAllPathFrom(start_node)**

```

1.  next_tmp.push(start_node)
2.  while !next_tmp.isempty() do
3.      init(tmpVec)
4.      for nextNode in next_tmp do
5.          if nextNode.next_nodes.isempty() then
6.              insert path(from start_node to nextNode) to  $X_{paths}$ 
7.          else
8.              push all next node in nextNode.next_nodes to tmpVec

```

```

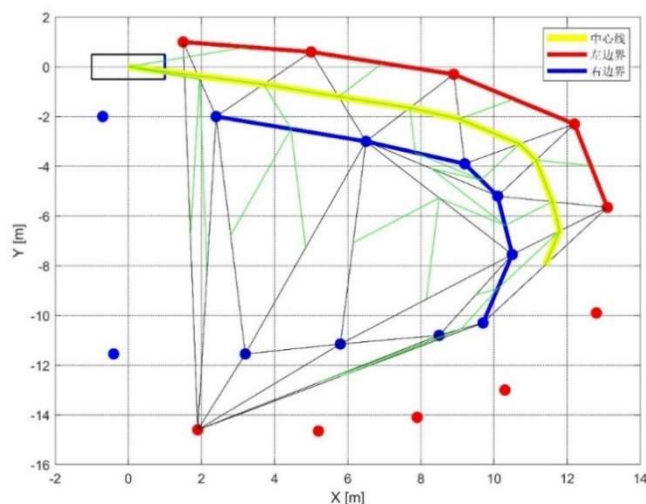
9.      end
10.     next_tmp = tmpVec
11.  end
12.  return  $X_{paths}$ 

```

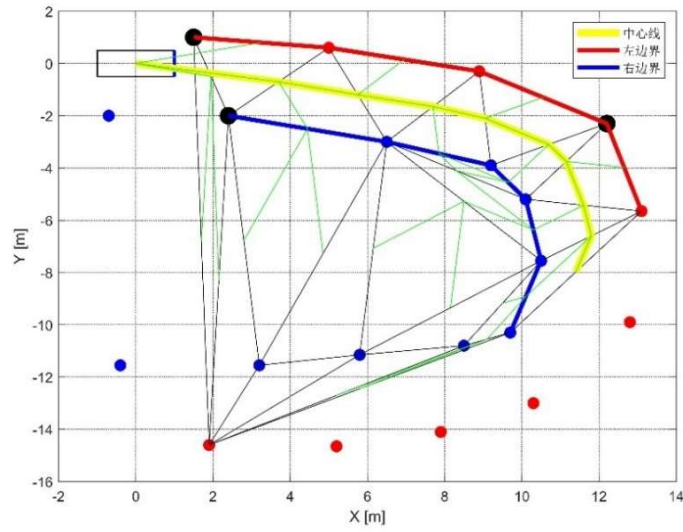
在获取所有潜在路径后，下一步就是挑选最优路径，计算每条路径对应的代价函数，然后从中选取代价函数最小的路径作为最优路径返回，该最优路径就作为待优化的中心线，这就是整个算法的工作流程。

3.4 算法仿真结果分析

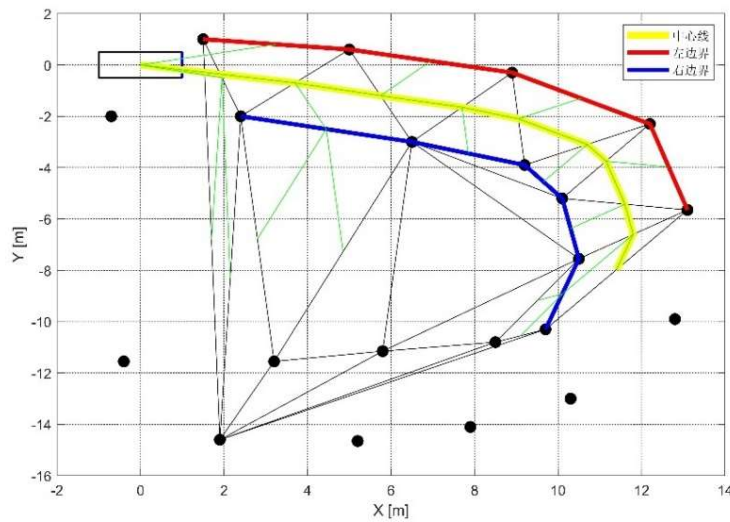
为了验证算法的性能，对算法进行了仿真，仿真数据是一组 U 型弯道数据，算法利用 opencv3.4.6 版本的库，通过 C++实现，各个代价量的权重在附录中给出。最大迭代次数 MAX_ITER_NUM 设置为 30，仿真结果如图 3-4 所示。三种结果的求解时间分别为 0.279s、0.264s、0.226s，虽然三者的求解时间都比采样周期 0.05s 大，但是运行算法时，车辆的行驶速度并不大，一般在 10km/h 左右，求解时间在 0.5s 内都不会使车辆驶出赛道或者产生危险，所以 0.25s 左右的求解时间是可以接受的。



a) 颜色信息完整的仿真结果



b)部分颜色信息缺失的仿真结果



c)所有颜色信息缺失的仿真结果

图 3-4 中心线提取算法仿真结果，其中红（蓝）色点分别是左（右）桩桶，黑色点是未知颜色的桩桶，黑色线是空间点集的 Delaunay 三角剖分，绿色线是所有潜在路径，红（蓝）色线分别是左（右）边界，黄色线是算法得到的中心线

考虑到车辆后方的点对于产生一条车辆前方的路径没有贡献，还有车辆传感器对于距离越远的桩桶点，颜色感知精度就会越低，同时为了减小搜索空间，提升效率，因此在进行 Delaunay 三角剖分前，首先对数据进行了预处理，把那些位于车辆后方（即车辆坐标系下 x 坐标小于 0）的点和与车辆距离大于 15m 的点都剔除了，这里的 15m 是个可

选值，跟具体传感器精度有关，如果传感器精度较高，可以适当提高这个值，但是也不能选得过大，因为会导致算法的搜索时间变长，不满足实时性要求，因此这个值的选取需要权衡。上面的仿真结果图中，没有进入全局搜索空间的点就是被剔除的数据点。

图 3-4 中图 a)~图 c)分别显示了在颜色信息完整、有缺失、完全缺失情况下的结果，哪怕完全没有颜色信息，由于道路的宽度是标准的，而且两同色桩桶间的距离是均匀的，所以算法仍然能给出正确的结果。三种不同的情况下，算法都能按照要求给出准确的中心线与左右边界向量，证明了算法不仅是可行的，而且对颜色信息有一定的容错能力，有一定的鲁棒性。

仿真结果中的绿色线是从车辆当前点出发的所有潜在路径，通过对比 AMZ Racing 车队的中心线提取结果（图 3-3），可以发现本文使用的算法没有盲目搜索整个空间，而是通过对一些结点进行剪枝，大大减少了潜在路径的数量，从而缩短了搜索时间，提升了效率。

最后，要对得到的中心线进行进一步处理，具体过程就是根据赛道宽度和车辆宽度对中心线进行左右平移，产生每个中心线点对应的虚拟左右边界点，以便后续的优化。

3.5 小结

本章阐述了如何在一系列凌乱的桩桶数据中提取赛道中心线和检测赛道的边界，首先明确了本章研究的问题，提出了对算法的要求。随后通过 Delaunay 三角剖分算法，把状态空间离散化，使得后续的中心线搜索算法可以在有限步内终止，其中搜索空间定义为所有 Delaunay 三角形的中点，对于每个搜索点，还定义了它的临近结点。

接着介绍了搜索算法的具体实现过程，首先通过定义代价函数明确了作为中心线的最优路径，并阐述了代价函数中各个代价量的含义和挑选理由。然后根据代价函数，使用贪婪策略进行中心线的搜索，同时通过对 visited 表进行标记实现搜索过程中的剪枝，减小搜索时间。搜索过程主要分为搜索所有潜在路径和挑选最优路径两部分，文中给出了搜索算法关键部分的伪代码，并详细讲述了实现的过程。

最后，对算法进行了仿真验证，实验数据是一组 U 型弯道的数据，分别在无颜色缺失、部分颜色缺失和完全颜色缺失的情况下进行了仿真，这几种情况下算法都可以给出准确的中心线和赛道边界，表明了算法具备一定的鲁棒性。

第四章 基于预测模型与边界约束的规划算法

4.1 问题概述

上一章通过贪婪算法得到了一系列中心线点 $(X_k^{ref}, Y_k^{ref}) (k=1,2,3,...)$ ，这些中心线点一般不能满足赛车的实际行驶需要，不能直接用于轨迹跟踪，要对它们进行优化处理，进一步规划出一条优化的局部路径或者优化的全局路径，使得车辆可以充分利用赛道的宽度，提升圈速。

基于最优化理论，本章规划算法的设计思路来源于模型预测轮廓控制（Model Predictive Contouring Control, MPCC）^[22]，通过动力学或者运动学模型确定系统的状态转移方程，通过输入不同的控制量来获取系统不同的状态向量，再通过合理设计目标函数和相关的约束控制运动轮廓，把问题转化为优化问题进行求解。

本章的具体安排如下。4.2 节从非线性模型预测控制出发，阐述了优化问题的大体求解思路。4.3~4.6 节介绍了集规划控制一体化的局部控制器的算法框架，详细介绍了输出量和相关约束的定义方法，推导了线性时变预测模型（Linear Time-Varying Predictive Model, 简称 LTVPM），以及如何把预测优化问题转化为便于计算机求解的标准二次规划（quadratic programming, 简称 QP）问题，通过求解该问题，可以得到最优状态量，里面包含了待跟踪局部路径的信息。由于算法对实时性的要求较高，因此需要进行进一步优化，通过物理变量的缩放、优化变量的升维处理等措施优化了算法，大幅减小了求解的时间。同时，本文还考虑了求解失败时的处理，提高了算法的鲁棒性。4.7 节将在上述局部规划控制器的基础上，介绍全局路径生成算法，以使用更加简单或者实用效果更好的控制算法进行轨迹跟踪，使规划与控制解耦合。4.8 节总结了本章的内容。

4.2 非线性模型预测控制

首先介绍非线性模型预测控制问题的一般形式，以了解问题的大体求解思路与轮廓，模型的转化与详细的推导将放在后面的小节。

考虑如下离散系统：

$$\begin{aligned}\xi(t+1) &= f(\xi(t), \mathbf{u}(t)) \\ \xi(t) &\in X \\ \mathbf{u}(t) &\in U\end{aligned}\tag{4-1}$$

其中： $f(\cdot, \cdot): \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^n$ 是关于当前状态变量和输入控制变量的离散非线性状态转移方程； $\xi(t) \in \mathbb{R}^n$ 是 n 维的状态变量； $\mathbf{u}(t) \in \mathbb{R}^m$ 是 m 维的输入变量； $X \in \mathbb{R}^n$ 是 n 维的状态变量集合； $U \in \mathbb{R}^m$ 是 m 维的输入变量集合；对于 $f(\cdot, \cdot)$ ， $f(\mathbf{0}, \mathbf{0}) = \mathbf{0}$ ，也即在零状态和零输入的情况下，系统处于平衡态。

定义 $\eta(t) \in \mathbb{R}^p$ 是 p 维输出变量， N 是预测时域的大小，对于 $\forall N \in \mathbb{Z}^+$ ，考虑如下代价函数：

$$\begin{aligned}J_N(\xi(t), \mathbf{U}(t)) &= \sum_{i=1}^N \left\| \eta(t+i|t) - \eta_{ref}(t+i|t) \right\|_Q^2 + \sum_{i=1}^{N-1} \left\| \mathbf{u}(t+i|t) \right\|_R^2 \\ &\quad + \sum_{k=t}^{t+N-1} \sum_{i=1}^j \gamma_i l_i(\xi(k), \mathbf{u}(k))\end{aligned}\tag{4-2}$$

式中： $J_N(\cdot, \cdot): \mathbb{R}^n \times \mathbb{R}^{Nm} \rightarrow \mathbb{R}^+$ ， $\mathbf{U}(t) = [\mathbf{u}(t), \mathbf{u}(t+1), \dots, \mathbf{u}(t+N-1)]^T$ 是 N 个预测时域内的输入量序列， $\xi(t)$ 是当前的状态量， $\xi(t+k) (k=1, \dots, N)$ 是系统在控制输入序列 $\mathbf{U}(t)$ 作用下的各个状态量， $l(\cdot, \cdot): \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^+$ 是关于状态量和控制量的一次代价量，假设每个预测时域内有 j 个， $\gamma_i (i=1, \dots, j)$ 是各个一次代价量的权重。 $\|\mathbf{X}\|_Q^2$ 代表加权 2-范数的平方， $\mathbf{X}$ 是列向量时它的计算式为 $\|\mathbf{X}\|_Q^2 = \mathbf{X}^T \mathbf{Q} \mathbf{X}$ 。

在每个采样周期内，预测时域内的优化问题可描述如下：

$$\begin{aligned}\min_{\mathbf{U}_t, \xi_{t+1,t}, \dots, \xi_{t+N,t}} \quad & \mathbf{J}_N(\xi(t), \mathbf{U}(t)) = \sum_{i=1}^N \left\| \eta(t+i|t) - \eta_{ref}(t+i|t) \right\|_Q^2 + \sum_{i=1}^{N-1} \left\| \mathbf{u}(t+i|t) \right\|_R^2 \\ & + \sum_{k=t}^{t+N-1} \sum_{i=1}^j \gamma_i l_i(\xi(k), \mathbf{u}(k))\end{aligned}\tag{4-3a}$$

$$\text{s.t.} \quad \xi_{k+1,t} = f(\xi_{k,t}, \mathbf{u}_{k,t}), \quad k = t, \dots, N-1\tag{4-3b}$$

$$\xi_{k,t} \in X, \quad k = t+1, \dots, t+N\tag{4-3c}$$

$$\mathbf{u}_{k,t} \in \mathbf{U}, \quad k = t, \dots, t + N - 1 \quad (4-3d)$$

$$\xi_{t,t} = \xi(t) \quad (4-3e)$$

$$\xi_{N,t} \in \mathbf{X}_f \quad (4-3f)$$

式（4-3c）和式（4.3d）分别为状态变量和控制变量的上下限约束，（4-3e）为初始状态变量约束，（4-3f）为终端状态变量约束。

求解（4-3）所示的非线性优化问题，可以求得在当前时刻 t 下的最优控制量序列 $\mathbf{U}^*(t) = [\mathbf{u}^*(t), \mathbf{u}^*(t+1), \dots, \mathbf{u}^*(t+N-1)]^T$ 。最优状态量序列 $\xi^*(k) (k = t+1, \dots, t+N)$ 可以通过代入 $\mathbf{U}^*(t)$ 到（4-3b）求得。

非线性模型预测控制通过滚动优化的方式，反复求解（4-3），得到相应的局部路径，分析方程（4-3）可知，一共有 $N \times (m+n)$ 个待优化变量，（4-3b）有 $N \times n$ 个关于状态量的非线性约束，（4-3c）和（4-3d）分别有 $N \times n$ 和 $N \times m$ 个线性不等式约束，因此，优化问题的求解难度与状态方程的阶数以及预测时域大小等因素有关，对于阶数较高的复杂非线性系统，需要花费大量时间去求解预测优化问题，难以满足实时性的需要。

因此，为了提高求解的效率，需要对优化问题进行适当的简化，对非线性系统进行线性化处理可以大大提升求解的效率。线性化系统根据其是否随时间而变化的特点，可分为线性时不变系统（Linear Time Invariant, LTI）和线性时变系统（Linear Time Variant, LTV），[23]说明了线性时不变系统在高速下误差较大，而线性时变系统通过在当前点附近线性化，不仅提升了求解效率，而且获得了比 LTI 模型更为精确的结果。

4.3 输出量的定义

参照上一节的思路，在建立线性时变预测模型之前，需要明确输出量 $\eta(t)$ 的物理意义，本节把输出量定义为横纵向误差 e^e 、 e^l 和角速度 ω ，下面的小节将给出详细的说明。

4.3.1 参考轨迹参数化处理

在第三章中，已经得到了中心线数据和虚拟的左右边界，假设得到的数据是正确的，这些数据点一般都是很稀疏的，所以需要对这些数据进行参数化和密集化处理，定义

$\theta \in [0, L]$ 是参考路径的弧长变量, L 是路径的总长度, 参数化就是把 X 、 Y 表示成关于 θ 的函数 $X^{\text{ref}}(\theta)$ 和 $Y^{\text{ref}}(\theta)$ 。

利用三次样条插值进行密集化与参数化的处理^[24], 处理是离线进行的, 处理后的数据可以保存到一个库中, 作为固定的数据被规划算法使用。有了参数方程, 参考路径上某点相对于 X 轴的偏转角也可以求出,

$$\Phi(\theta) \triangleq \arctan \left\{ \frac{\partial Y^{\text{ref}}(\theta)}{\partial X^{\text{ref}}(\theta)} \right\} \quad (4-4)$$

三次样条插值的方法比单纯线性插值精度高, 而且实现简单, 能有效描述一组数据点之间的曲线形状, 对于路径规划和轨迹跟踪来说都是十分重要的。

4.3.2 横纵向误差的度量

横纵向误差可以评价车辆实际轨迹与参考中心线之间的偏离程度, 使用它们作为其中两个输出量, 具体定义为车辆当前点 X 、 Y 与当前点所对应的参考点 $X^{\text{ref}}(\theta)$ 、 $Y^{\text{ref}}(\theta)$ 的偏差。令 $\mathbf{P} : \mathbb{R}^2 \rightarrow [0, L]$ 是当前点坐标 X 、 Y 到最近参考点的映射, 也即,

$$\mathbf{P}(X, Y) \triangleq \arg \min_{\theta} [(X - X^{\text{ref}}(\theta))^2 + (Y - Y^{\text{ref}}(\theta))^2] \quad (4-5)$$

简便起见, 令 $\theta_p \triangleq \mathbf{P}(X, Y)$, 那么横向误差就可以定义为当前点和对应参考点之间的正交距离, 算式如下:

$$e^c(X, Y, \theta_p) \triangleq \sin(\Phi(\theta_p))(X - X^{\text{ref}}(\theta_p)) - \cos(\Phi(\theta_p))(Y - Y^{\text{ref}}(\theta_p)) \quad (4-6)$$

其中 $\Phi(\cdot)$ 代表的是参考点相对于 X 轴的偏转角, 具体算式如 (4-4) 所示。根据该定义, 这种情况下的纵向误差恒为 0, 因为已经找到车辆的对应参考点, 关于横向误差的实际物理意义如图 4-1 所示, 此时纵向误差为 0。

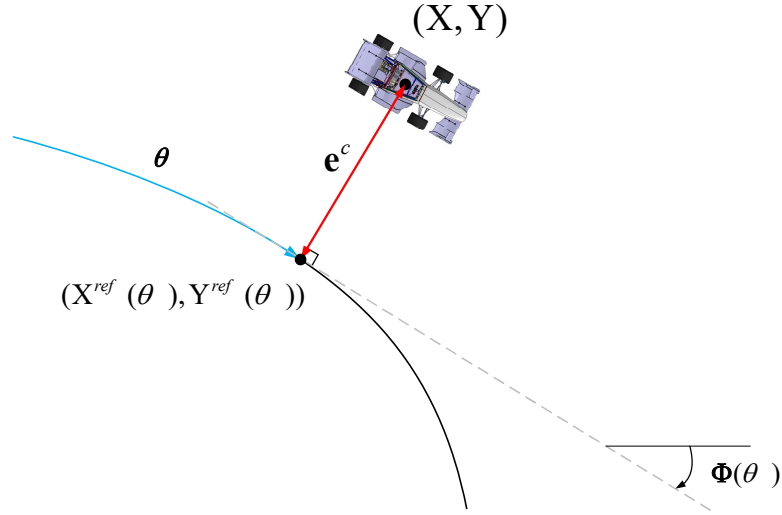


图 4-1 车辆的横向误差

对于预测模型来说，这种输出量的定义是不合适的，因为对于预测时域内每个状态点，难以计算 θ_p 的大小，更准确地说，(4-5) 无法表达成方便优化问题求解的形式，难以求解，因为它本身就是一个迭代求解的优化问题。所以，为了能够把横纵向误差表达成预测优化问题能够在线求解的形式，需要对 θ_p 进行估计，令 θ_A 是 θ_p 的估计，其中 θ_A 是由控制器决定的自变量，也即在原有状态变量 $\tilde{x}/\tilde{y}$ 的基础上，增加一维状态量 θ_A ，作为对于横纵向误差的估计。此时，纵向误差便不是恒为 0 了，它可以作为衡量估计的好坏程度，由 θ_A 和 θ_p 的关系，可得，

$$e^l(X, Y, \theta_A) \triangleq |\theta_A - \theta_p| \quad (4-7)$$

然而，纵向误差的这个定义仍然免不了要计算 (4-5) 的迭代优化问题，为了能够更加简单、明确地表达横纵向误差，可以使用它们的近似值表示，计算公式如下，

$$e^c \approx \hat{e}^c(X, Y, \theta_A) \triangleq \sin(\Phi(\theta_A))(X - X^{\text{ref}}(\theta_A)) - \cos(\Phi(\theta_A))(Y - Y^{\text{ref}}(\theta_A)) \quad (4-8a)$$

$$e^l \approx \hat{e}^l(X, Y, \theta_A) \triangleq -\cos(\Phi(\theta_A))(X - X^{\text{ref}}(\theta_A)) - \sin(\Phi(\theta_A))(Y - Y^{\text{ref}}(\theta_A)) \quad (4-8b)$$

它们具体的物理意义如图 4-2 所示，图中还展示了实际横纵向误差与近似横纵向误差的差异。

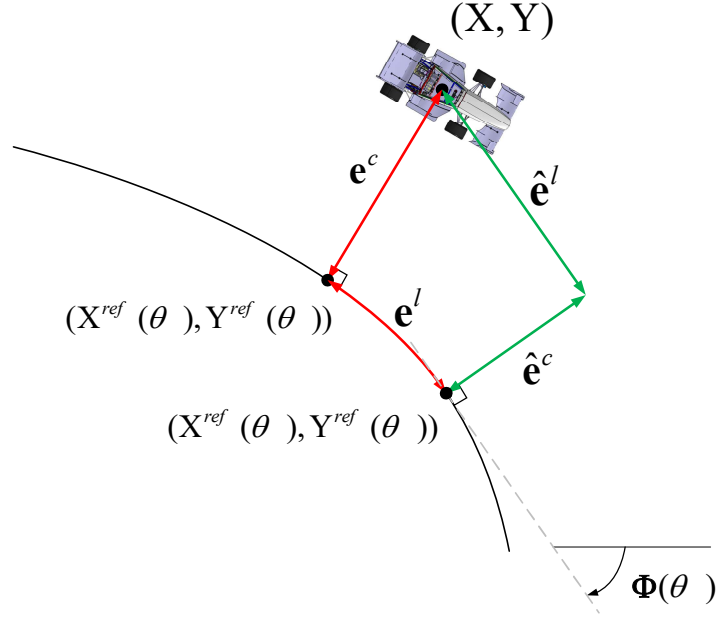


图 4-2 车辆实际横纵向误差与估计横纵向误差

从图 4-2 中也可以发现，关于（4-8）横纵向误差的计算，就是车辆位置 X 、 Y 和 $X^{\text{ref}}(\theta_A)$ 、 $Y^{\text{ref}}(\theta_A)$ 之间的距离沿道路法线和切线方向的分量。

最后还有一个问题就是 θ_A 的如何确定，作为新加入的状态量， θ_A 会受控制量的影响，因此，在原有控制量 $\tilde{u}/\tilde{\omega}$ 的基础上，增加一维控制量 V （运动学模型 $\tilde{u}$ 中增加的是增量 ΔV ），表示 θ_A 变化的速度，具体计算如下，

$$\dot{\theta}_A = V \quad (4-9)$$

V 作为待优化的控制变量存在，（4-8）和（4-9）组成了输出变量的非线性模型。

定义输出量 $\boldsymbol{\eta} = [e^c, e^l, \omega]^T$ ，根据上面的推导，便可得到输出量与状态量、控制量的非线性关系。

4.4 线性时变预测模型

线性时变系统因为其巨大优势被广泛应用，本节将介绍如何将非线性系统转化为线性时变系统，给出了详细的公式推导过程。

4.4.1 动力学模型的线性化和离散化处理

第二章已经建立了车辆动力学自行车模型，本小节将推导如何对模型进行简化，以减小求解时间。

与 4.2 的定义类似，这里假设 $\xi \in \mathbb{R}^n$ 是 n 维状态变量， $u \in \mathbb{R}^m$ 是 m 维的输入变量。定义 $\xi = [X, Y, \varphi, v_x, v_y, \omega, \theta_A]^T$ 为系统的状态量， $u = [d, \delta, V]^T$ 为系统的控制输入量，系统方程 (2-1a) ~ (2-1f) 和 (4-9) 在 r 时刻可表示成如下形式，

$$\dot{\xi}_r = f(\xi_r, u_r) \quad (4-10)$$

首先需要对模型进行线性化处理，线性化处理就是将系统的微分方程 (4-10) 在点 (ξ_r, u_r) 处进行一阶近似泰勒展开，得到 k 时刻系统方程的近似：

$$\dot{\xi}_k = f(\xi_r, u_r) + \left. \frac{\partial f}{\partial \xi} \right|_{\xi_r, u_r} (\xi_k - \xi_r) + \left. \frac{\partial f}{\partial u} \right|_{\xi_r, u_r} (u_k - u_r) \quad (4-11)$$

其中， $\left. \frac{\partial f}{\partial \xi} \right|_{\xi_r, u_r}$ 和 $\left. \frac{\partial f}{\partial u} \right|_{\xi_r, u_r}$ 分别是 f 对 ξ 和 u 的 Jacobi 矩阵，计算如下，

$$\left. \frac{\partial f}{\partial \xi} \right|_{\xi_r, u_r} = \left[\left. \frac{\partial f}{\partial X} \right|_{\xi_r, u_r}, \left. \frac{\partial f}{\partial Y} \right|_{\xi_r, u_r}, \left. \frac{\partial f}{\partial \varphi} \right|_{\xi_r, u_r}, \left. \frac{\partial f}{\partial v_x} \right|_{\xi_r, u_r}, \left. \frac{\partial f}{\partial v_y} \right|_{\xi_r, u_r}, \left. \frac{\partial f}{\partial \omega} \right|_{\xi_r, u_r}, \left. \frac{\partial f}{\partial \theta_A} \right|_{\xi_r, u_r} \right] \quad (4-12)$$

$$\left. \frac{\partial f}{\partial u} \right|_{\xi_r, u_r} = \left[\left. \frac{\partial f}{\partial d} \right|_{\xi_r, u_r}, \left. \frac{\partial f}{\partial \delta} \right|_{\xi_r, u_r}, \left. \frac{\partial f}{\partial V} \right|_{\xi_r, u_r} \right] \quad (4-13)$$

令 $A = \left. \frac{\partial f}{\partial \xi} \right|_{\xi_r, u_r}$ ， $B = \left. \frac{\partial f}{\partial u} \right|_{\xi_r, u_r}$ ，则 (4-11) 可写成如下形式，

$$\dot{\xi}_k = A\xi_k + Bu_k + g \quad (4-14)$$

式中： $g = f(\xi_r, u_r) - A\xi_r - Bu_r$ 。如此，便完成了模型的线性化处理。

下一步是对模型进行离散化处理，所用的方法与[25]的类似，以预测时域内的采样时间 T_s 为步长，使用 0 阶保持 (ZOH) 进行离散化，首先设计增广向量 $z_s = [\xi, u, I]^T$ ，其中， I 是单位矩阵，根据 (4-14)，有，

$$\begin{bmatrix} \dot{\xi}_k \\ \dot{\mathbf{u}}_k \\ I \end{bmatrix} = \begin{bmatrix} \mathbf{A} & \mathbf{B} & \mathbf{g} \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \xi_k \\ \mathbf{u}_k \\ I \end{bmatrix} = \mathbf{G}_s \mathbf{z}_s(k) \quad (4-15)$$

式 (4-15) 中, 通过对 $\mathbf{G}_s T_s$ 进行矩阵指数运算, 可得,

$$\begin{bmatrix} \xi(k+1) \\ \mathbf{u}(k+1) \\ I(k+1) \end{bmatrix} = \begin{bmatrix} \mathbf{A}_{k,t} & \mathbf{B}_{k,t} & \mathbf{g}_{k,t} \\ 0 & I & 0 \\ 0 & 0 & I \end{bmatrix} \begin{bmatrix} \xi(k) \\ \mathbf{u}(k) \\ I(k) \end{bmatrix} \quad (4-16)$$

从上述结果中可知,

$$\xi(k+1) = \mathbf{A}_{k,t} \xi(k) + \mathbf{B}_{k,t} \mathbf{u}(k) + \mathbf{g}_{k,t} \quad (4-17)$$

式中: $\xi(k)$ 是 $\xi(k|t)$ 的简写, 代表 t 时刻, 预测时域内第 $k(k=t, t+1, \dots, t+N)$ 个时刻的状态量。 $\mathbf{u}(k)$ 是 $\mathbf{u}(k|t)$ 的简写, 代表 t 时刻, 预测时域内第 $k(k=t, t+1, \dots, t+N)$ 个时刻输入的控制量。 $\mathbf{A}_{k,t}$ 、 $\mathbf{B}_{k,t}$ 和 $\mathbf{g}_{k,t}$ 分别是 t 时刻, 预测时域内第 $k(k=t, t+1, \dots, t+N)$ 个时刻的转移矩阵。式 (4-17) 就是车辆动力学模型的线性时变预测模型。

然而, 上述模型在相邻采样周期内, 由于每组控制量之间没有相互耦合的关系, 可能会出现控制量在某几个预测时刻来回突变的情况, 然而实际应用中, 输入需要响应时间, 如果输入突变的控制量一定会来不及响应, 所以为了使模型更加符合实际, 考虑将控制量 $\mathbf{u}(k)$ 改成控制增量 $\Delta \mathbf{u}(k)$ 的形式, 假设

$$\mathbf{u}(k) = \mathbf{u}(k-1) + \Delta \mathbf{u}(k) \quad (4-18)$$

代入到 (4-17), 得

$$\xi(k+1) = \mathbf{A}_{k,t} \xi(k) + \mathbf{B}_{k,t} \mathbf{u}(k-1) + \mathbf{B}_{k,t} \Delta \mathbf{u}(k) + \mathbf{g}_{k,t} \quad (4-19)$$

对状态量进行增广, 即令

$$\tilde{\xi}(k|t) = \begin{pmatrix} \xi(k|t) \\ \mathbf{u}(k-1|t) \end{pmatrix} \quad (4-20)$$

再定义新的状态转移矩阵,

$$\tilde{\mathbf{A}}_{k,t} = \begin{pmatrix} \mathbf{A}_{k,t} & \mathbf{B}_{k,t} \\ \mathbf{0}_{m \times n} & \mathbf{I}_m \end{pmatrix}$$

$$\tilde{\mathbf{B}}_{k,t} = \begin{pmatrix} \mathbf{B}_{k,t} \\ \mathbf{I}_m \end{pmatrix} \quad (4-21)$$

$$\tilde{\mathbf{g}}(k|t) = \begin{pmatrix} \mathbf{g}(k|t) \\ \mathbf{0}_m \end{pmatrix}$$

根据 (4-19), 可知

$$\tilde{\xi}(k+1|t) = \tilde{\mathbf{A}}_{k,t} \tilde{\xi}(k|t) + \tilde{\mathbf{B}}_{k,t} \Delta \mathbf{u}(k|t) + \tilde{\mathbf{g}}_{k,t} \quad (4-22)$$

(4-22) 就是最终车辆动力学模型的线性时变预测模型, 根据该模型就可以通过控制输入增量来预测整个预测时域内车辆的状态。

4.4.2 运动学模型的线性化和离散化处理

第二章中介绍了车辆的运动学模型以及使用的原因, 本章会对运动学模型进行进一步处理, 使状态量与控制量与动力学模型统一。

为了与 4.4.1 节兼容, 运动学模型使用的状态量定义为 $\tilde{\xi} = [X, Y, \phi, v_x, v_y, \omega, \theta_A, d, \delta, V]^T$, 控制量为 $\Delta \mathbf{u} = [\Delta d, \Delta \delta, \Delta V]^T$, 根据控制量与控制增量的关系可知

$$\begin{aligned} d(k+1) &= d(k) + \Delta d(k) \\ \delta(k+1) &= \delta(k) + \Delta \delta(k) \\ V(k+1) &= V(k) + \Delta V(k) \end{aligned} \quad (4-23)$$

也即,

$$\begin{aligned} \dot{d} &= \frac{d(k+1) - d(k)}{T_s} = \frac{\Delta d}{T_s} \\ \dot{\delta} &= \frac{\delta(k+1) - \delta(k)}{T_s} = \frac{\Delta \delta}{T_s} \\ \dot{V} &= \frac{V(k+1) - V(k)}{T_s} = \frac{\Delta V}{T_s} \end{aligned} \quad (4-24)$$

综合 (2-20a) ~ (2-20f)、(4-9)、(4-24), r 时刻的方程可表示成下面的一般形式,

$$\dot{\tilde{\xi}}_r = f(\tilde{\xi}_r, \Delta \mathbf{u}_r) \quad (4-25)$$

与 4.4.1 类似，定义 $\tilde{A} = \frac{\partial f}{\partial \tilde{\xi}} \bigg|_{\tilde{\xi}_r, \Delta \mathbf{u}_r}$ ， $\tilde{B} = \frac{\partial f}{\partial \Delta \mathbf{u}} \bigg|_{\tilde{\xi}_r, \Delta \mathbf{u}_r}$ 分别是 f 对 $\tilde{\xi}$ 和 $\Delta \mathbf{u}$ 的 Jacobi 矩阵，再

用与 (4-15)、(4-16) 类似的零阶保持方法进行离散化，就得到了车辆运动学的线性时变预测模型，

$$\tilde{\xi}(k+1|t) = \tilde{\mathbf{A}}_{(kinetic)k,t} \tilde{\xi}(k|t) + \tilde{\mathbf{B}}_{(kinetic)k,t} \Delta \mathbf{u}(k|t) + \tilde{\mathbf{g}}_{(kinetic)k,t} \quad (4-26)$$

式中： $\tilde{\mathbf{A}}_{(kinetic)k,t}$ 、 $\tilde{\mathbf{B}}_{(kinetic)k,t}$ 、 $\tilde{\mathbf{g}}_{(kinetic)k,t}$ 是运动学模型的状态转移矩阵。

4.4.3 车辆运动学与动力学模型的结合

推导完动力学和运动学线性时变预测模型之后，就要在这两个模型间进行权衡选择，由 4.4.1 与 4.4.2 可知，两个模型已经统一了状态量与控制量，即 $\tilde{\xi} = [X, Y, \varphi, v_x, v_y, \omega, \theta_A, d, \delta, V]^T$ ， $\Delta \mathbf{u} = [\Delta d, \Delta \delta, \Delta V]^T$ 。根据 (4-22) 与 (4-26)，将两个模型线性组合在一起，得，

$$\begin{aligned} \tilde{\xi}(k+1|t) = & \lambda(\tilde{\mathbf{A}}_{(dynamic)k,t} \tilde{\xi}(k|t) + \tilde{\mathbf{B}}_{(dynamic)k,t} \Delta \mathbf{u}(k|t) + \tilde{\mathbf{g}}_{(dynamic)k,t}) \\ & + (1-\lambda)(\tilde{\mathbf{A}}_{(kinetic)k,t} \tilde{\xi}(k|t) + \tilde{\mathbf{B}}_{(kinetic)k,t} \Delta \mathbf{u}(k|t) + \tilde{\mathbf{g}}_{(kinetic)k,t}) \end{aligned} \quad (4-27)$$

其中， λ 是与当前车速有关的系数，计算方式如下，

$$\lambda = \min \left(\max \left(\frac{v_x - v_{x,blend \min}}{v_{x,blend \max} - v_{x,blend \min}}, 0 \right), 1 \right) \quad (4-28)$$

也即当当前车速 $v_x < v_{x,blend \min}$ 时， $\lambda = 0$ ，状态完全按照车辆运动学模型进行更新，当车速 $v_x > v_{x,blend \max}$ 时， $\lambda = 1$ ，状态完全按照车辆动力学模型进行更新，车速在 $v_{x,blend \min}$ 和 $v_{x,blend \max}$ 之间时，将使用二者的混合模型。本文选取 $v_{x,blend \min} = 3m/s$ ， $v_{x,blend \max} = 5m/s$ 。

4.4.4 输出量的线性化处理

根据前面小节，输出量表示成如下形式，

$$\boldsymbol{\eta} = \begin{bmatrix} e^c \\ e^l \\ \omega \end{bmatrix} = \begin{bmatrix} \sin(\Phi(\theta_A))(X - X^{\text{ref}}(\theta_A)) - \cos(\Phi(\theta_A))(Y - Y^{\text{ref}}(\theta_A)) \\ -\cos(\Phi(\theta_A))(X - X^{\text{ref}}(\theta_A)) - \sin(\Phi(\theta_A))(Y - Y^{\text{ref}}(\theta_A)) \\ \omega \end{bmatrix} = f(\tilde{\xi}) \quad (4-29)$$

由于输出量是非线性的，不符合 QP 问题的特性，所以需要输出量进行线性化处理。

假设 k 时刻的输出量为 η_k ，将方程 (4-29) 在 $\tilde{\xi}_r$ 处进行一阶泰勒展开，得

$$\eta_k = f(\tilde{\xi}_r) + \left. \frac{\partial f}{\partial \tilde{\xi}} \right|_{\tilde{\xi}_r} (\tilde{\xi}_k - \tilde{\xi}_r) \quad (4-30)$$

$\left. \frac{\partial f}{\partial \tilde{\xi}} \right|_{\tilde{\xi}_r}$ 的计算如下，

$$\left. \frac{\partial f}{\partial \tilde{\xi}} \right|_{\tilde{\xi}_r} = \begin{bmatrix} \frac{\partial e^c}{\partial \tilde{\xi}} \\ \frac{\partial e^l}{\partial \tilde{\xi}} \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (4-31)$$

$\frac{\partial e^c}{\partial \tilde{\xi}}$ 、 $\frac{\partial e^l}{\partial \tilde{\xi}}$ 的计算如下，

$$\frac{\partial e^c}{\partial \tilde{\xi}} = [\sin(\Phi(\theta_A)) \quad -\cos(\Phi(\theta_A)) \quad 0 \quad 0 \quad 0 \quad 0 \quad \frac{\partial e^c}{\partial \theta_A} \quad 0 \quad 0 \quad 0] \quad (4-32)$$

$$\frac{\partial e^l}{\partial \tilde{\xi}} = [-\cos(\Phi(\theta_A)) \quad -\sin(\Phi(\theta_A)) \quad 0 \quad 0 \quad 0 \quad 0 \quad \frac{\partial e^l}{\partial \theta_A} \quad 0 \quad 0 \quad 0] \quad (4-33)$$

误差对 θ_A 的导数如下，

$$\frac{\partial e^c}{\partial \theta_A} = \left[\frac{d\Phi(\theta_A)}{d\theta_A} \quad 1 \right] \begin{bmatrix} X - X^{\text{ref}}(\theta_A) & Y - Y^{\text{ref}}(\theta_A) \\ \frac{dY^{\text{ref}}(\theta_A)}{d\theta_A} & -\frac{dX^{\text{ref}}(\theta_A)}{d\theta_A} \end{bmatrix} \begin{bmatrix} \cos(\theta_A) \\ \sin(\theta_A) \end{bmatrix} \quad (4-34)$$

$$\frac{\partial e^l}{\partial \theta_A} = \left[\frac{d\Phi(\theta_A)}{d\theta_A} \quad 1 \right] \begin{bmatrix} -(Y - Y^{\text{ref}}(\theta_A)) & X - X^{\text{ref}}(\theta_A) \\ \frac{dX^{\text{ref}}(\theta_A)}{d\theta_A} & \frac{dY^{\text{ref}}(\theta_A)}{d\theta_A} \end{bmatrix} \begin{bmatrix} \cos(\theta_A) \\ \sin(\theta_A) \end{bmatrix} \quad (4-35)$$

式中： $\frac{dX(\theta_A)}{d\theta_A}$ 和 $\frac{dY(\theta_A)}{d\theta_A}$ 可以通过三次样条插值得到， $\frac{d\Phi(\theta_A)}{d\theta_A}$ 的计算如下，

$$\frac{d\Phi(\theta_A)}{d\theta_A} = \frac{d\left(\frac{dY^{\text{ref}}(\theta_A)}{dX^{\text{ref}}(\theta_A)}\right)}{d\theta_A} = \frac{\frac{d^2 Y^{\text{ref}}(\theta_A)}{d\theta_A^2} \frac{dX^{\text{ref}}(\theta_A)}{d\theta_A} - \frac{d^2 X^{\text{ref}}(\theta_A)}{d\theta_A^2} \frac{dY^{\text{ref}}(\theta_A)}{d\theta_A}}{\left(\frac{dX^{\text{ref}}(\theta_A)}{d\theta_A}\right)^2} \quad (4-36)$$

综合 (4-31) ~ (4-36) 即可把 $\frac{\partial f}{\partial \tilde{\xi}} \bigg|_{\tilde{\xi}_r}$ 求得。令 $\tilde{C}_r = \frac{\partial f}{\partial \tilde{\xi}} \bigg|_{\tilde{\xi}_r}$ ， $\tilde{e}_r = f(\tilde{\xi}_r) - \frac{\partial f}{\partial \tilde{\xi}} \bigg|_{\tilde{\xi}_r} \tilde{\xi}_r$ ，则

$\tilde{C}_{k,t} = \frac{\partial f}{\partial \tilde{\xi}} \bigg|_{\tilde{\xi}_{k,t}}$ ， $\tilde{e}_{k,t} = f(\tilde{\xi}_{k,t}) - \frac{\partial f}{\partial \tilde{\xi}} \bigg|_{\tilde{\xi}_{k,t}} \tilde{\xi}_{k,t}$ ，如此便得：

$$\eta(k|t) = \tilde{C}_{k,t} \tilde{\xi}(k|t) + \tilde{e}_{k,t} \quad (4-37)$$

4.4.5 线性时变预测模型的整合

经过上面小节的处理，已经得到了以下线性时变系统，

$$\tilde{\xi}(k+1|t) = \tilde{\mathbf{A}}_{k,t} \tilde{\xi}(k|t) + \tilde{\mathbf{B}}_{k,t} \Delta \mathbf{u}(k|t) + \tilde{\mathbf{g}}_{k,t} \quad (4-38)$$

$$\eta(k|t) = \tilde{C}_{k,t} \tilde{\xi}(k|t) + \tilde{e}_{k,t} \quad (4-39)$$

为了能够让计算机更方便地处理，需要把它整合成矩阵的形式。从 (4-38) 中可知， $\tilde{\xi}(k+1|t)$ 与 $\tilde{\xi}(k|t)$ 有关，而 $\tilde{\xi}(k|t)$ 与 $\tilde{\xi}(k-1|t)$ 有关，因此可以通过不断迭代 (4-38)，一直迭代到 $\tilde{\xi}(t|t)$ ，也即当前状态，便可以得到 k 时刻的系统状态量，

$$\tilde{\xi}(k|t) = \prod_{i=t}^{k-1} \tilde{\mathbf{A}}_{i,t} \tilde{\xi}(t|t) + \sum_{i=t}^{k-1} \prod_{j=i+1}^{k-1} \tilde{\mathbf{A}}_{j,t} [\tilde{\mathbf{B}}_{i,t} \Delta \mathbf{u}(i|t) + \tilde{\mathbf{g}}_{i,t}] \quad (4-40)$$

假设控制时域与预测时域相等，均为 N ，则在预测时域 N 内的状态量可用如下公式计算：

$$\mathbf{X}(t) = \Psi_{(state)t} \tilde{\xi}(t|t) + \Theta_{(state)t} \Delta \mathbf{U}(t) + \Gamma_{(state)t} \Phi(t) \quad (4-41)$$

其中：

$$\mathbf{X}(t) = \begin{bmatrix} \tilde{\xi}(t+1|t) \\ \tilde{\xi}(t+2|t) \\ \vdots \\ \tilde{\xi}(t+H|t) \end{bmatrix} \quad (4-42a)$$

$$\Delta \mathbf{U}(t) = \begin{bmatrix} \Delta \mathbf{u}(t+1|t) \\ \Delta \mathbf{u}(t+2|t) \\ \vdots \\ \Delta \mathbf{u}(t+H-1|t) \end{bmatrix} \quad (4-42b)$$

$$\Psi_{(state)t} = \begin{bmatrix} \tilde{\mathbf{A}}_{t,t} \\ \tilde{\mathbf{A}}_{t+1,t} \tilde{\mathbf{A}}_{t,t} \\ \vdots \\ \prod_{i=t}^{t+H-1} \tilde{\mathbf{A}}_{i,t} \end{bmatrix} \quad (4-42c)$$

$$\Theta_{(state)t} = \begin{bmatrix} \tilde{\mathbf{B}}_{t,t} & \mathbf{0}_{10 \times 3} & \cdots & \mathbf{0}_{10 \times 3} \\ \tilde{\mathbf{A}}_{t+1,t} \tilde{\mathbf{B}}_{t,t} & \tilde{\mathbf{B}}_{t+1,t} & \cdots & \mathbf{0}_{10 \times 3} \\ \vdots & \vdots & \ddots & \vdots \\ \prod_{i=t+1}^{t+H-1} \tilde{\mathbf{A}}_{i,t} \tilde{\mathbf{B}}_{t,t} & \prod_{i=t+2}^{t+H-1} \tilde{\mathbf{A}}_{i,t} \tilde{\mathbf{B}}_{t+1,t} & \cdots & \prod_{i=t+H}^{t+H-1} \tilde{\mathbf{A}}_{i,t} \tilde{\mathbf{B}}_{t+H-1,t} \end{bmatrix} \quad (4-42d)$$

$$\Gamma_{(state)t} = \begin{bmatrix} \mathbf{I}_{10 \times 10} & \mathbf{0}_{10 \times 10} & \cdots & \mathbf{0}_{10 \times 10} \\ \tilde{\mathbf{A}}_{t+1,t} & \mathbf{I}_{10 \times 10} & \cdots & \mathbf{0}_{10 \times 10} \\ \vdots & \vdots & \ddots & \vdots \\ \prod_{i=t+1}^{t+H-1} \tilde{\mathbf{A}}_{i,t} & \prod_{i=t+2}^{t+H-1} \tilde{\mathbf{A}}_{i,t} & \cdots & \mathbf{I}_{10 \times 10} \end{bmatrix} \quad (4-42e)$$

$$\Phi(t) = \begin{bmatrix} \tilde{\mathbf{g}}_{t,t} \\ \tilde{\mathbf{g}}_{t+1,t} \\ \vdots \\ \tilde{\mathbf{g}}_{t+H-1,t} \end{bmatrix} \quad (4-42f)$$

式中： $\mathbf{X}(t) \in \mathbb{R}^{10N}$, $\Delta \mathbf{U}(t) \in \mathbb{R}^{3N}$, $\Psi_{(state)t} \in \mathbb{R}^{10N}$, $\Theta_{(state)t} \in \mathbb{R}^{10N \times 3N}$, $\Gamma_{(state)t} \in \mathbb{R}^{10N \times 10N}$,

$\Phi(t) \in \mathbb{R}^{10N}$ 。

再根据输出量与状态量的关系（4-39），通过不断迭代，也可得到如下公式：

$$\eta(k|t) = \tilde{\mathbf{C}}_{k,t} \prod_{i=t}^{k-1} \tilde{\mathbf{A}}_{i,t} \tilde{\boldsymbol{\xi}}(t|t) + \tilde{\mathbf{C}}_{k,t} \sum_{i=t}^{k-1} \prod_{j=i+1}^{k-1} \tilde{\mathbf{A}}_{j,t} \left[\tilde{\mathbf{B}}_{i,t} \Delta \mathbf{u}(i|t) + \tilde{\mathbf{g}}_{i,t} \right] + \tilde{\mathbf{e}}_{k,t} \quad (4-43)$$

同样，（4-43）可写成如下的矩阵形式，

$$\mathbf{Y}(t) = \Psi_{(out)t} \tilde{\boldsymbol{\xi}}(t|t) + \Theta_{(out)t} \Delta \mathbf{U}(t) + \Gamma_{(out)t} \Phi(t) + \Lambda(t) \quad (4-44)$$

其中：

$$\mathbf{Y}(t) = \begin{bmatrix} \eta(t+1|t) \\ \eta(t+2|t) \\ \vdots \\ \eta(t+H|t) \end{bmatrix} \quad (4-45a)$$

$$\Delta U(t) = \begin{bmatrix} \Delta \mathbf{u}(t+1|t) \\ \Delta \mathbf{u}(t+2|t) \\ \vdots \\ \Delta \mathbf{u}(t+H-1|t) \end{bmatrix} \quad (4-45b)$$

$$\Psi_{(out)t} = \begin{bmatrix} \tilde{\mathbf{C}}_{t+1,t} \tilde{\mathbf{A}}_{t,t} \\ \tilde{\mathbf{C}}_{t+2,t} \tilde{\mathbf{A}}_{t+1,t} \tilde{\mathbf{A}}_{t,t} \\ \vdots \\ \tilde{\mathbf{C}}_{t+H,t} \prod_{i=t}^{t+H-1} \tilde{\mathbf{A}}_{i,t} \end{bmatrix} \quad (4-45c)$$

$$\Theta_{(out)t} = \begin{bmatrix} \tilde{\mathbf{C}}_{t+1,t} \tilde{\mathbf{B}}_{t,t} & \mathbf{0}_{3 \times 3} & \cdots & \mathbf{0}_{3 \times 3} \\ \tilde{\mathbf{C}}_{t+2,t} \tilde{\mathbf{A}}_{t+1,t} \tilde{\mathbf{B}}_{t,t} & \tilde{\mathbf{C}}_{t+2,t} \tilde{\mathbf{B}}_{t+1,t} & \cdots & \mathbf{0}_{3 \times 3} \\ \vdots & \vdots & \ddots & \vdots \\ \tilde{\mathbf{C}}_{t+N,t} \prod_{i=t+1}^{t+H-1} \tilde{\mathbf{A}}_{i,t} \tilde{\mathbf{B}}_{t,t} & \tilde{\mathbf{C}}_{t+N,t} \prod_{i=t+2}^{t+H-1} \tilde{\mathbf{A}}_{i,t} \tilde{\mathbf{B}}_{t+1,t} & \cdots & \tilde{\mathbf{C}}_{t+N,t} \prod_{i=t+H}^{t+H-1} \tilde{\mathbf{A}}_{i,t} \tilde{\mathbf{B}}_{t+H-1,t} \end{bmatrix} \quad (4-45d)$$

$$\Gamma_{(out)t} = \begin{bmatrix} \tilde{\mathbf{C}}_{t+1,t} & \mathbf{0}_{3 \times 10} & \cdots & \mathbf{0}_{3 \times 10} \\ \tilde{\mathbf{C}}_{t+2,t} \tilde{\mathbf{A}}_{t+1,t} & \tilde{\mathbf{C}}_{t+2,t} & \cdots & \mathbf{0}_{3 \times 10} \\ \vdots & \vdots & \ddots & \vdots \\ \tilde{\mathbf{C}}_{t+N,t} \prod_{i=t+1}^{t+H-1} \tilde{\mathbf{A}}_{i,t} & \tilde{\mathbf{C}}_{t+N,t} \prod_{i=t+2}^{t+H-1} \tilde{\mathbf{A}}_{i,t} & \cdots & \tilde{\mathbf{C}}_{t+N,t} \end{bmatrix} \quad (4-45e)$$

$$\Phi(t) = \begin{bmatrix} \tilde{\mathbf{g}}_{t,t} \\ \tilde{\mathbf{g}}_{t+1,t} \\ \vdots \\ \tilde{\mathbf{g}}_{t+H-1,t} \end{bmatrix} \quad (4-45f)$$

$$\Lambda(t) = \begin{bmatrix} \tilde{\mathbf{e}}_{t+1,t} \\ \vdots \\ \tilde{\mathbf{e}}_{t+N,t} \end{bmatrix} \quad (4-45g)$$

式中: $\mathbf{Y}(t) \in \mathbb{R}^{3N}$, $\Delta \mathbf{U}(t) \in \mathbb{R}^{3N}$, $\Psi_{(out)t} \in \mathbb{R}^{3N}$, $\Theta_{(out)t} \in \mathbb{R}^{3N \times 3N}$, $\Gamma_{(out)t} \in \mathbb{R}^{3N \times 10N}$, $\Phi(t) \in \mathbb{R}^{10N}$,

$\Lambda(t) \in \mathbb{R}^{3N}$ 。

(4-41) 与 (4-44) 便是线性时变预测模型的矩阵表示, 若已知当前状态 $\tilde{\xi}(t|t)$, 可通过不同的输入量序列 $\Delta \mathbf{U}(t)$ 来获得不同的状态量与输出量, 达到预测的目的。

4.5 线性时变模型预测局部优化路径生成器

本节利用最优化理论，设计了线性时变模型预测局部优化路径生成器。首先设计合理的目标函数和约束，并用上节推导的线性时变预测模型把带约束的优化问题转化为二次规划问题进行求解。

4.5.1 目标函数的设计

预测时域 N 内的各种状态受当前状态 $\tilde{\xi}(t|t)$ 和输入量序列 $\Delta U(t)$ 控制，其中当前状态是可以通过定位层获取的，是无法改变的，而 $\Delta U(t)$ 则可以有无数种选择，因此设计合理的代价函数并使其最小化，可以达到对 $\Delta U(t)$ 的挑选，并产生最优控制量序列 $\Delta U^*(t)$ 和相应的最优状态量序列 $X^*(t)$ 。

考虑以下目标函数，

$$J = \sum_{k=1}^{N-1} (\|e_k^c(X_k, Y_k, \theta_{A,k})\|_{q_c}^2 + \|e_k^l(X_k, Y_k, \theta_{A,k})\|_{q_l}^2 + \|\omega_k\|_{q_\omega}^2) + \sum_{k=1}^N (-\gamma V_k + \|\Delta u_k\|_{R_u}^2) + \|e_N^c(X_N, Y_N, \theta_{A,N})\|_{q_{cN}}^2 + \|e_N^l(X_N, Y_N, \theta_{A,N})\|_{q_{lN}}^2 + \|\omega_N\|_{q_{\omega N}}^2 \quad (4-46)$$

这样的定义是合理的，在车辆行驶过程中，目标函数中横纵向误差的惩罚会关系到车辆行驶轨迹与中心线的偏差程度； ω 的惩罚是要防止车辆轨迹蜿蜒曲折； V 的一次项惩罚是为了能够让车辆在预测时域内行驶得尽可能远，虽然 V 只是与泰勒展开点的估计量 θ_A 有关，但是可以假设估计 θ_A 是准确的，那么在预测时域内， V 越大， θ_A 也越大，车辆将行驶得越远，此处行驶的远近定义为车辆从当前点运动到预测时域的终点后，相对于中心线行驶过的弧长；为了防止控制量出现突变的情况，对控制增量也给予了惩罚，优先选择控制增量小的轨迹；最后要注意，预测时域终点处 e^c 、 e^l 、 ω 的权重与预测时域内其他时刻的稍有不同，表示在目标函数最后 3 项上。

关于各个惩罚量权重的选择， q_c 可以取较小的值，因为车辆行驶过程中不一定要沿着中心线行驶，相对于完全沿着中心线，车辆走得越远对于无人驾驶赛车来说更为重要，所以 q_c 不需要取太大的值，而预测时域终点处的 q_{cN} 应该取稍大的值，因为终点处离中

心线较近会对算法更有利； q_l 则应该取较大的值，从 e' 的定义和图 4-2 可以看出 e' 对于估计量 θ_A 来说是至关重要的，关系到估计量是否准确，如果整个预测时域内 e' 都很小，那么可以认为车辆的估计量 θ_A 与实际量 θ_p 相等，估计是准确的，同理在终点处的 q_{lv} 也应该取较大的值；而 ω 的权重有多种选择，如果取值较大，轨迹较为平直，取值较小，轨迹较为弯曲光滑，但过大或过小都是不合适的，过大的话会使预测时域内 θ_A 减小，车辆的圈速时间变长，过小的话会导致轨迹蜿蜒曲折，无法跟踪，所以应该适当选择；此外 γ 、 R_u 的值也应该合理选择。

4.5.2 赛道边界约束

上一小节定义的代价函数可以有效优化无人驾驶赛车的轨迹轮廓，可以得到光滑并且行驶估计量 θ_A 较大的轨迹，从而使得赛车可以完成“切弯”的动作。除此之外，对无人驾驶赛车来说，行驶在限定的赛道内也是至关重要的，因此需要对无人赛车的行驶轨迹进行边界限定，使得车辆可以在红蓝桩桶限定的赛道内行驶。

赛道边界约束也是依靠 θ_A 进行估计的，对于预测时域内的每一个点，都对应一个 θ_A 值，也就对应一个中心线上的点 $(X^{ref}(\theta_A), Y^{ref}(\theta_A))$ ，该点分别对应左右边界上的一个点 $(X^{LBound}(\theta_A), Y^{LBound}(\theta_A))$ ， $(X^{RBound}(\theta_A), Y^{RBound}(\theta_A))$ ，它们的关系如下：

$$X^{LBound}(\theta_A) = X^{ref}(\theta_A) - \frac{w}{2} \sin(\Phi(\theta_A)) \quad (4-47a)$$

$$Y^{LBound}(\theta_A) = Y^{ref}(\theta_A) + \frac{w}{2} \cos(\Phi(\theta_A)) \quad (4-47b)$$

$$X^{RBound}(\theta_A) = X^{ref}(\theta_A) + \frac{w}{2} \sin(\Phi(\theta_A)) \quad (4-47c)$$

$$Y^{RBound}(\theta_A) = Y^{ref}(\theta_A) - \frac{w}{2} \cos(\Phi(\theta_A)) \quad (4-47d)$$

式中：上标 LBound 和 RBound 分别代表左边界(left boundary)和右边界(right boundary)； w 是赛道的等效宽度宽度，计算如下：

$$w = \mu(w_{road} - \frac{w_{car}}{2}) \quad (4-48)$$

式中： w_{road} 代表道路宽度， w_{car} 是赛车宽度， μ 是安全系数， $\mu \leq 1$ 。图 4-3 显示了各个点的具体位置。

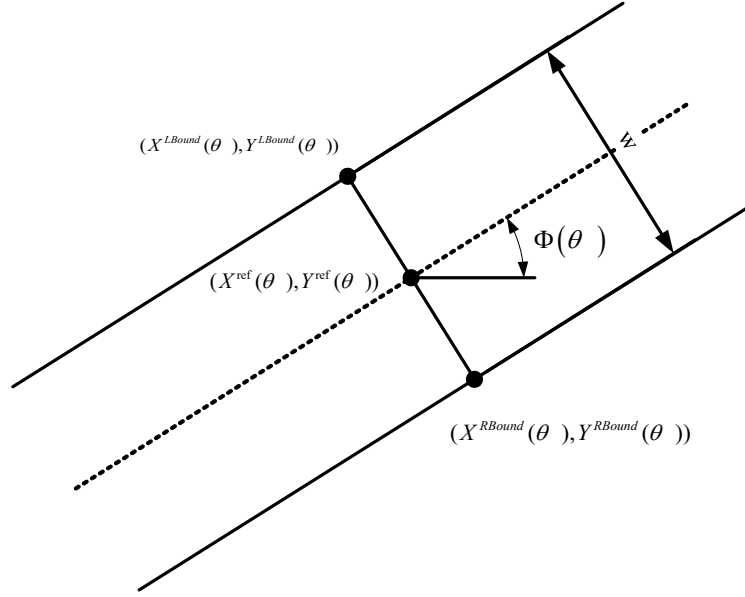


图 4-3 中心线上某点对应的左右边界点

预测时域内每个点 (X_k, Y_k) ，都对应一个 θ_{Ak} ，将它的边界约束定义为左右边界点 $(X^{LBound}(\theta_A), Y^{LBound}(\theta_A))$ 和 $(X^{RBound}(\theta_A), Y^{RBound}(\theta_A))$ 的切线组成的半空间，如图 4-4 所示的这样。其中，图 a) 中红点组成的是车辆的预测轨迹，绿点组成的是道路中心线，大红点是大蓝点对应的中心线点，对应的边界半空间约束是两条边界点的切线。图 b) 是多个预测轨迹点的情况。

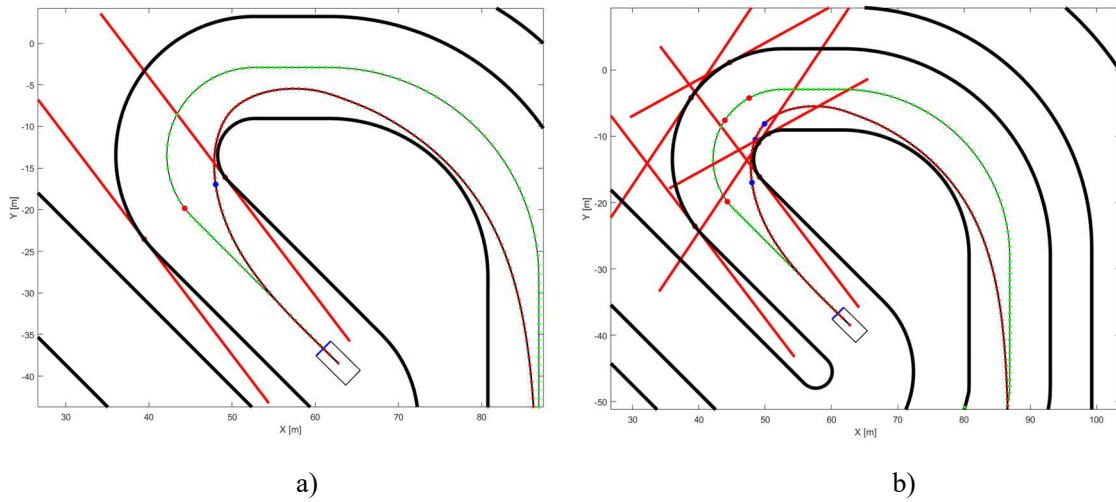


图 4-4 图 a): 一个点（大蓝点）的半空间约束。

图 b): 多个点的半空间约束

令 $\lambda_x = -(X^{LBound} - X^{RBound})$, $\lambda_y = Y^{LBound} - Y^{RBound}$ 左右边界点的切线和过预测轨迹点

(X, Y) 的切线可以表示如下:

$$\begin{aligned} b^L &= Y^{LBound} - \frac{\lambda_x}{\lambda_y} X^{LBound} = \frac{\lambda_y Y^{LBound} - \lambda_x X^{LBound}}{\lambda_y} \\ b^R &= Y^{RBound} - \frac{\lambda_x}{\lambda_y} X^{RBound} = \frac{\lambda_y Y^{RBound} - \lambda_x X^{RBound}}{\lambda_y} \\ b &= Y - \frac{\lambda_x}{\lambda_y} X = \frac{\lambda_y Y - \lambda_x X}{\lambda_y} \end{aligned} \quad (4-49)$$

要想让 (X, Y) 在约束半空间内, 必须满足:

$$\min(b^L, b^R) \leq b \leq \max(b^L, b^R) \quad (4-50)$$

假设这个点是预测时域的第 k 个状态点 ($k = 1, 2, \dots, N$), 上式可以写成:

$$f_k^{low} \leq F_k \tilde{\xi}(t+k|t) \leq f_k^{up} \quad (4-51)$$

式中:

$$F_k = [-\lambda_{x,k}, \lambda_{y,k}, \mathbf{0}_{1 \times 8}] \quad (4-52a)$$

$$f_k^{low} = \min(b_k^L, b_k^R) \quad (4-52b)$$

$$f_k^{up} = \max(b_k^L, b_k^R) \quad (4-52c)$$

4.5.3 QP 问题转化

为了方便计算机的编程实现, 需要把上述的预测优化问题转化为标准二次规划(QP)问题, 前面章节已经设计了目标函数和边界约束, 此外还有对于控制量和状态量的约束; 为了使得问题有效求解, 把边界约束设计为软约束, 也即加入了松弛因子, 允许车辆稍微驶出边界。综上, 优化问题有如下形式,

$$\begin{aligned} \min_{\Delta U(t), \varepsilon} \quad & \sum_{k=1}^{N-1} (\|e_k^c(X_k, Y_k, \theta_{A,k})\|_{q_c}^2 + \|e_k^l(X_k, Y_k, \theta_{A,k})\|_{q_l}^2 + \|\omega_k\|_{q_w}^2) + \sum_{k=1}^N (-\gamma V_k + \|\Delta u_k\|_{R_u}^2) \\ & + \|e_N^c(X_N, Y_N, \theta_{A,N})\|_{q_{cN}}^2 + \|e_N^l(X_N, Y_N, \theta_{A,N})\|_{q_{lN}}^2 + \|\omega_N\|_{q_{wN}}^2 + \rho \varepsilon^2 \end{aligned} \quad (4-53a)$$

$$\text{s.t. } \tilde{\xi}(t|t) = \tilde{\xi} \quad (4-53b)$$

$$\tilde{\xi}(k+1|t) = \tilde{\mathbf{A}}_{k,t} \tilde{\xi}(k|t) + \tilde{\mathbf{B}}_{k,t} \Delta \mathbf{u}(k|t) + \tilde{\mathbf{g}}_{k,t}, \quad k = t, t+1, \dots, t+N-1 \quad (4-53c)$$

$$f_k^{low} - \varepsilon \leq F_k \tilde{\xi}(k|t) \leq f_k^{up} + \varepsilon, \quad k = t, t+1, \dots, t+N-1 \quad (4-53d)$$

$$\varepsilon \geq 0, \quad (4-53e)$$

$$\tilde{\xi}_{low} \leq \tilde{\xi}(k|t) \leq \tilde{\xi}_{up}, \quad k = t, t+1, \dots, t+N-1 \quad (4-53f)$$

$$\mathbf{u}_{low} \leq \mathbf{u}(k|t) \leq \mathbf{u}_{up}, \quad k = t, t+1, \dots, t+N-1 \quad (4-53g)$$

$$\Delta \mathbf{u}_{low} \leq \Delta \mathbf{u}(k|t) \leq \Delta \mathbf{u}_{up}, \quad k = t, t+1, \dots, t+N-1 \quad (4-53h)$$

其中， ε 是松弛因子。

(4-53a) 的优化目标函数可以写成如下二次型，

$$J = \sum_{k=1}^{N-1} (\|\boldsymbol{\eta}(t+k|t)\|_Q^2) + \sum_{k=1}^N (-\gamma \mathbf{M} \tilde{\xi}(t+k|t) + \|\Delta \mathbf{u}(t+k|t)\|_R^2) + \|\boldsymbol{\eta}(t+N|t)\|_{Q_N}^2 + \rho \varepsilon^2 \quad (4-54)$$

式中：

$$Q = \text{diag}(q_c, q_l, q_\omega) \quad (4-55a)$$

$$R = R_u \quad (4-55b)$$

$$\mathbf{M} = [0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1] \quad (4-55c)$$

把 (4-41) ~ (4-45) 代入 (4-54) 可得，

$$J = \mathbf{Y}(t)^T \bar{Q} \mathbf{Y}(t) + \Delta \mathbf{U}(t)^T \bar{R} \Delta \mathbf{U}(t) - \gamma \bar{\mathbf{M}} \mathbf{X}(t) + \rho \varepsilon^2 \quad (4-56)$$

式中：

$$\bar{Q} = \text{diag}(Q, Q, \dots, Q_N) \quad (4-57a)$$

$$\bar{R} = \text{diag}(R, R, \dots, R) \quad (4-57b)$$

$$\bar{\mathbf{M}} = [\mathbf{M}, \mathbf{M}, \dots, \mathbf{M}] \quad (4-57c)$$

其中 $\bar{Q} \in \mathbb{R}^{3N \times 3N}$ ， $\bar{R} \in \mathbb{R}^{3N \times 3N}$ ， $\bar{\mathbf{M}} \in \mathbb{R}^{1 \times 10N}$ 。

令 $\mathbf{E}(t) = -(\boldsymbol{\Psi}_{(out)t} \tilde{\xi}(t|t) + \boldsymbol{\Gamma}_{(out)t} \boldsymbol{\Phi}(t) + \boldsymbol{\Lambda}(t))$ ，进一步化简如下，

$$\begin{aligned}
 J &= [\Psi_{(out)t} \tilde{\xi}(t|t) + \Theta_{(out)t} \Delta U(t) + \Gamma_{(out)t} \Phi(t) + \Lambda(t)]^T \bar{Q} [\Psi_{(out)t} \tilde{\xi}(t|t) + \\
 &\quad \Theta_{(out)t} \Delta U(t) + \Gamma_{(out)t} \Phi(t) + \Lambda(t)] + \Delta U(t)^T \bar{R} \Delta U(t) \\
 &\quad - \gamma \bar{M} [\Psi_{(state)t} \tilde{\xi}(t|t) + \Theta_{(state)t} \Delta U(t) + \Gamma_{(state)t} \Phi(t)] + \rho \varepsilon^2 \\
 &= [\Theta_{(out)t} \Delta U(t) - E(t)]^T \bar{Q} [\Theta_{(out)t} \Delta U(t) - E(t)] + \Delta U(t)^T \bar{R} \Delta U(t) \\
 &\quad - \gamma \bar{M} \Theta_{(state)t} \Delta U(t) - \gamma \bar{M} (\Psi_{(state)t} \tilde{\xi}(t|t) + \Gamma_{(state)t} \Phi(t)) + \rho \varepsilon^2 \\
 &= \Delta U(t)^T [\Theta_{(out)t}^T \bar{Q} \Theta_{(out)t} + \bar{R}] \Delta U(t) + [-2E^T(t) \bar{Q} \Theta_{(out)t} - \gamma \bar{M} \Theta_{(state)t}] \Delta U(t) \\
 &\quad + \rho \varepsilon^2 + P_t
 \end{aligned}$$

式中： $P_t = -\gamma \bar{M} (\Psi_{(state)t} \tilde{\xi}(t|t) + \Gamma_{(state)t} \Phi(t)) + E(t)^T \bar{Q} E(t)$ 是常量，对优化问题的解没有影响，所以在目标函数中可以忽略，做如下设定：

$$H_t = \begin{bmatrix} 2(\Theta_{(out)t}^T \bar{Q} \Theta_{(out)t} + \bar{R}) & \mathbf{0}_{3N \times 1} \\ \mathbf{0}_{1 \times 3N} & 2\rho \end{bmatrix} \quad (4-58)$$

$$G_t = [-2E^T(t) \bar{Q} \Theta_{(out)t} - \gamma \bar{M} \Theta_{(state)t} \quad \mathbf{0}] \quad (4-59)$$

其中 $H_t \in \mathbb{R}^{(3N+1) \times (3N+1)}$ ， $G_t \in \mathbb{R}^{1 \times (3N+1)}$ 。这里 H_t 为正定的 Hessian 矩阵。

最终，目标函数化成如下标准二次型：

$$J = \frac{1}{2} \begin{bmatrix} \Delta U(t) \\ \varepsilon \end{bmatrix}^T H_t \begin{bmatrix} \Delta U(t) \\ \varepsilon \end{bmatrix} + G_t \begin{bmatrix} \Delta U(t) \\ \varepsilon \end{bmatrix} \quad (4-60)$$

对于边界约束，做如下设定：

$$\bar{F} = \begin{bmatrix} F_1 & \mathbf{0}_{1 \times 7} & \cdots & \mathbf{0}_{1 \times 7} \\ \mathbf{0}_{1 \times 7} & F_2 & \cdots & \mathbf{0}_{1 \times 7} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0}_{1 \times 7} & \mathbf{0}_{1 \times 7} & \cdots & F_N \end{bmatrix} \quad (4-61a)$$

$$\bar{f}_{low} = \begin{bmatrix} f_1^{low} \\ \vdots \\ f_N^{low} \end{bmatrix} \quad (4-61b)$$

$$\bar{f}_{up} = \begin{bmatrix} f_1^{up} \\ \vdots \\ f_N^{up} \end{bmatrix} \quad (4-61c)$$

(4-53d) 可整理成,

$$\bar{f}_{low} - \varepsilon \leq \bar{F}\mathbf{X}(t) \leq \bar{f}_{up} + \varepsilon \quad (4-62)$$

把 (4-41) 代入, 可得,

$$\begin{bmatrix} \bar{F}\Theta_{(state)t} & -\mathbf{1}_{N \times 1} \end{bmatrix} \begin{bmatrix} \Delta U(t) \\ \varepsilon \end{bmatrix} \leq \bar{f}_{up} - \bar{F}\Psi_{(state)t} \tilde{\xi}(t|t) - \bar{F}\Gamma_{(state)t} \Phi(t) \quad (4-63a)$$

$$\begin{bmatrix} -\bar{F}\Theta_{(state)t} & -\mathbf{1}_{N \times 1} \end{bmatrix} \begin{bmatrix} \Delta U(t) \\ \varepsilon \end{bmatrix} \leq -\bar{f}_{low} + \bar{F}\Psi_{(state)t} \tilde{\xi}(t|t) + \bar{F}\Gamma_{(state)t} \Phi(t) \quad (4-63b)$$

这两个式子表示了预测时域内所有状态的边界约束。

对于控制量约束, 做如下设定:

$$\mathbf{K} = \begin{bmatrix} 1 & 0 & \cdots & \cdots & 0 \\ 1 & 1 & 0 & \cdots & 0 \\ \cdots & \cdots & \cdots & \ddots & \vdots \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} \quad (4-64a)$$

令 $\mathbf{T}_{\Delta u} = \mathbf{K} \otimes \mathbf{I}_3$, 式中: $\mathbf{K} \in \mathbb{R}^{N \times N}$, $\mathbf{T}_{\Delta u} \in \mathbb{R}^{3N \times 3N}$, $\otimes$ 是克罗内克积符号; $\mathbf{u}(t-1|t)$

代表上一个时刻的控制量, 则 (4-53g) 可整理成:

$$\begin{bmatrix} \mathbf{T}_{\Delta u} & \mathbf{0}_{3N \times 1} \end{bmatrix} \begin{bmatrix} \Delta U(t) \\ \varepsilon \end{bmatrix} \leq U_{up} - \mathbf{u}(t-1|t) \quad (4-65a)$$

$$\begin{bmatrix} -\mathbf{T}_{\Delta u} & \mathbf{0}_{3N \times 1} \end{bmatrix} \begin{bmatrix} \Delta U(t) \\ \varepsilon \end{bmatrix} \leq \mathbf{u}(t-1|t) - U_{low} \quad (4-65b)$$

对于状态量约束, 整理 (4-53f) 并把 (4-41) 代入, 可得,

$$\begin{bmatrix} \Theta_{(state)t} & \mathbf{0}_{10N \times 1} \end{bmatrix} \begin{bmatrix} \Delta U(t) \\ \varepsilon \end{bmatrix} \leq \tilde{\xi}_{up} - \Theta_{(state)t} \Delta U(t) - \Gamma_{(state)t} \Phi(t) \quad (4-66a)$$

$$\begin{bmatrix} -\Theta_{(state)t} & \mathbf{0}_{10N \times 1} \end{bmatrix} \begin{bmatrix} \Delta U(t) \\ \varepsilon \end{bmatrix} \leq -\tilde{\xi}_{low} + \Theta_{(state)t} \Delta U(t) + \Gamma_{(state)t} \Phi(t) \quad (4-66b)$$

综上, 转化为 QP 问题后, 形式如下:

$$\min_{\Delta U(t), \varepsilon} \quad \frac{1}{2} \begin{bmatrix} \Delta U(t) \\ \varepsilon \end{bmatrix}^T H_t \begin{bmatrix} \Delta U(t) \\ \varepsilon \end{bmatrix} + G_t \begin{bmatrix} \Delta U(t) \\ \varepsilon \end{bmatrix} \quad (4-67a)$$

$$s.t. \quad \begin{bmatrix} \Delta U_{low} \\ 0 \end{bmatrix} \leq \begin{bmatrix} \Delta U(t) \\ \varepsilon \end{bmatrix} \leq \begin{bmatrix} \Delta U_{up} \\ \varepsilon_{up} \end{bmatrix} \quad (4-67b)$$

$$\begin{bmatrix} \bar{F}\Theta_{(state)t} & -\mathbf{1}_{N \times 1} \\ -\bar{F}\Theta_{(state)t} & -\mathbf{1}_{N \times 1} \\ \mathbf{T}_{\Delta u} & \mathbf{0}_{3N \times 1} \\ -\mathbf{T}_{\Delta u} & \mathbf{0}_{3N \times 1} \\ \Theta_{(state)t} & \mathbf{0}_{10N \times 1} \\ -\Theta_{(state)t} & \mathbf{0}_{10N \times 1} \end{bmatrix} \begin{bmatrix} \Delta U(t) \\ \varepsilon \end{bmatrix} \leq \begin{bmatrix} \bar{f}_{up} - \bar{F}\Psi_{(state)t} \tilde{\xi}(t|t) - \bar{F}\Gamma_{(state)t} \Phi(t) \\ -\bar{f}_{low} + \bar{F}\Psi_{(state)t} \tilde{\xi}(t|t) + \bar{F}\Gamma_{(state)t} \Phi(t) \\ U_{up} - \mathbf{u}(t-1|t) \\ \mathbf{u}(t-1|t) - U_{low} \\ \tilde{\xi}_{up} - \Theta_{(state)t} \Delta U(t) - \Gamma_{(state)t} \Phi(t) \\ -\tilde{\xi}_{low} + \Theta_{(state)t} \Delta U(t) + \Gamma_{(state)t} \Phi(t) \end{bmatrix} \quad (4-67c)$$

若已知 t 时刻的状态量 $\tilde{\xi}(t|t)$ 和上一个时刻发出的控制量 $\mathbf{u}(t-1|t)$ ，通过使用二次规划求解器求解 (4-67)，便可得到预测时域 N 内的最优控制增量序列 $\Delta U^*(t)$ ，把它代入 $\mathbf{X}(t) = \Psi_{(state)t} \tilde{\xi}(t|t) + \Theta_{(state)t} \Delta U(t) + \Gamma_{(state)t} \Phi(t)$ ，便可得到最优状态量序列 $\mathbf{X}^*(t)$ ，最优局部路径的信息便包含在其中。最优控制量序列 $\mathbf{U}^*(t)$ 也可以通过 $\Delta U^*(t)$ 求出。

最后要说明的是，上面的优化问题仍有一个待解决的问题，那就是状态转移矩阵 $\tilde{\mathbf{A}}_{k,t}$ 、 $\tilde{\mathbf{B}}_{k,t}$ 、 $\tilde{\mathbf{g}}_{k,t}$ 和 $\tilde{\mathbf{C}}_{k,t}$ 的求取问题。计算这几个矩阵需要知道预测时域内每一个点的一阶泰勒展开点，本文使用不断更新泰勒展开点的方法，首先，在初始时刻，通过设定一个初始速度 V_0 ，计算 $\theta_{A,k} = \theta_{A,0} + kV_0 T_s (k=1,2,\dots,N)$ ，从而知道初始轨迹的泰勒展开点 $\tilde{\xi}_{k,t} = [X^{\text{ref}}(\theta_{A,k}), Y^{\text{ref}}(\theta_{A,k}), \Phi(\theta_{A,k}), V_0, 0, 0, \theta_{A,k}, 0, 0, 0]^T$ ， $\mathbf{u}_{k,t} = [0, 0, 0]^T$ ， $\Delta \mathbf{u}_{k,t} = [0, 0, 0]^T$ ($k=1,2,\dots,N$)。有了第一组泰勒展开点，就可以产生第一组状态转移矩阵，并且通过求解 (4-67) 产生第一组优化轨迹。后续处理就是在前一次得到的优化轨迹序列 $\mathbf{X}^*(t)$ 的基础上，把优化轨迹序列的第一个状态舍弃，第二个状态用当前 GPS 得到的定位数据替代，第 $k(k=3,4,\dots,N)$ 个状态保持不变，最后再在末尾插入一个新的状态，该状态是把前一个时刻最优控制量序列 $\mathbf{U}^*(t)$ 的最后一个控制量输入到最后的状态量 $\tilde{\xi}(t+N|t)$ 上，通过动力学模型 (2-1)，使用四阶龙格-库塔法得到状态 $\tilde{\xi}(t+N+1|t)$ ，这样就得到了该时

刻的展开点 $\tilde{\xi}_{t+k,t+1}(k=1,2,\dots,N+1)$ ；至于 $\mathbf{u}_{t+k,t+1}(k=1,2,\dots,N+1)$ 和 $\Delta\mathbf{u}_{t+k,t+1}(k=1,2,\dots,N+1)$ 则通过舍弃 $\mathbf{U}^*(t)$ 和 $\Delta\mathbf{U}^*(t)$ 的第一个控制量和保持最后一个控制量得到。

除此之外，参照序列二次规划（Sequential quadratic programming，简称为 SQP）的思想，在求得一组最优控制量与状态量后，先不把最优控制量直接输出到底层，而是进一步地优化，通过把前一个泰勒展开点和最优状态量进行线性组合，作为下一次最优轨迹的泰勒展开点，再次求解 QP 问题（4-67），从而得到进一步优化后的最优变量，可求解多次，本文选择在每个采样点均求解 2 次（4-67）后再把最优控制量输出到底层进行控制，这样得到的轨迹会更加有利于车辆的快速行驶。如图 4-5 所示，图 b) 展示经过该处理后的结果，对比图 a) 可知该轨迹整体曲率更小，而且车辆可以做到贴着内弯道行驶，有利于车辆的快速行驶。

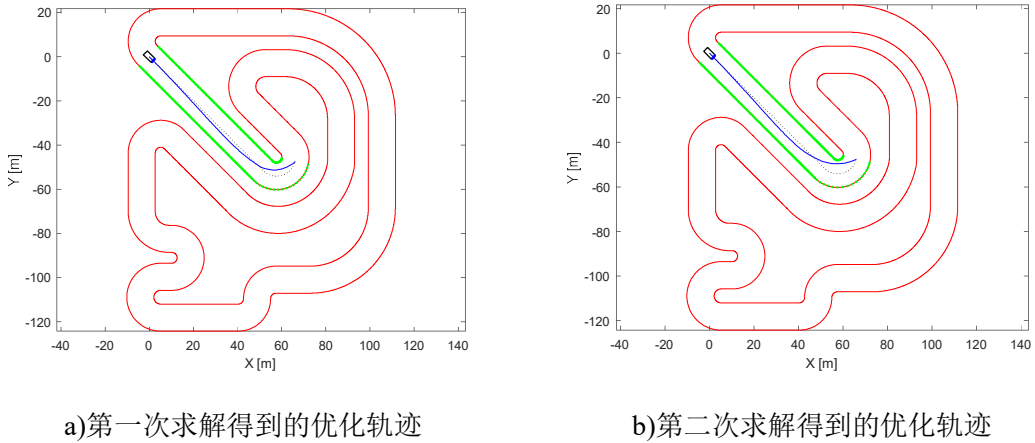


图 4-5 SQP 优化后的结果

4.6 算法优化

本文使用 matlab 的 quadprog 函数进行 QP 问题的求解，然而实际使用中发现，它不能满足实时求解的需求，在 N 较大时，其求解时间较长，可是车辆想要高速行驶又要求 N 不能过小，所以对算法进行优化是很有必要的。

此外，由于求解器可能会出现在求解精度范围内无解的情况，为了提高算法鲁棒性，还需要对算法进行进一步处理。

4.6.1 物理量的缩放

由于物理量的差异，不同物理量数值上可能相差很大，比如文中的控制量 δ （单位为 rad）和 V （单位为 m/s）， δ 的值一般在 $\pm 0.5\text{rad}$ 之间，而 V 则可能会达到 30m/s 或更大，二者的差距可能会达到 1 个甚至 2 个数量级，变量间相差几个数量级对于优化算法的求解来说是不利的，会使得优化算法进行很多无关的搜索，增大求解时间。所以可以通过对输入量进行缩放，把它们缩放到 ± 1 之间，将更加有利于求解。

令 $\mathbf{S}_{\Delta U}$ 代表输入量 $\Delta \mathbf{U}(t)$ 的缩放矩阵，则，

$$\Delta \mathbf{U}_{scale}(t) = \mathbf{S}_{\Delta U} \Delta \mathbf{U}(t) \quad (4-68a)$$

$$\Delta \mathbf{U}(t) = \mathbf{S}_{\Delta U}^{-1} \Delta \mathbf{U}_{scale}(t) \quad (4-68b)$$

把 $\Delta \mathbf{U}_{scale}(t)$ 作为待优化的变量，并把（4-68b）代入（4-67）各式中，即可得到关于缩放变量 $\Delta \mathbf{U}_{scale}(t)$ 的最优化问题，求得结果后再代回（4-68a）即可得到原变量。

除此之外，还对目标函数进行了缩放，在 Hessian 矩阵 H_t 和线性化项 G_t 前乘一个缩放因子，以防止纵向误差过大导致相应的矩阵过大，影响求解的质量与速度。

4.6.2 求解器的选择

由于（4-67）中 Hessian 矩阵 H_t 是稠密的，而 quadprog 函数对于稠密 QP 问题的求解效率相当低下，因此借助可以求解稠密 QP 的其他求解器可以大幅提升效率。

在不用 CarSim 仿真的情况下本文选择使用开源求解器 hpipm 进行求解²，其求解速度比 quadprog 快了几个数量级，具体求解时间差距在第五章仿真实验中给出。hpipm 的文档可以参照文献[30]。

4.6.3 优化变量升维处理

实验中发现，相对于求解含有稠密矩阵 H_t 的 QP 问题，求解器对于稀疏 QP 问题有

² <https://github.com/giaf/hpipm>

更高的求解速度。根据这个思路，可以通过增加优化变量提升问题的维度，达到提升求解速度的目的。具体做法是，把所有状态量也作为待优化变量放入优化问题（4-53）中，并把车辆的线性时变预测模型作为等式约束放到求解器中，同时也要对状态量进行物理量的缩放。

由于优化变量的改变，所以输出量只需定义为横纵向误差，而目标函数的二次型也要计算，做如下设定：

$$\nabla e_{k,t} = \begin{bmatrix} \frac{\partial e^c}{\partial \tilde{\xi}} \bigg|_{\tilde{\xi}_{k,t}} \\ \frac{\partial e^l}{\partial \tilde{\xi}} \bigg|_{\tilde{\xi}_{k,t}} \end{bmatrix} \quad (4-69)$$

$$\tilde{\eta}(k|t) = \begin{bmatrix} e^c(k|t) \\ e^l(k|t) \end{bmatrix} = \begin{bmatrix} e^c_{k,t} \\ e^l_{k,t} \end{bmatrix} + \nabla e_{k,t}(\tilde{\xi}(k|t) - \tilde{\xi}_{k,t}) = e_{k,t} + \nabla e_{k,t}(\tilde{\xi}(k|t) - \tilde{\xi}_{k,t}) \quad (4-70)$$

式中：下标中的 k 和 t 代表的是在 t 时刻，预测时域内第 k 个状态的泰勒展开点，而括号内的 k 和 t 代表在 t 时刻，预测时域内第 k 个状态的实际值。

式（4-53a）代价函数可化成如下的形式，

$$\begin{aligned} \min_{\tilde{\xi}, \Delta U(t), e} \quad & \sum_{k=1}^{N-1} (\tilde{\eta}^T(t+k|t) \mathbf{q} \tilde{\eta}(t+k|t) + \|\omega_k\|_{q_\omega}^2) + \sum_{k=0}^{N-1} (-\gamma V_k + \|\Delta u_k\|_{R_u}^2) + \\ & + \tilde{\eta}^T(t+N|t) \mathbf{q}_N \tilde{\eta}(t+N|t) + \|\omega_N\|_{q_{\omega N}}^2 + \rho \varepsilon^2 \end{aligned} \quad (4-71)$$

式中： $\mathbf{q} = \text{diag}(q_c, q_l)$ ， $\mathbf{q}_N = \text{diag}(q_{cN}, q_{lN})$ 。

上式除了对 $\tilde{\eta}^T(k|t) \mathbf{q} \tilde{\eta}(k|t)$ 的求解，其余的求解思路与 4.5 节类似。把（4-70）代入 $\tilde{\eta}^T(k|t) \mathbf{q} \tilde{\eta}(k|t)$ ，可得，

$$[e_{k,t} + \nabla e_{k,t}(\tilde{\xi}(k|t) - \tilde{\xi}_{k,t})]^T \mathbf{q} [e_{k,t} + \nabla e_{k,t}(\tilde{\xi}(k|t) - \tilde{\xi}_{k,t})] \quad (4-72)$$

化简可得，

$$\tilde{\xi}^T(k|t) \nabla e_{k,t}^T \mathbf{q} \nabla e_{k,t} \tilde{\xi}(k|t) + [2e_{k,t}^T \mathbf{q} \nabla e_{k,t} - 2\tilde{\xi}_{k,t}^T \nabla e_{k,t}^T \mathbf{q} \nabla e_{k,t}] \tilde{\xi}(k|t) \quad (4-73)$$

其中： $\nabla e_{k,t}^T \mathbf{q} \nabla e_{k,t}$ 可以作为 Hessian 矩阵 H_t 的一部分， $[2e_{k,t}^T \mathbf{q} \nabla e_{k,t} - 2\tilde{\xi}_{k,t}^T \nabla e_{k,t}^T \mathbf{q} \nabla e_{k,t}]$ 可以作为 G_t 的一部分。再把相应的 q_ω 、 γ 和 R_u 填入 H_t 和 G_t 相应的位置，即可完成如下

标准二次型的构建：

$$J = \frac{1}{2} \begin{bmatrix} \tilde{\xi}(t|t) \\ \Delta \mathbf{u}(t|t) \\ \vdots \\ \Delta \mathbf{u}(t+N-1|t) \\ \tilde{\xi}(t+N|t) \\ \varepsilon \end{bmatrix}^T H_t \begin{bmatrix} \tilde{\xi}(t|t) \\ \Delta \mathbf{u}(t|t) \\ \vdots \\ \Delta \mathbf{u}(t+N-1|t) \\ \tilde{\xi}(t+N|t) \\ \varepsilon \end{bmatrix} + G_t \begin{bmatrix} \tilde{\xi}(t|t) \\ \Delta \mathbf{u}(t|t) \\ \vdots \\ \Delta \mathbf{u}(t+N-1|t) \\ \tilde{\xi}(t+N|t) \\ \varepsilon \end{bmatrix} \quad (4-74)$$

4.6.4 失效处理

求解器在对上述优化问题进行实际求解的过程中，可能会出现求解失败的情况，对于车辆的控制来说，这是很严重的问题，因为求解失败意味着控制量无法更新，只能保持上一时刻计算的控制量，无法给车辆输入更为精确的控制量，达不到模型预测控制中的滚动优化的目的，同时车辆轨迹也无法进一步优化更新，如果此时车辆车速较快，车辆很有可能冲出边界或者撞击其他障碍物，造成不必要的人员伤亡和财物损失，极其危险。因此，想要在比赛中应用甚至在实车中应用，算法必须考虑到求解失败的情况，并进行相应的处理，提升其鲁棒性。

在求解失败时，常见的处理就是让车辆紧急停车，这种情况虽然可以防止车辆跑出车道边界和防止碰撞，保证车内人员的安全，但是这种野蛮的处理是不可采取的，第一个原因是我们无法保证车辆在复杂道路上不会出现频繁求解失败的情况；第二个原因是紧急停车后不能让车辆一直停在那里，需要考虑重新启动跟踪的问题；第三个原因是在无人驾驶方程式大赛中，紧急停车会使圈速时间变长，成绩变差；第四个原因是没有充分利用车辆已知的中心线等道路信息。

鉴于上述原因，本文提出了一种在求解失败时退化为 stanley 控制器跟踪中心线的方法，当求解失败的次数超过一个阈值 N_{fail} 之后，横向控制器变为 stanley 跟踪中心线，关于 stanley 控制器的介绍，可参考文献[31]。此时，纵向控制器采用比例控制器，设定一个较低的速度 v_c ，通过控制 d 来使车速保持在 v_c 。在计算出 stanley 的控制量后，要更新一阶泰勒展开点，与初始化泰勒展开点时类似，对状态点的初始化，选取与车辆当前位置欧式距离最近的点往后 N 个中心线上的点作为状态量的泰勒展开点，再根据求解失

败次数的阈值更新控制量的泰勒展开点，把前 N_{fail} 个控制量的占空比 d 和 δ 更新为与比例控制器计算的 d 和 stanley 计算的 $\delta_{stanely}$ 对应的数值，控制增量泰勒展开点设定为 $\mathbf{0}$ 向量。

经过以上处理后， $\delta_{stanely}$ 会被发送到底层，底层会控制相应的机构，使得车辆跟踪中心线行驶。当车辆不断接近中心线时，由于车辆位置等原因，求解器很有可能会重新求得优化问题的解，此时局部规划控制器再次接管整个规划控制系统，产生相应的最优局部路径，这样便充分利用了车辆已知的中心线等信息，同时车辆也不会因此紧急停车，而是按照设定的速度 v_c 行驶，较为安全而且能够持续行驶。

4.7 全局路径生成算法

目前车辆运动控制领域已经有较为成熟的横纵向控制算法，上述规划控制算法是横纵向控制耦合的，这种控制算法理论上可行，但是还缺乏实车实验，此外，它需要的模型参数较多，实车应用前需要做大量标定，所以离实际应用仍然有一段距离，然而，由于它能产生优秀的轨迹，特别是能产生相对于中心线弧长最长的轨迹，这种优点对于无人赛车来说是很重要的，它可以使得赛车完成“切弯”的动作，充分利用了赛道的空间，产生更快的圈速。为了弥补实车实验上的欠缺，充分利用其优点，可以通过把局部路径拼接在一起形成全局路径，然后把全局路径发送到控制层，用更加简单有效的控制算法或者车辆运动控制领域比较成熟实用的控制算法进行轨迹跟踪，这样就可以做到扬长避短，而且可以保证控制算法足够实用有效。

全局路径规划算法的实现过程描述如下，假设已经得到了一条最优局部轨迹，接着把轨迹的前半部分作为全局路径的一部分加入到全局路径向量中，后半部分作为下一时刻的状态量泰勒展开点，车辆位置也移动到最优局部轨迹中点位置，剩余后半部分状态量泰勒展开点由最优轨迹最后一个点对应的中心线点及其后面的中心线点构成。下一时刻控制量的泰勒展开点前半部分也是用最优轨迹对应的最优控制量的后半部分，而后半部分用的是 $\mathbf{0}$ 向量。下一时刻控制增量的泰勒展开点和控制量泰勒展开点的更新方式一致。图 4-6 显示了状态量泰勒展开点的选择，前半部分（红点）加入全局路径向量，后半部分加中心线（蓝点）作为下一个时刻的泰勒展开点。虽然在中点处会出现不连续的

状况，但是这只是求解优化问题的泰勒展开点，并不是最终加入全局路径的数据点，通过优化问题的求解，会求解出一条连续的局部轨迹再加入全局路径向量中。按照以上过程不断迭代到终点，便得到一条完整的全局路径。如果在某一个时刻出现求解失败的情况，处理方式是直接把该时刻泰勒展开点前半部分拿出放入全局路径向量，而不是像 4.6.4 节那样更新各个泰勒展开点。所以求解失败时，全局路径有可能会局部不连续的状况，但是这个问题对于整体的轨迹跟踪来说影响并不是很大，可以通过增大点之间的间隔对路径点进行重采样，再进行样条插值消除这种不连续性。

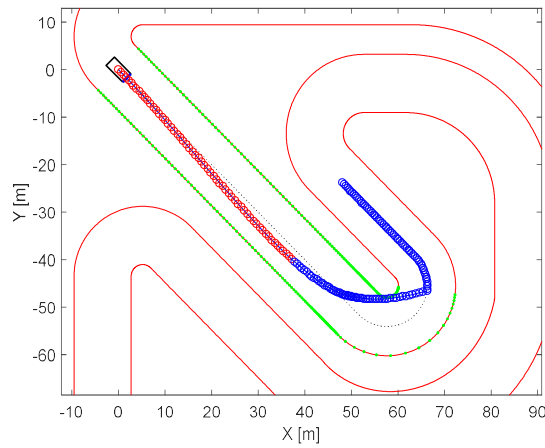


图 4-6 状态量泰勒展开点的选择

4.8 小结

本章介绍了两种规划算法，一种是集规划控制一体化的局部规划控制算法，另一种是全局路径生成算法，全局算法是以局部规划控制算法为基础的。

首先介绍了局部规划控制算法的框架，从非线性 MPC 出发，介绍模型预测问题的基本思路。接着分别详细介绍了输出量的定义和 LTVPM 的推导过程，以及如何将模型预测问题转化为便于计算机求解的 QP 问题。为了保证算法实时性和鲁棒性，还对算法进行了相关优化和失效处理。

然后，考虑到局部轨迹控制器还不够成熟，在上述优化算法框架的基础上，介绍了全局路径生成算法，使得车辆可以使用车辆运动控制领域内更加成熟实用的横纵向控制算法或者使用一些更为简单高效的控制算法进行轨迹跟踪，保证控制层的精确性。

第五章 仿真结果分析

本章主要是对第四章所阐述的算法进行仿真验证与结果分析，仿真器有两个，一个是直接用（2-1）的车辆动力学自行车模型，通过四阶龙格-库塔法更新状态的仿真器，另一个是 CarSim8.02 与 Simulink 的联合仿真器。因为 hpipm 求解器仅仅支持 linux 系统，而 CarSim8.02 未支持 linux 系统，所以 hpipm 的仿真结果只能通过第一个仿真器进行仿真验证。联合仿真器中仍然使用 matlab 的 quadprog 函数进行求解。

5.1 四阶龙格-库塔法仿真器仿真分析

本节使用的仿真器基于（2-1）的车辆动力学模型，利用四阶龙格-库塔法求解（2-1）方程组进行状态的更新，主要进行优化前算法和优化后算法性能的对比，还有对 hpipm 求解器性能的验证。

5.1.1 车辆参数与模型参数

使用车辆动力学模型（2-1）进行 hpipm 与 quadprog 求解器的性能对比仿真以及 hpipm 求解结果的进一步分析，主要的车辆模型参数如表 5-1 所示。

表 5-1 用于仿真的车辆模型参数

参数	数值
整车质量 m / kg	1950
横摆转动惯量 I_z / ($\text{kg}\cdot\text{m}^2$)	3332.14
质心到前轴距离 l_f / m	1.07
质心到后轴距离 l_r / m	1.597
车辆宽度 w_{car} / m	1.8
直流电机模型系数 C_{m1} / N	17303
直流电机模型系数 C_{m2} / ($\text{N}\cdot\text{m}^{-1}\cdot\text{s}$)	175

表 5-1 用于仿真的车辆模型参数（续）

参数	数值
滚动阻力 C_r / N	286.65
等效空阻系数 C_d / ($\text{N} \cdot \text{m}^{-2} \cdot \text{s}^2$)	0.5359
前轮侧向力系数 B_f	13
前轮侧向力系数 C_f	2
前轮侧向力系数 D_f	13745.69
后轮侧向力系数 B_r	13
后轮侧向力系数 C_r	2
后轮侧向力系数 D_r	9209.7
附加力矩比例系数 P_{TV}	100

后面的章节会通过仿真结果，证明对优化变量进行升维处理可以大大提升算法的求解效率。由于优化变量升维处理后的模型（本章称之为优化模型）与未升维处理的模型（本章称之为未优化模型）稍有不同，所以控制器参数也稍有不同，两个控制器的参数如下所示。

（1）未优化模型的参数如下：

采样周期： $T_s = 0.05\text{s}$

预测时域： $N = 120$

$$\text{权重矩阵： } Q = \begin{bmatrix} 0.01 & & \\ & 1000 & \\ & & 15 \end{bmatrix}, \quad Q_N = \begin{bmatrix} 100 & & \\ & 1000 & \\ & & 15 \end{bmatrix},$$

$$R_u = \begin{bmatrix} 0.1 & & \\ & 1 & \\ & & 0.1 \end{bmatrix}, \quad \gamma = 10, \quad \rho = 200$$

状态量极值： $\tilde{\xi}_{up} = [10^6, 10^6, 30, 20, 30, 5, 10^6]^T$,

$$\tilde{\xi}_{low} = [-10^6, -10^6, -30, 2.5, -30, -5, 0]^T$$

$$\text{控制量极值: } U_{up} = [1, 0.5, 100]^T, \quad U_{low} = [-1, -0.5, 0]^T$$

$$\text{控制增量极值: } \Delta U_{up} = [0.25, 0.2, 10]^T, \quad \Delta U_{low} = [-0.5, -0.2, -10]^T$$

$$\text{最大的松弛因子: } \varepsilon_{up} = 10$$

$$\text{初始时刻预测泰勒展开点的速度: } V_0 = 2.5\text{m/s}$$

$$\text{缩放矩阵: } \mathbf{S}_{\Delta U} = \begin{bmatrix} 1 & & \\ & 2 & \\ & & 0.1 \end{bmatrix}$$

$$\text{求解失败的阈值: } N_{fail} = 6$$

$$\text{stanley 控制速度: } v_c = 6\text{m/s}$$

(2) 优化模型的参数如下:

$$\text{采样周期: } T_s = 0.05\text{s}$$

$$\text{预测时域: } N = 120$$

$$\text{权重矩阵: } Q = \begin{bmatrix} 0.01 & & \\ & 1000 & \\ & & 5 \end{bmatrix}, \quad Q_N = \begin{bmatrix} 100 & & \\ & 1000 & \\ & & 5 \end{bmatrix},$$

$$R_u = \begin{bmatrix} 0.1 & & \\ & 1 & \\ & & 0.1 \end{bmatrix}, \quad \gamma = 0.5, \quad \rho = 200$$

$$\text{状态量极值: } \tilde{\xi}_{up} = [10^6, 10^6, 3, 2, 3, 1, 10^5]^T,$$

$$\tilde{\xi}_{low} = [-10^6, -10^6, -3, 0.25, -3, -1, 0]^T$$

$$\text{控制量极值: } U_{up} = [1, 1, 10]^T, \quad U_{low} = [-1, -1, 0]^T$$

$$\text{控制增量极值: } \Delta U_{up} = [0.25, 0.2, 10]^T, \quad \Delta U_{low} = [-0.5, -0.2, -10]^T$$

$$\text{最大的松弛因子: } \varepsilon_{up} = 10$$

$$\text{初始时刻预测泰勒展开点的速度: } V_0 = 2.5\text{m/s}$$

$$\text{缩放矩阵: } \mathbf{S}_{\Delta U} = \begin{bmatrix} 1 & & \\ & 2 & \\ & & 0.1 \end{bmatrix}, \quad \mathbf{S}_U = \mathbf{S}_{\Delta U},$$

$$\mathbf{S}_{\xi} = \text{diag}(1, 1, \frac{1}{2\pi}, 0.1, 0.1, 0.2, 0.1)$$

求解失败的阈值: $N_{fail} = 6$

stanley 控制速度: $v_c = 6\text{m/s}$

需要注意的一点是，上述两组模型参数中与优化变量有关的极值约束都是经过缩放后的值，比如上述第（1）组参数的 ΔU_{up} 与 ΔU_{low} ，第（2）组参数的 ΔU_{up} 、 ΔU_{low} 、 U_{up} 、 U_{low} 、 $\tilde{\xi}_{up}$ 与 $\tilde{\xi}_{low}$ 都是缩放后优化变量的取值范围；此外，权重矩阵也是对应缩放后的优化变量。

5.1.2 hpipm 与 quadprog 求解器性能对比

仿真所用的地图数据与文献[15]的一致，如图 5-1 所示。该地图包含多种不同类型的弯道，如 U 型弯（①号弯）、急弯（⑤、⑥号弯）等；还有可用“切弯”动作通过的弯道，如⑥号弯和⑧号弯等。因为有多多种多样的弯道，所以这个地图可以有效验证算法的性能。

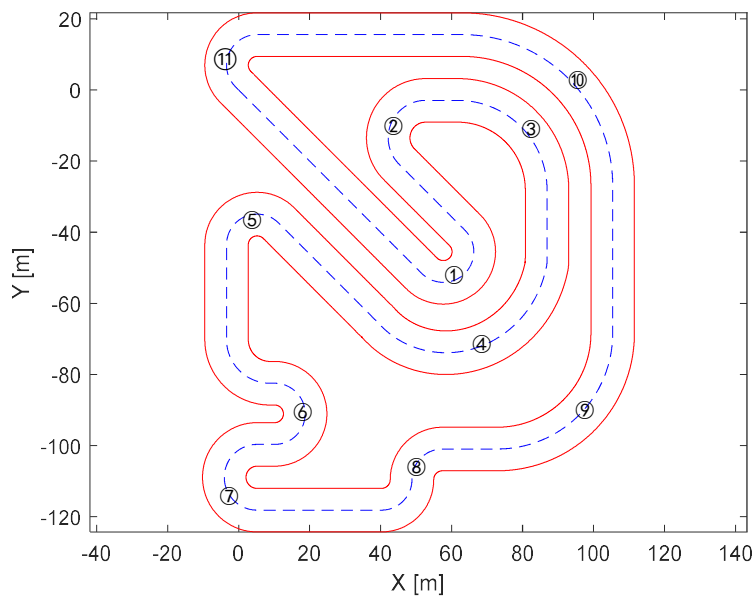


图 5-1 用于验证算法的地图

设定仿真时间 $T = 200\text{s}$ ，由于仿真只用了动力学模型 (2-1)，所以预测模型中也没引入运动学模型，也即令 $\lambda = 1$ 。分别用 **hpipm** 和 **quadprog** 对未优化模型和优化模型进行仿真，仿真结果如图 5-2 所示。

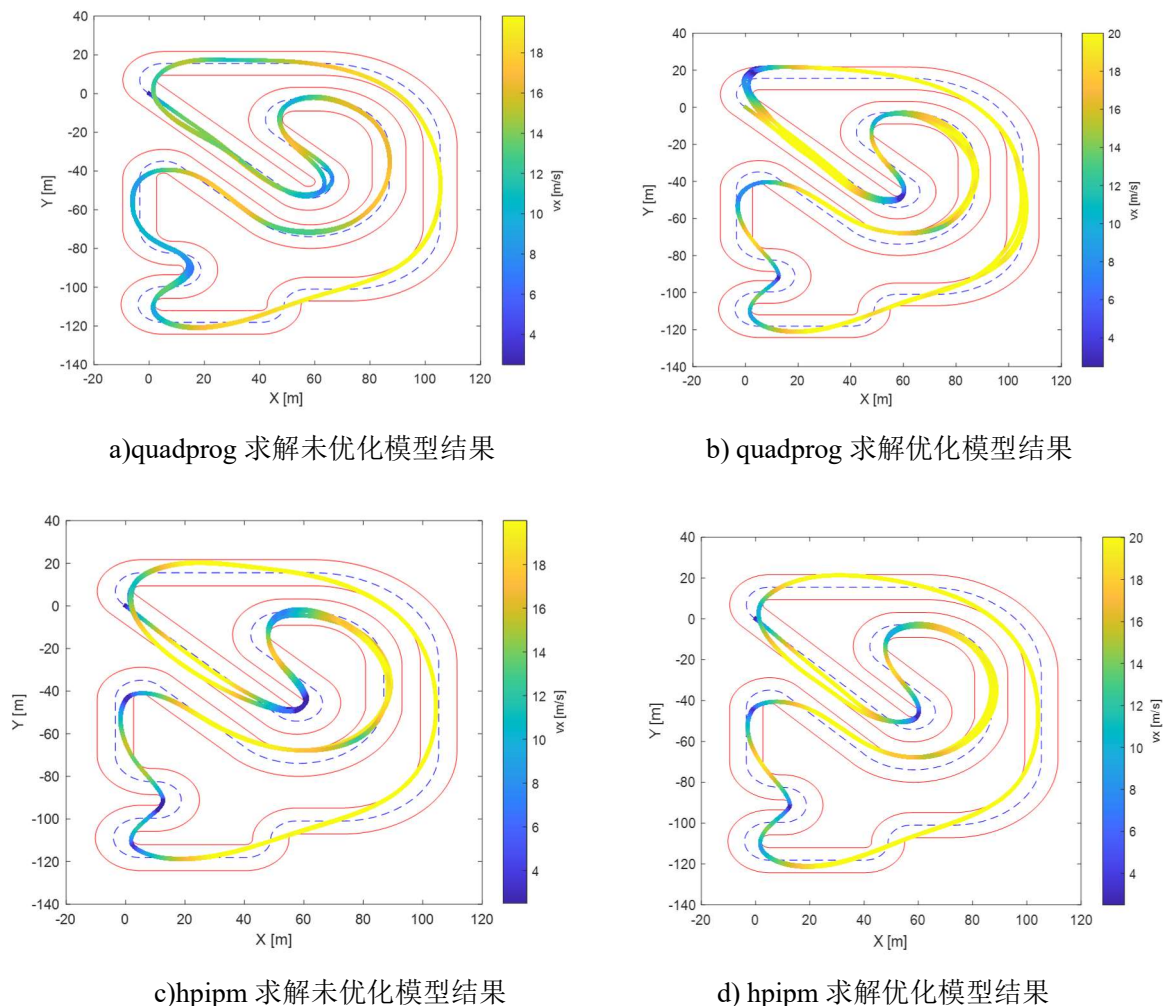


图 5-2 hpipm 与 quadprog 求解器仿真结果对比

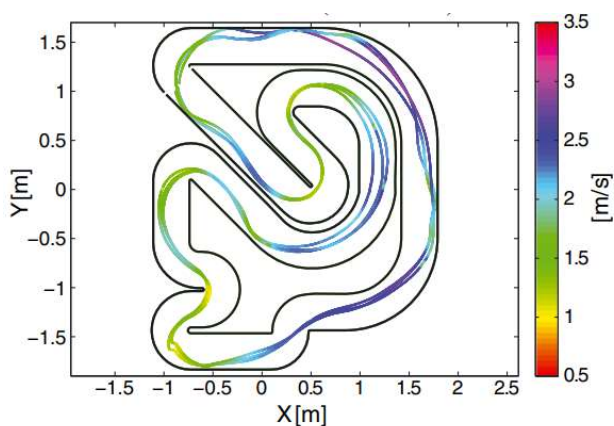


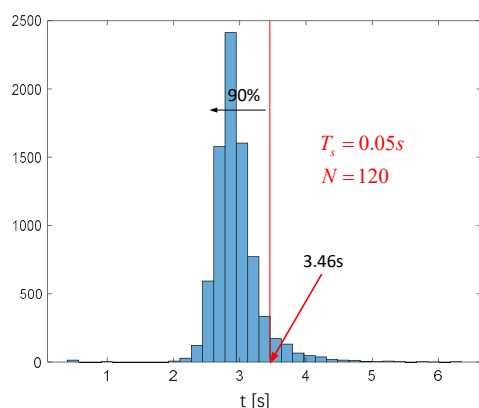
图 5-3 文献[15]的实验结果

图中用不同颜色代表不同的速度，从图中可以看出，未优化和优化后的模型在两种求解器下均能完整地跑完一圈，而且能完成类似人类驾驶员的“切弯”动作，最高速度可达 20m/s，并且车速只有在起点或者转急弯时才会降低，保证车辆得以减速过弯，整个过程车辆都没有驶出边界（红线），结果证明了算法是可行的。

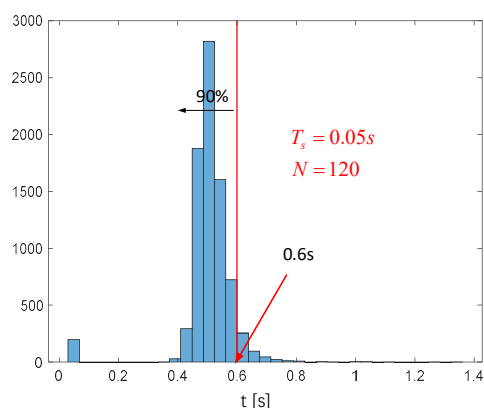
对比图 5-2 中的图 a)、图 c)或者图 b)、图 d)可看出，不同求解器对同一个模型会产生不同的效果，图 a)中，汽车除了在起点，整个运动过程中速度最低点也没达到最低速度 2.5m/s，而图 c)有好几个弯道都减速到了最低速度，并且图 c)在左上角进入终点前的弯道（⑩号弯道）处，车辆会转一个很大的转弯，而在图 a)中相同位置，车辆并没有出现大转弯的情况。图 b)与图 d)也有较大区别，图 d)中，每圈的车辆轨迹较为均匀一致，而图 b)相对来说没那么均匀，而且图 b)在⑩号弯道处转弯没有图 c)轻松与流畅。

还可以把图 5-2 中的结果与文献[15]的结果（图 5-3）进行比较，可以看出本文算法产生的轨迹更加平滑，而且除非求解失败，不然不会出现如[15]中那样仍然跟踪中心线的情况。此外，文献[15]使用了 1:43 的小车数据进行实车实验，但是算法的实际效果还有待实车验证，在实车验证前，本文的实车数据仿真结果具有一定的参考价值。

一个更关键的问题是，不同求解器与模型，其求解时间是否能够满足实时性的要求，一般在中低速情况下，采样频率设置为 20Hz，也即求解器在 50ms 以内求出结果并输入底层控制器对于车辆来说是最完美的，图 5-4 显示了图 5-2 中四种不同情况在求解时间上的表现。



a) quadprog 求解未优化模型结果



b) quadprog 求解优化模型结果

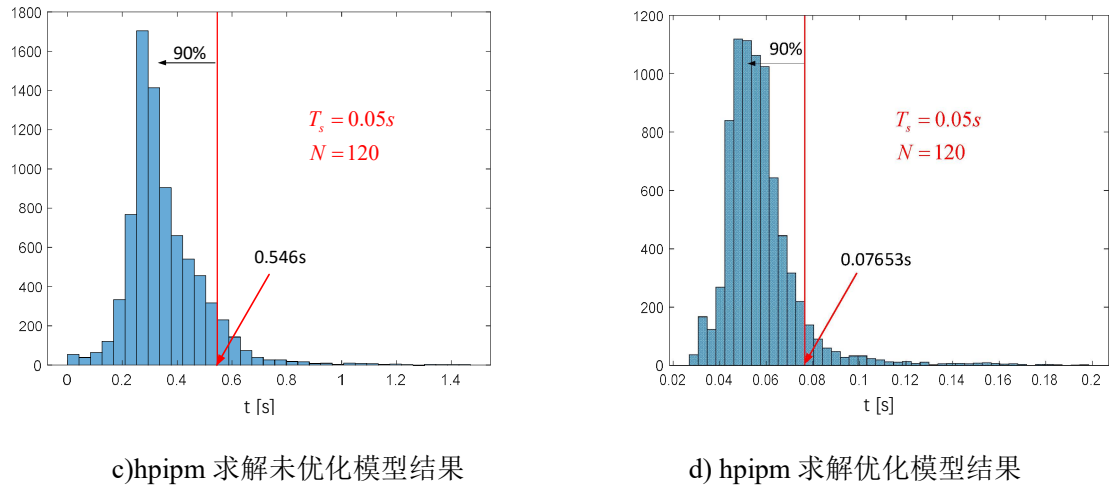


图 5-4 求解时间直方图

从上面直方图中可以看出，未优化模型在 `quadprog` 中表现极差（图 a）），在 $N = 120$ 的情况下每个采样时刻的求解时间几乎都超过 2s，而且大部分集中在 3s 附近，说明除非降低 N ，否则不可能在实车上使用，而只限于仿真中实现；而用 `hpipm` 求解未优化模型（图 c）），求解时间比图 a)快了大约 10 倍，且求解时间大部分集中在 0.3s 附近，虽然有部分时刻求解时间超过了 1s，但是 90% 以上的时刻都在 0.546s 内求解完成，说明对于同一模型，`hpipm` 的性能表现大大优于 `quadprog`；而通过优化变量的升维处理，并且进行相应物理量的缩放后，`quadprog` 的求解速度（图 b））明显比未优化改进了（图 a）），求解速度快了 5 倍多，求解时间集中分布在 0.5s 附近，偶然间也会出现求解时间超过 1s 的情况；效果最好的就是用 `hpipm` 对优化模型进行求解的结果（图 c）），前三种情况在 $N = 120$ 时都没有达到 50ms 内求解完成的要求，而且离 50ms 还有较大的差距，而用 `hpipm` 求解优化后的模型，32.45% 的时刻可以在 50ms 内求解出结果，90% 以上时刻都可以在 0.07653s 内得出结果，而且最慢也不会超过 0.2s，性能提高了一个级别。目前常见的 MPC， N 一般设置为 40 左右，很少有超过 100 的，`hpipm` 相对 `quadprog` 能在 $N = 120$ 时产生如此令人满意的结果，体现了 `hpipm` 求解器优异的性能。以上结果一方面说明了选择适合模型的求解器十分重要，另一方面说明了目前关于 QP 问题的求解器对于稠密矩阵的处理不如稀疏矩阵，所以通过提升维度的方法，可以大大提升求解的速度。

上述四种情况的平均求解时间列在表 5-2 内，从中也能看出 `hpipm` 求解器的优势。表 5-3 显示了四种情况的平均圈速，可以发现由于模型差异，优化后的模型可使得车辆

平均圈速时间降低，平均来说，优化模型比未优化模型快 5s 左右。

表 5-2 不同求解器对不同模型的平均求解时间

求解器 \ 模型	未优化模型	优化模型
quadprog	2.9469 s	0.5049 s
hpipm	0.3579 s	0.0595 s

表 5-3 不同情况的平均圈速

求解器 \ 模型	未优化模型	优化模型
quadprog	49.2554 s	45.8048 s
hpipm	49.7247 s	43.5771 s

5.1.3 hpipm 求解结果分析

最后将进一步分析算法预测轨迹的性能，通过对车辆在某个时刻的预测轨迹分析车辆的行为。

本节选取车辆在⑥号复杂弯道时状态进行分析，如图 5-5 所示。当车辆运动到图示位置时，获取了前方有连续弯道的信息，绿色点显示了车辆已知的前方信息，通过求解最优化问题，车辆正确计算出接下来要采取的行动，也就是减速过弯，从图中每个轨迹点的速度可以看出，车辆正确地预测了后续状态，而且为了寻求 θ_d 的最大化，车辆选择了一组可以贴弯行驶的控制量，做到了类似人类驾驶员的“切弯”动作，这样便能够节省过弯时间，提升圈速。可以看出，在过掉两个弯道之后，车辆预测的速度又开始上升，很符合实际中过弯后加速的过程。图 b)中显示了整个过程中车辆给予底层的控制量，可以看出由于代价函数的限制，控制量不会发生剧烈的突变，整体上变化平稳，可以满足实际需要。

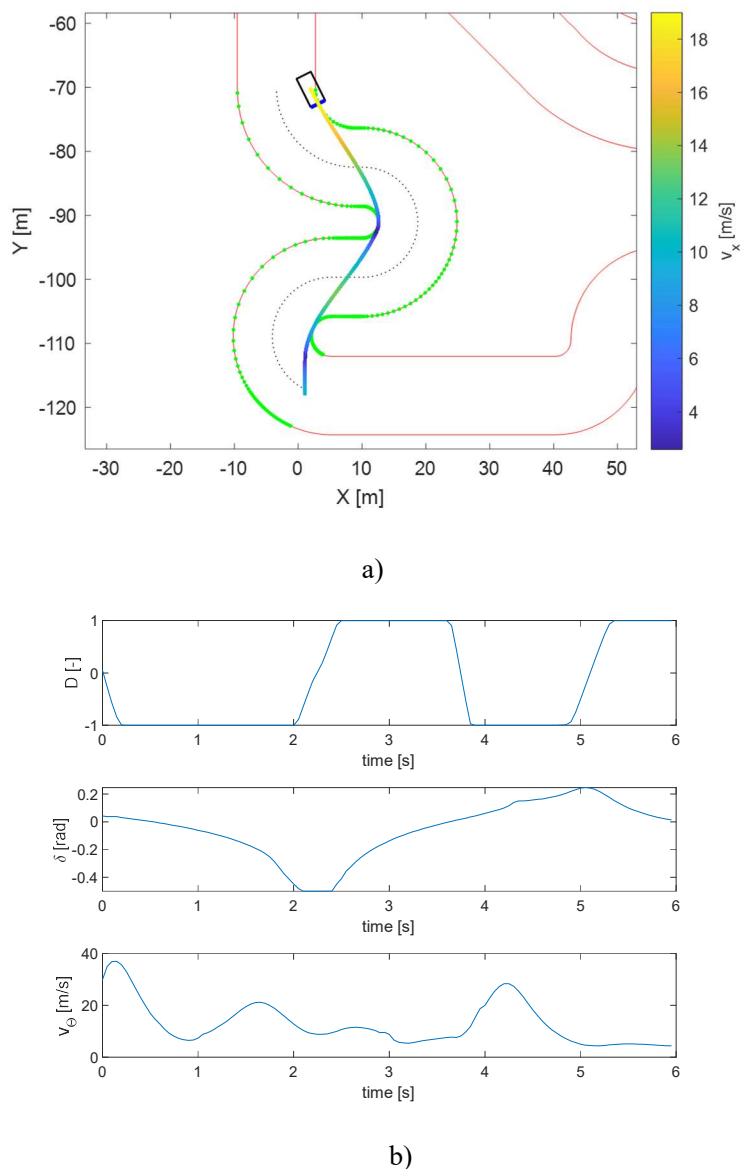


图 5-5 汽车过⑥号弯道时的预测轨迹(图 a))及其控制量(图 b))

5.2 CarSim/Simulink 联合仿真结果分析

相对于四阶龙格-库塔法仿真器，CarSim 有更加成熟的车辆模型，可以展示更加真实的结果，本节通过 CarSim/Simulink 联合仿真环境，在两个地图下对集规划控制一体化的局部控制器进行了仿真，分析了整个过程中车辆的轨迹、速度、横纵向加速度等数据，验证了算法的可行性与鲁棒性。

最后，还对全局路劲生成算法进行了仿真验证，通过对比跟踪中心线的结果，表明了算法的有效性。

5.1.4 车辆参数与模型参数

仿真测试使用的 CarSim 模型是 Stock Car 模型，车辆具体的参数如图 5-6 所示。

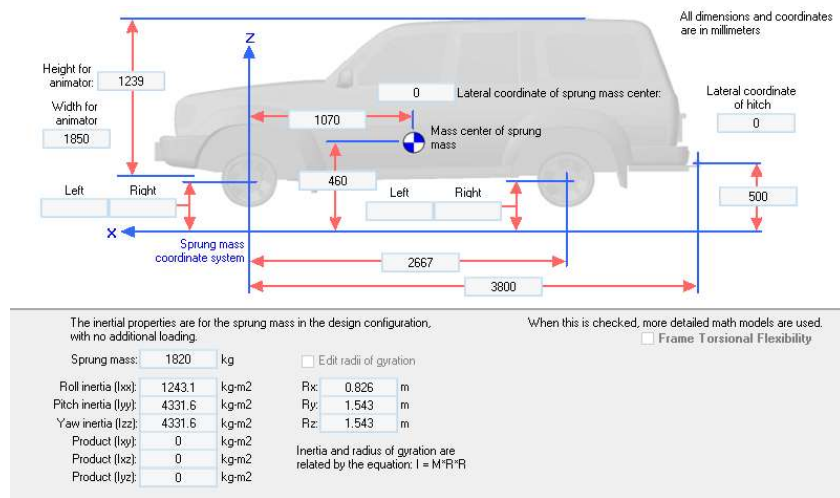


图 5-6 CarSim 模型参数

需要注意的是，图 5-6 只显示了簧载质量，整车质量还应包含悬架质量，所以总质量 $m = 1950\text{kg}$ 。把相应车辆模型参数放入预测模型中就可以完成预测模型的构建，有一点与上节不同的是，本节的预测模型还包含了车辆运动学模型，关于动力学与运动学模型的选择问题已在第四章中阐述，本节不再赘述。图 5-6 中没有给出的车辆模型参数，取值与表 5-1 一致。还有一部分模型参数是与控制器有关的，这里选用 5.1.1 节（2）中的参数。设定仿真时间为 200s。CarSim 更新状态周期是软件设定的，设定更新频率为 500Hz。

还要强调的一点是，由于此处使用的求解器是 quadprog，不是 hpipm，所以为了保证算法的实时性，重新设定 $N = 90$ 。

5.1.5 局部规划控制器仿真分析

首先介绍局部控制器的仿真结果，用到的地图有两个，一个是 5.1.2 小节中的图 5-1，另一个是 2018 年大学生方程式德国赛（Formula Student Germany Competition，简称为 FSGC）的地图，如图 5-7 所示。图中红三角代表红色桩桶，蓝三角代表蓝色桩桶，这些桩桶限定了赛道的边界。起始时刻已知的只有这些桩桶信息，可以通过第三章中介绍的中心线提取算法，把中心线提取出来（蓝色虚线），再根据汽车宽度和赛道宽度，把中心线（蓝色虚线）左右平移，产生图中的红线，即虚拟的左右边界，这样就得到了优

化算法的输入。

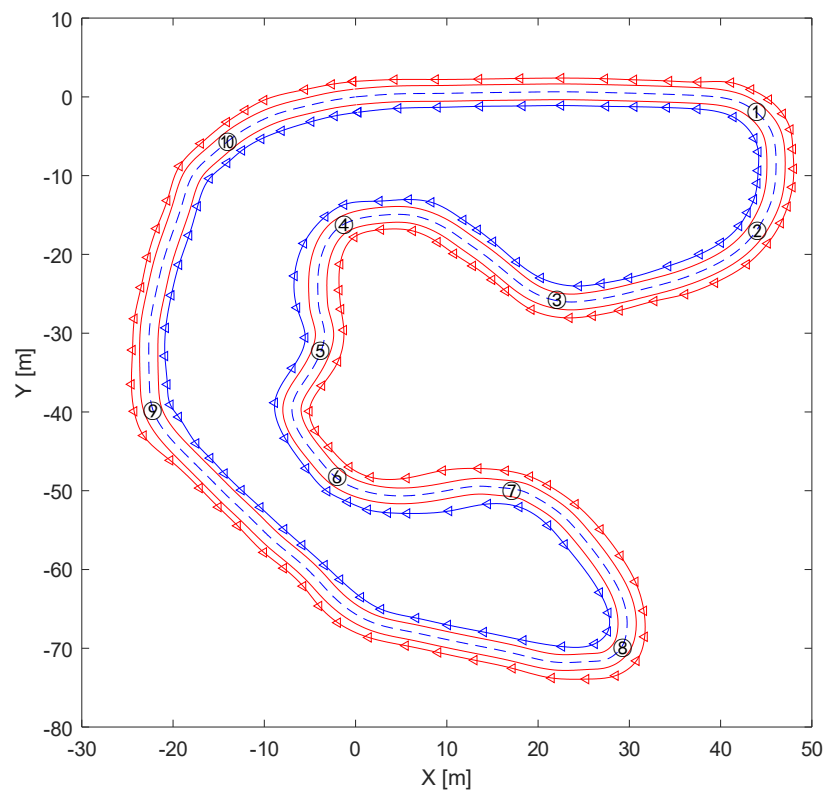
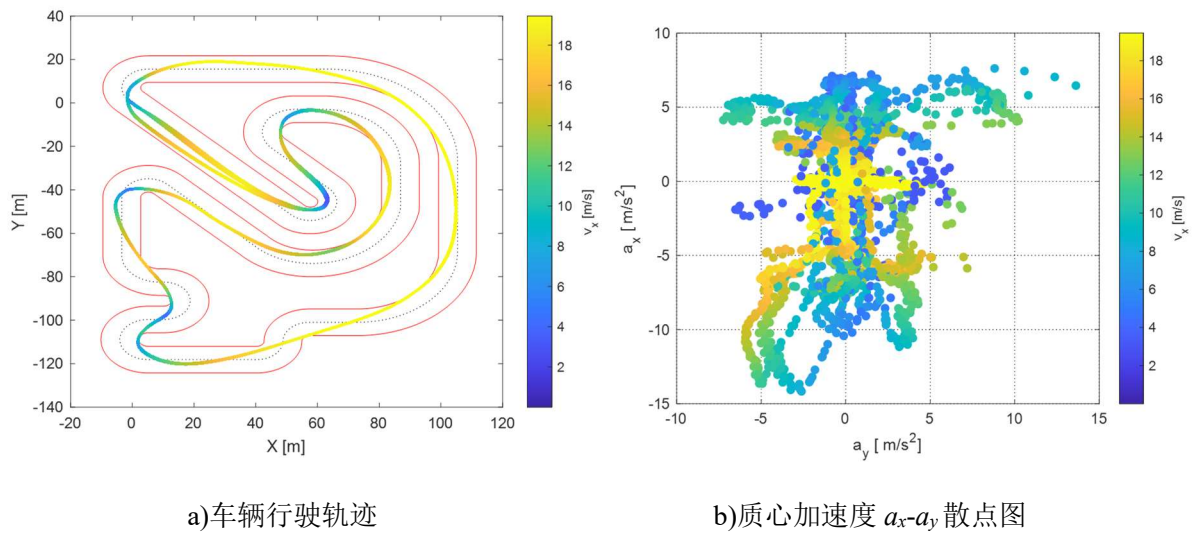


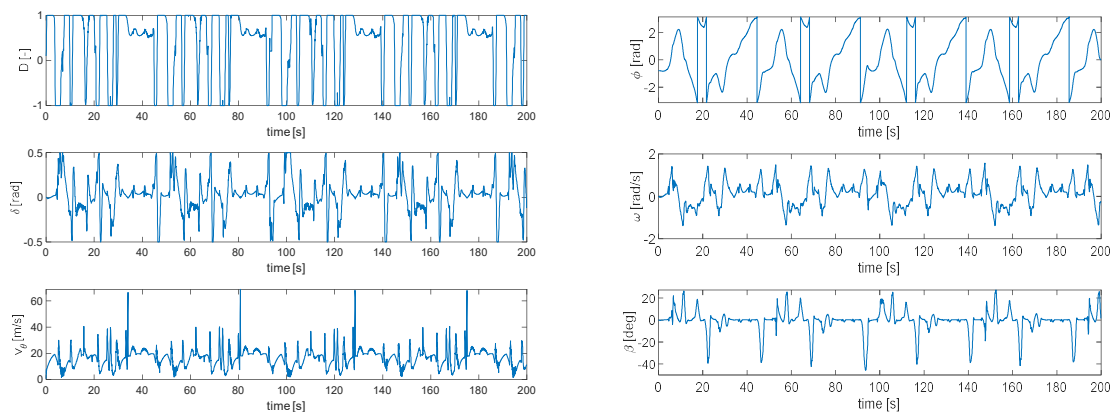
图 5-7 2018 年 FSGC 的地图

首先给出地图 1 的仿真结果，如下面一组图，图 5-8 所示。



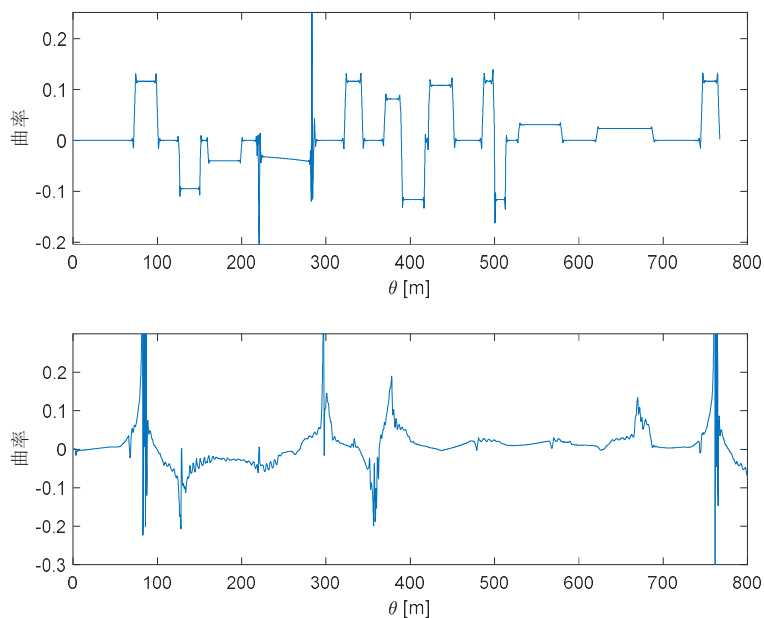
a)车辆行驶轨迹

b)质心加速度 a_x - a_y 散点图



c)控制量曲线

d)航向角、角速度和侧偏角曲线



e)中心线曲率（上）与车辆第一圈实际行驶轨迹曲率（下）

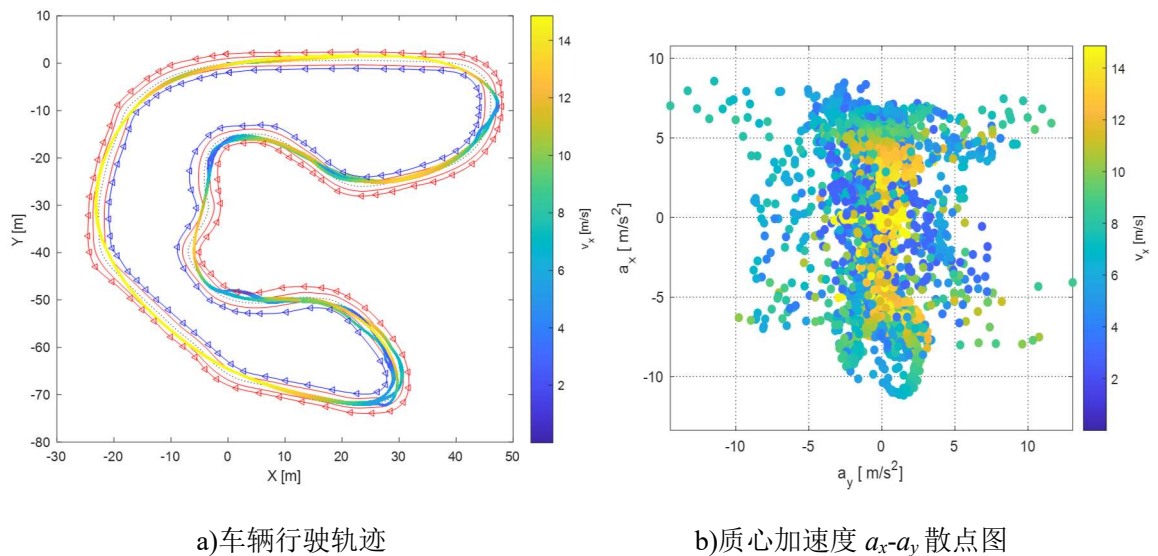
图 5-8 地图 1 的联合仿真结果

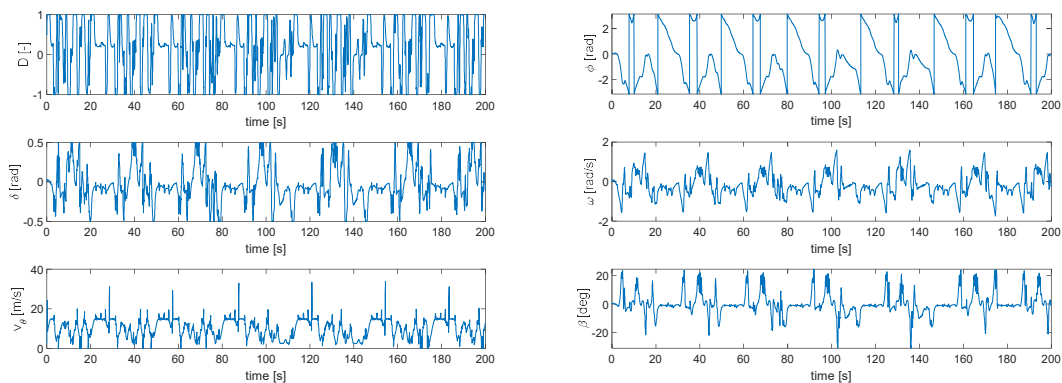
图 5-8a 显示，就算使用 CarSim 的车辆模型，算法也能够支持车辆完整完成一圈，说明算法是可行的，而且整体轨迹与 5.1.2 小节的结果类似，整条轨迹都较为平滑，平均求解时间虽然较长，达到 0.3468s，但仍然能够支撑 CarSim 模型的行驶，没有驶出赛道。车辆完整跑完了 4 圈，平均圈速为 47.57s，相比 5.1.2 小节的 45.8048s 长，这可能是预测时域 N 的减小和使用了更加精确可靠的车辆模型进行仿真导致的。从图 5-8c 中看出，控制量具有周期性，说明该算法稳定性良好，保证汽车每一圈的行驶轨迹几乎是

一样的，而且车辆没有驶出边界。图 5-8b 是质心横纵向加速度关系的散点图，车辆整体横向加速度在大部分分布在 $(-5,5)$ 这个区间，横向加速度较小，保证了车辆具有良好的稳定性，不至于出现前轴侧滑或者后轴侧滑的情况。而纵向加速度中最大减速度比最大加速度大，是因为优化模型参数中，对 PWM 占空比 d 增量的约束上下限不同导致的，这样设置可以防止车辆的急加速，并且保证紧急情况下车辆能够有效减速，从而增加车辆的安全性。图 5-8d 展示车辆行驶过程中的几个状态量，由于时间（横坐标）跨度较大，因此角速度看上去并不是很平稳，但是单独看某个预测时域的角速度可以看出由于代价函数的惩罚作用，角速度整体不会很大，通常选择的惩罚权重越大，整体角速度就越趋于小值；图 5-8d 中还显示了侧偏角曲线，可以发现有几个地方侧偏角绝对值达到 40° 以上，这是由于车辆过急弯时减速，纵向速度减小，而横向速度变化不大，造成计算出的侧偏角很大，这会导致模型的侧向力不准确，是动力学自行车模型的缺陷。最后的图 5-8e 显示了第一圈车辆行驶轨迹的曲率和中心线曲率，对比二者，发现中心线曲率大的地方一般是过弯处，而车辆行驶到这些地方一般不会出现整体曲率很大的情况（图 5-8e 下图），除了有些地方曲率会有突变，这可能是曲率离散算法的计算误差或者车辆在某个位置微调导致某点不连续造成的，总体上看，车辆会选择转弯半径大曲率小的路径行驶，以便过弯时有足够的速度，从而提升单位时间行驶的距离。

地图 2 的联合仿真结果如下图 5-9 所示。

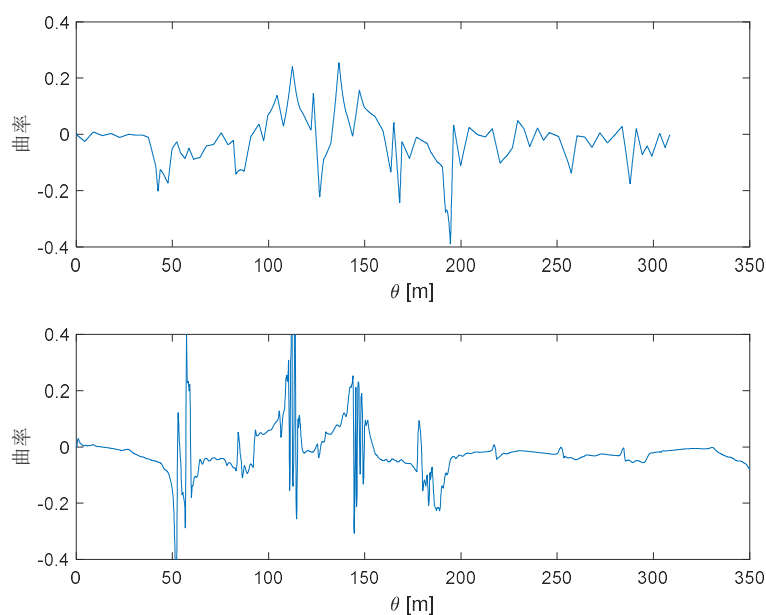
相对于地图 1, 地图 2 的赛道宽度更窄，模拟了真实的赛道，所以留给车辆的操作空间减小了很多，这点从图 5-9a 中可以看出。





c)控制量曲线

d)航向角、角速度和侧偏角曲线



e)中心线曲率（上）与车辆第一圈实际行驶轨迹曲率（下）

图 5-9 地图 2 的联合仿真结果

分析图 5-9a，可以看出由于赛道变窄，车辆在一些弯道会驶出虚拟中心线，如图 5-7 的②号弯道、⑧号弯道等，车辆在一些弯道还碰到了桩桶，如图 5-7 中的⑥号弯道，可以看到车辆碰到了左前方的红色桩桶，出现这种情况有两个原因，一个是由于算法加入了松弛因子导致边界约束是软约束，另一个是出现了求解失败导致车辆短暂失控，但同时车辆退化为 stanley 控制算法，能够马上调整回边界内，所以没有出现跟踪失败的情况，如此也能看出算法具有较高的鲁棒性。另一方面，该赛道是为方程式赛车打造的，而 CarSim 上的模拟车辆相对于方程式赛车来说，质量较大，方程式赛车质量一般为 200~300kg，而该车质量达到了 1950kg，所以转弯较为笨重，模型求不出解导致车辆失

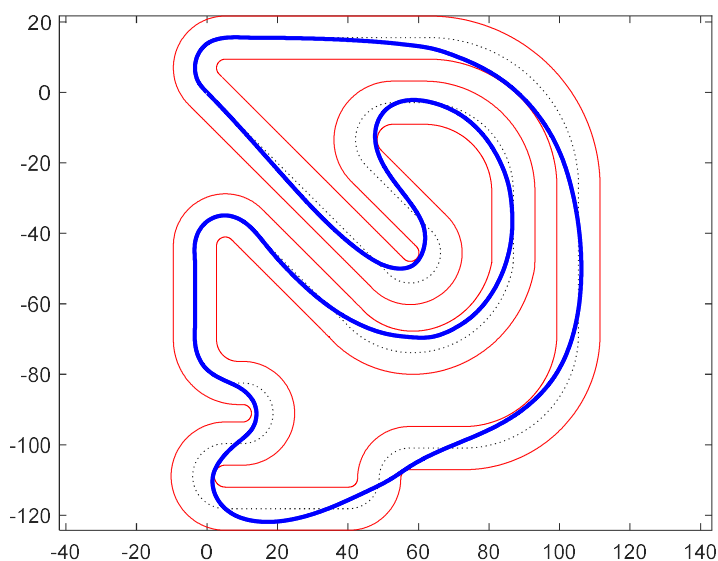
控驶出弯道也是可以预见的。

200s 仿真时间内，车辆完整跑完了 6 圈，平均圈速为 30.6493s，与 AMZRacing 车队の結果相比^[21]，平均每圈慢了 2s，作者认为是仿真车辆模型的不同导致的，如果把该仿真模型换成准确的方程式赛车模型，圈速时间应该会更低。整个过程平均求解时间为 0.3529s，虽然比较长，但是并不妨碍得到理想的仿真结果。图 5-9b 中质心加速度相对于图 5-8b 有所不同，横向加速度更加分散，这是由于赛道变窄导致的。在图 5-9c 和图 5-9d 中给出了全程了控制量与状态量曲线，同样表现出周期性。在图 5-9e 的实际行驶曲率曲线（下图）中，由于汽车偶尔会驶出虚拟边界，由 stanley 接管控制，所以一些地方曲率突变较大，转弯半径小，但是后半段曲率明显比中心线的曲率小，说明汽车尽量选择大转弯半径的路径行驶，充分利用了赛道的宽度。

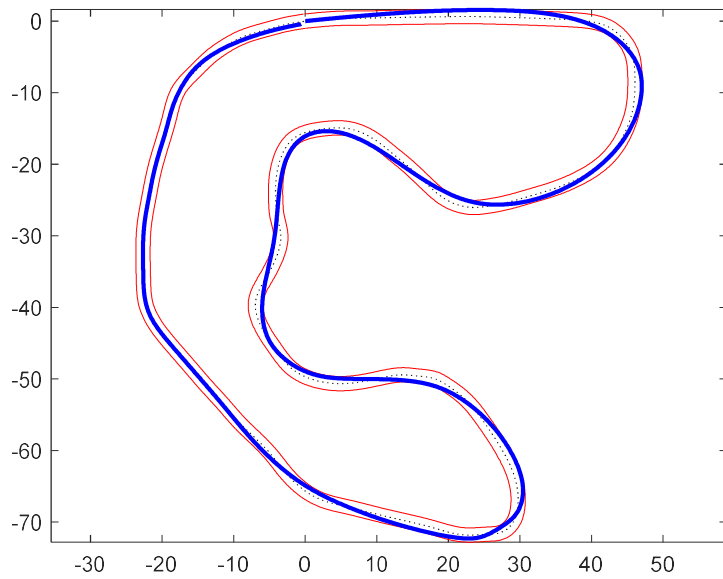
5.1.6 全局路径仿真分析

全局路径生成算法的仿真所用地图数据仍然是图 5-1 和图 5-7 的数据，通过 4.7 节阐述的算法，产生最优全局轨迹，并且使用 stanley 控制算法，通过 CarSim 与 Simulink 进行联合仿真，对比了用 stanley 跟踪中心线与跟踪最优路径的差异。

产生的全局路径如图 5-10 所示，可以看出，通过对路径点进行重采样，再进行样条插值处理产生了较为平滑的路径。



a)地图 1



b)地图 2

图 5-10 地图 1 和地图 2 路径信息，蓝色粗线为全局路径，
黑色虚线为中心线，红色实线为边界

表 5-4 测试 10 次，两幅地图的平均生成时间和方差

地图	平均生成时间	方差
1	2.8459 s	0.0074 s ²
2	1.8056 s	0.0015 s ²

在获得了所有桩桶数据后，全局路径可以离线生成，两个地图的生成时间及方差如表 5-4 所示，在 FSAC 高速循迹动态赛中，规定车辆完成 3 圈的行驶^[1]，一般第一圈的车速较慢，因为要获取所有的桩桶数据，同时完成建图，而第二圈开始就可以充分利用这些数据，规划出一条最优的全局路径，并使车辆高速行驶。在规划全局路径期间，也就是第一圈刚结束时，可以使车辆沿用第一圈的策略暂时慢速跟踪中心线，由于全局路径生成算法运行时间一般为 2~3s，并没有长到不可接受，所以这些时间的消耗是值得的。

地图 1 中全局路径总长度为 700.451m，中心线总长度为 765.887m，地图 2 中全局路径总长度为 304.8707m，中心线总长度为 308.5941m。两个图全局优化路径总长度都比中心线长度小，而且随着赛道宽度的增大，其差距也增大，地图 1 甚至相差了 60 多

米，说明优化算法可以使得路径总长度减小，这将有利于圈速时间的减小，提升车辆的总体速度。

通过 CarSim 上自带的纵向控制器控制汽车恒速行驶，地图 1 速度设为 30km/h，地图 2 由于较窄，速度设为 25km/h，输入量只有前轮转角，后轮转角恒为 0，前轮转角数值通过 stanley 控制器进行计算，仿真时间设置为 200s，两个地图的仿真结果如图 5-11 和图 5-12 所示。表 5-5~表 5-8 是仿真完成后的得到的一些数据。

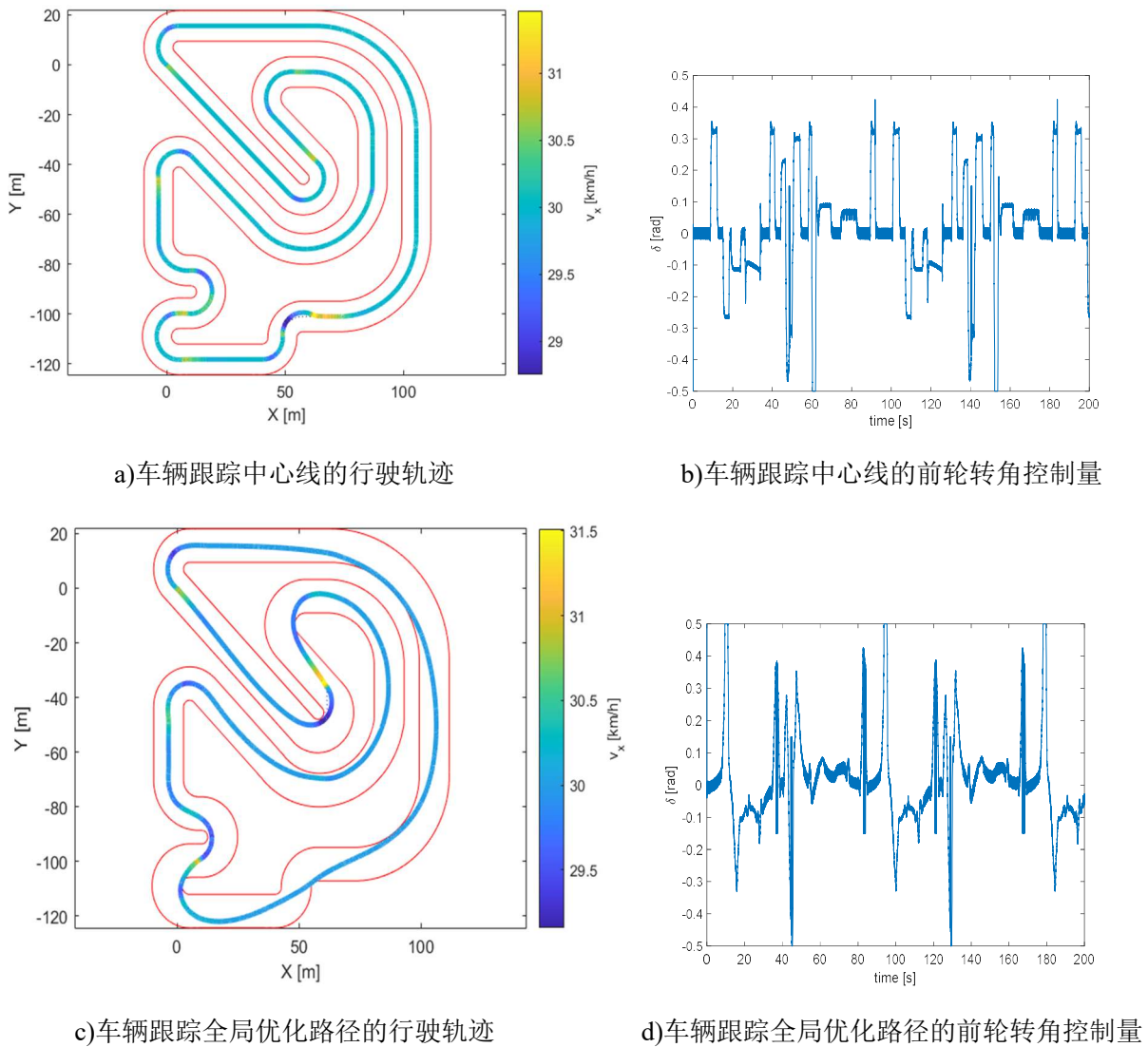


图 5-11 地图 1 的仿真结果

表 5-5 地图 1 跟踪中心线的相关数据

数据	平均圈速	路径最大曲率 (绝对值)	路径平均曲率 (绝对值)
数值	91.9336s	0.2518m ⁻¹	0.03765m ⁻¹

表 5-6 地图 1 跟踪最优路径的相关数据

数据	平均圈速	路径最大曲率 (绝对值)	路径平均曲率 (绝对值)
数值	84.3713s	0.20015m ⁻¹	0.033785m ⁻¹

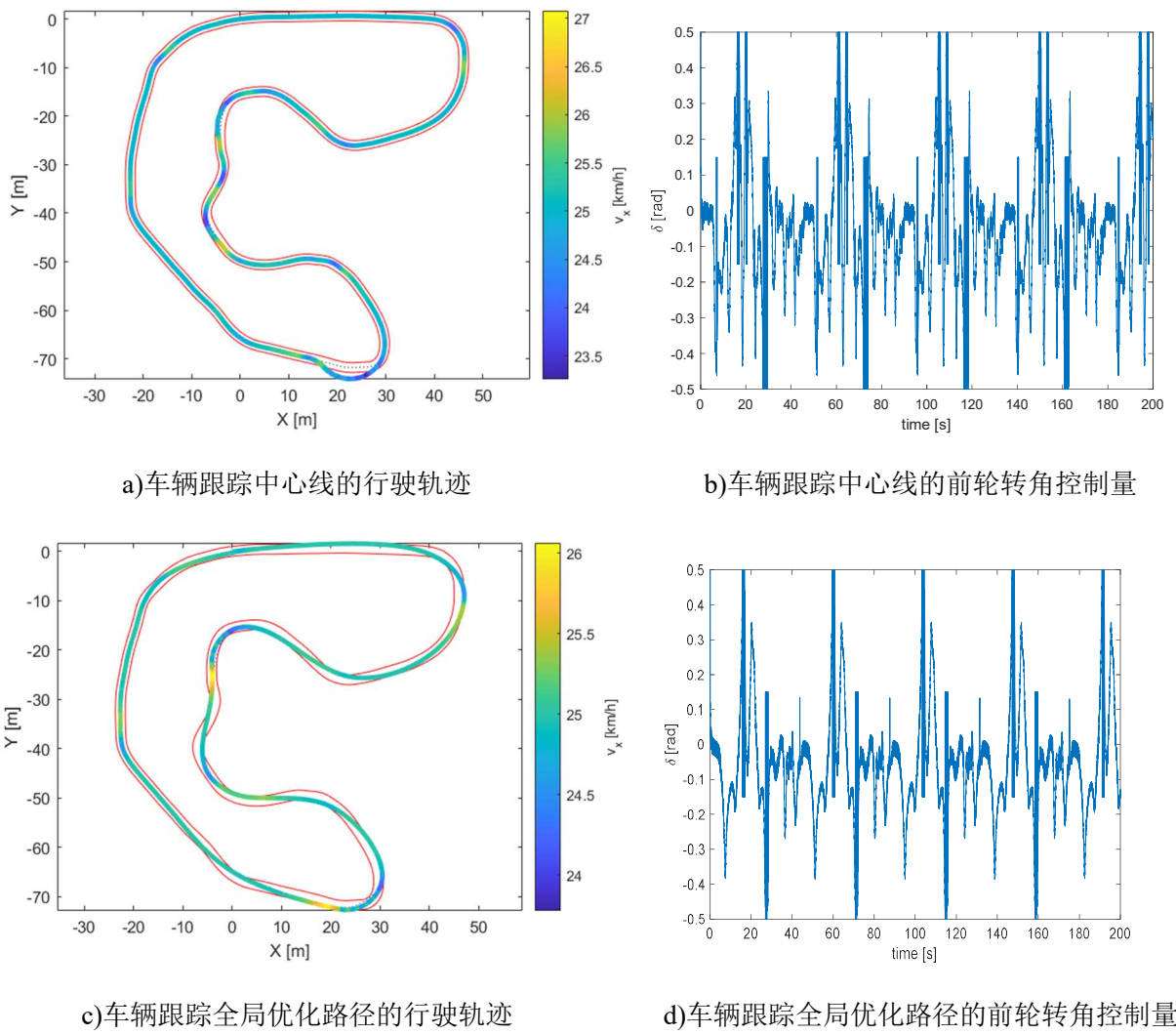


图 5-12 地图 2 的仿真结果

表 5-7 地图 2 跟踪中心线的相关数据

数据	平均圈速	路径最大曲率 (绝对值)	路径平均曲率 (绝对值)
数值	44.4252s	0.38922m ⁻¹	0.054853m ⁻¹

表 5-8 地图 2 跟踪最优路径的相关数据

数据	平均圈速	路径最大曲率 (绝对值)	路径平均曲率 (绝对值)
数值	43.7818s	0.2338m ⁻¹	0.046228m ⁻¹

两个地图跟踪最优路径的圈速时间都比跟踪中心线小，而且第 1 个地图足足快了有 7s，这是因为赛道更加宽敞了，车辆拥有的操作空间更多了。圈速变快的主要原因有两个，一个是总体路径长度缩短了，另一个是车辆的轨迹跟踪更加轻松了，像图 5-7 中的⑤号弯道，在图 5-12a 与图 5-12c 中有明显区别，图 5-12a 由于要跟踪中心线，所以还是转弯通过，而在图 5-12c 中直接直线通过，减少了不必要的转弯，这样跟踪的横向误差就不会太大，有利于提升控制算法的性能。两个地图中，全局最优路径的最大曲率和平均曲率都小于中心线，有利于车辆用更快的圈速完成跑动。此外，图中还给出了控制量的曲线，图 5-11b 的控制量相对较平缓，而图 5-11d 的控制量则变化较大，控制量曲线与具体的控制算法有关，stanley 算法是利用路径参考点的航向角偏差与横向误差来计算控制量的，观察地图 1 的全局优化路径可以发现有些地方路径的航向角变化较大，导致了控制量频繁达到最大或最小值。相对来说，图 5-12b 和图 5-12d 更能说明全局优化路径的优势，由于赛道没那么宽敞，而且曲率普遍大于图 5-11，所以中心线与全局最优路径的航向角变化都比较大，导致跟踪中心线和最优路径的控制量变化均较大，但是跟踪最优路径时控制量的变化明显没那么频繁，说明车辆选择转弯半径更大的路径行驶，减小了圈速时间，从图 5-12c 中也可以看出车辆少走了很多弯路，而且没有出现如图 5-12a 那样在最下方的弯道驶出赛道的情况。

通过以上分析，可以证明全局路径生成算法是可行的，虽然它有自己的缺陷，但是

其对于圈速的提升也是显著的。以全局优化路径作为待跟踪轨迹，如果使用更加成熟的控制算法进行跟踪，一定可以在更高的速度下完成更加令人满意的轨迹跟踪。

5.3 小结

本章通过四阶龙格-库塔法仿真器和 CarSim/Simulink 联合仿真器对算法的性能进行了验证，并对比分析了多种结果，证明了算法的有效性与鲁棒性。

总结与展望

1. 论文工作总结

论文主要针对无人驾驶方程式赛车规划层算法进行了研究，在已知感知层与定位层数据的情况下，详细研究了路径规划层的求解思路。规划层就像无人赛车的大脑，起着决策者的作用，良好的规划层算法不仅求解效率高，能在指定时间内给出问题的解，而且会考虑多种失效形式，并进行合适的处理，因此有效提升规划层算法的效率和鲁棒性，必将是未来一大发展趋势。主要研究工作总结如下：

(1) 建立了车辆动力学自行车模型与轮胎模型，为了弥补动力学模型在低速下的不足，又建立了车辆运动学模型，以这两个模型为基础作为优化问题的预测模型。同时研究了轮胎模型中各个参数的影响，确定各参数的大致取值范围，并且研究了如何拟合已知轮胎数据。

(2) 参照计算机科学中贪婪算法的思路，实现了中心线提取算法。利用 Delaunay 三角剖分，把连续的状态空间离散化；随后依据赛规和其他已知条件，合理地设计了代价函数，接着利用贪婪算法进行搜索，得出一系列潜在路径和其中的最优路径，并把最优路径作为中心线。最后，对该算法进行了仿真，结果显示算法在颜色信息有错误甚至全部错误的情况下仍然能够得出正确结果，说明算法具备一定的鲁棒性。

(3) 运用最优化理论，对中心线进行优化。首先实现了一个集规划控制一体化的局部控制器，从非线性 MPC 问题出发，研究了优化控制问题的基本求解思路。通过线性化和离散化处理，把车辆动力学和运动学模型转化为了线性时变预测模型，并通过它们的线性组合把二者结合到一起。根据横纵向误差定义了输出量并进行了线性化处理。合理设计了目标函数和赛道的边界约束，并利用预测模型把最优化问题转化为方便计算机求解的 QP 问题。考虑到实时性需要，对算法进行了优化，并且对算法求解失败的情况进行了处理。最后，基于局部控制器的算法框架，设计了全局路径生成算法，以方便车辆运用更加成熟的控制算法进行轨迹跟踪，使规划与控制解耦合。

(4) 对优化算法进行仿真分析。运用四阶龙格-库塔法仿真器和 CarSim/Simulink 联合仿真器对优化算法进行验证。在四阶龙格-库塔法仿真器里，对比了未优化算法和优化算法的性能，还有 quadprog 与 hpipm 求解器在求解速度与求解结果上的差异，证明了

对算法进行优化的必要性和 hpipm 求解器的优异性能。在 CarSim/Simulink 联合仿真器中，利用更为精确的 CarSim 车辆模型，对局部规划控制算法和全局路径生成算法进行了验证，产生了更加准确的仿真结果，充分展示了算法的有效性和稳定性。

最后要特别强调，尽管研究对象是无人赛车，但是只要得到道路的中心线和边界数据，论文所提优化算法完全可以拓展到乘用车上使用。

2. 工作展望

论文详细研究了路径规划层算法的设计，但是由于条件有限，很多问题的研究还不够深入，有很大的改良空间，这里主要指出以下几点：

(1) 中心线提取算法的改良与实车验证。中心线提取算法的效率仍然有提升的空间，为了达到 50ms 内求得解的要求，可以查阅更多资料，设计更加快速有效的算法。此外，论文的算法目前也只限于仿真数据，并没有经过实车验证，算法真实的效果要通过进一步的实车验证进行考量。

(2) 参数标定与实车实验。论文所用的车辆参数都是来自于其他平台，并不适用于华南理工大学的无人赛车，对于具体的方程式赛车，需要对轮胎参数与纵向力参数进行标定，使预测模型更加准确可靠。由于现实原因，路径优化算法只是在 MATLAB 平台和 CarSim/Simulink 联合仿真器中进行了仿真，算法缺乏实车实验，所以未来需要完成另一个工作是，把算法移植到 C++ 平台，通过 ROS 系统与感知层、定位层和控制层进行通信，以进一步验证算法的可行性与稳定性。

(3) 考虑侧偏角约束。如果希望车辆高速行驶时具备良好的稳定性，就需要把侧偏角约束考虑到模型内，论文运用了简化的模型，并没有考虑侧偏角约束，未来可以把侧偏角约束加入到优化问题的约束内，使车辆充分利用地面附着力稳定行驶。

(4) 建立更加精确的车辆动力学模型和轮胎模型。为了兼顾求解效率与预测精度，论文只是建立了简单的车辆动力学模型，如果想要更加准确地预测车辆行为，可以考虑牺牲一点求解的效率，建立更加复杂的车辆动力学模型，如 6 自由度车辆非线性动力学模型等。除此之外，文本所用的轮胎模型也是最简单的魔术轮胎模型，如果想要更准确地计算侧向力，可以考虑应用像 MF-Swift 轮胎模型等更加准确的模型。

(5) 由于时间与精力有限，论文只研究了两个 QP 问题的求解器，后续工作可以尝

试一些其他常用的求解器，找到比 `hpipm` 更为高效、合适的求解器；同时，也应该对求解器内部的相关求解参数进行研究，了解如何合理设定这些参数，以提升算法的求解效率。

参考文献

- [1] 胡纪滨. 中国大学生无人驾驶方程式大赛规则[Z]. 北京: 中国汽车工程学会, 2019.
- [2] 李永丹,马天力,陈超波,韦宏利,杨琼楠.无人驾驶车辆路径规划算法综述[J].国外电子测量技术,2019,38(06):72-79.
- [3] Dijkstra E W. A note on two problems in connexion with graphs[J]. Numerische mathematik, 1959, 1(1): 269-271.
- [4] Hart P E, Nilsson N J, Raphael B. A formal basis for the heuristic determination of minimum cost paths[J]. IEEE transactions on Systems Science and Cybernetics, 1968, 4(2): 100-107.
- [5] LaValle S M, Kuffner Jr J J. Randomized kinodynamic planning[J]. The international journal of robotics research, 2001, 20(5): 378-400.
- [6] Kuffner J J, LaValle S M. RRT-connect: An efficient approach to single-query path planning[C]. Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No. 00CH37065). IEEE, 2000, 2: 995-1001.
- [7] Karaman S, Frazzoli E. Sampling-based algorithms for optimal motion planning[J]. The international journal of robotics research, 2011, 30(7): 846-894.
- [8] Gammell J D, Srinivasa S S, Barfoot T D. Informed RRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic[C]. 2014 IEEE/RSJ International Conference on Intelligent Robots and Systems. IEEE, 2014: 2997-3004.
- [9] 杨一波,王朝立.基于改进的人工势场法的机器人避障控制及其 MATLAB 实现[J].上海理工大学学报,2013,35(05):496-500.
- [10] 安林芳,陈涛,成艾国,方威.基于人工势场算法的智能车辆路径规划仿真[J].汽车工程,2017,39(12):1451-1456.
- [11] 张明环,张科,张宇辰.车辆自主避障的触须算法研究[J].机械科学与技术,2012,31(12):1993-1996.

- [12] 史恩秀,陈敏敏,李俊,黄玉美.基于蚁群算法的移动机器人全局路径规划方法研究[J].农业机械学报,2014,45(06):53-57.
- [13] 黄辰,费继友,刘洋,李花,刘晓东.基于动态反馈 A~*蚁群算法的平滑路径规划方法[J].农业机械学报,2017,48(04):34-40+102.
- [14] 冯来春,梁华为,杜明博,余彪.基于 A~*引导域的 RRT 智能车辆路径规划算法[J].计算机系统应用,2017,26(08):127-133.
- [15] Liniger A, Domahidi A, Morari M. Optimization - based autonomous racing of 1: 43 scale RC cars[J]. Optimal Control Applications and Methods, 2015, 36(5): 628-647.
- [16] Christ F, Wischnewski A, Heilmeier A, et al. Time-optimal trajectory planning for a race car considering variable tyre-road friction coefficients[J]. Vehicle System Dynamics, 2019: 1-25.
- [17] Heilmeier A, Wischnewski A, Hermansdorfer L, et al. Minimum curvature trajectory planning and control for an autonomous race car[J]. Vehicle System Dynamics, 2019: 1-31.
- [18] 李松焱,闵永军,王良模,安丽华.轮胎动力学模型的建立与仿真分析[J].南京工程学院学报(自然科学版),2009,7(03):34-38.
- [19] Bakker E, Nyborg L, Pacejka H B. Tyre modelling for use in vehicle dynamics studies[J]. SAE Transactions, 1987: 190-204.
- [20] 周杰,丁贤荣,汪德燿.平面散点集 Delaunay 三角剖分的一种高效方法[J].测绘信息与工程,2003(06):21-23.
- [21] Kabzan J, Valls M I, Reijgwart V, et al. Amz driverless: The full autonomous racing system[J]. arXiv preprint arXiv:1905.05150, 2019.
- [22] Lam D, Manzie C, Good M. Model predictive contouring control[C]. 49th IEEE Conference on Decision and Control (CDC). IEEE, 2010: 6137-6142.
- [23] 孙银健. 基于模型预测控制的无人驾驶车辆轨迹跟踪控制算法研究[D].北京理工大学,2015.
- [24] 许小勇,钟太勇.三次样条插值函数的构造与 Matlab 实现[J].兵工自动化,2006(11):76-78.

-
- [25]刘凯,陈慧岩,龚建伟,陈舒平,张玉.高速无人驾驶车辆的操控稳定性研究[J].汽车工程,2019,41(05):514-521.
- [26]张亮修,吴光强,郭晓晓.自主车辆线性时变模型预测路径跟踪控制[J].同济大学学报(自然科学版),2016,44(10):1595-1603.
- [27]Jun N I, Jibin H U. Autonomous driving system design for formula student driverless racecar[C] 2018 IEEE Intelligent Vehicles Symposium (IV). IEEE, 2018: 1-6.
- [28]Katriniok A, Abel D. LTV-MPC approach for lateral vehicle guidance by front steering at the limits of vehicle dynamics[C]. 2011 50th IEEE Conference on Decision and Control and European Control Conference. IEEE, 2011: 6828-6833.
- [29]Barnes R J. Matrix differentiation[J]. Springs Journal, 2006: 1-9.
- [30]Frison G , Diehl M . HPIPM: a high-performance quadratic programming framework for model predictive control[J]. 2020.
- [31]Hoffmann G M , Tomlin C J , Montemerlo M , et al. Autonomous automobile trajectory tracking for off-road driving: Controller design, experimental validation and racing[C]. American Control Conference, 2007. ACC '07. IEEE, 2007.
- [32]Falcone P, Borrelli F, Tseng H E, et al. Linear time - varying model predictive control and its application to active steering systems: Stability analysis and experimental validation[J]. International Journal of Robust and Nonlinear Control: IFAC - Affiliated Journal, 2008, 18(8): 862-875.

附录

基于贪婪策略的中心线提取算法仿真实验中相关代价量的权重见附表 1 与附表 2。

附表 1 有颜色信息情况下中心线提取算法的权重参数

代价量	权重
最大偏转角	搜索所有潜在路径：0.5
$\beta_{\max} / \text{rad}$	搜索最优路径：0.4
左(右)边界邻近桩桶的距离与标准距离的标准差	搜索所有潜在路径：0.2
l_{std} / m	搜索最优路径：0.2
左右配对桩桶的距离与道路宽度的标准差	搜索所有潜在路径：0.25
w_{std} / m	搜索最优路径：0.2
左右桩桶颜色 (颜色正确输出 0, 有一个未知颜色输出 0.5, 两个颜色均相同或者均未知输出 1)	搜索所有潜在路径：0.3
	搜索最优路径：0.3
左右边界是否相交 (相交输出 1, 不相交输出 0)	搜索所有潜在路径：1
	搜索最优路径：1
路径长度	搜索所有潜在路径：0.25
s / m	搜索最优路径：0.25
左右边界桩桶数量的比 (超过 3 输出 1, 否则输出 0)	搜索所有潜在路径：0
	搜索最优路径：0.5

附表 2 无颜色信息 (颜色完全缺失) 情况下中心线提取算法的权重参数

代价量	权重
最大偏转角	搜索所有潜在路径：1
$\beta_{\max} / \text{rad}$	搜索最优路径：0.8
左(右)边界邻近桩桶的距离与标准距离的标准差	搜索所有潜在路径：0.2
l_{std} / m	搜索最优路径：0.2

附表 3 无颜色信息（颜色完全缺失）情况下中心线提取算法的权重参数（续）

代价量	权重
左右配对桩桶的距离与道路宽度的标准差 w_{std} / m	搜索所有潜在路径：0.25 搜索最优路径：0.2
左右桩桶颜色 （颜色正确输出 0，有一个未知颜色输出 0.5，两个颜色均相同或者均未知输出 1）	搜索所有潜在路径：0 搜索最优路径：0
左右边界是否相交 （相交输出 1，不相交输出 0）	搜索所有潜在路径：1 搜索最优路径：1
路径长度 s / m	搜索所有潜在路径：0.25 搜索最优路径：0.25
左右边界桩桶数量的比 （超过 3 输出 1，否则输出 0）	搜索所有潜在路径：0 搜索最优路径：0.5

致谢

至此，本科毕业设计算是告一段落了，由于疫情原因，本次毕业设计基本是在家里完成的，其中遇到了不少困难，但所幸最后都克服了。大学四年如白驹过隙，转眼即逝，在此非常感谢那些成长路上给予过我帮助的人，特别是我的家人们，他们一直默默支持我，感谢陪伴我到大学毕业的每一个人、每一个朋友。

本次本科毕业设计的顺利完成，首先要感谢我的指导老师李巍华老师，李老师严谨的学术态度、不厌其烦的指导使我受益匪浅。李老师总在百忙之中抽出时间给予我以及实验室的同门耐心的指导，并且提出合理的建议，指出研究中的不足，在此向李巍华老师致以衷心的感谢。

感谢所有帮助过我的同门师兄，特别是钟思祺师兄，在我选题迷茫时给予过我很大的帮助，而且每次遇到困难，他都耐心地帮我解决或者给予我建议，跟他在一起学习使我掌握了更多车辆控制方面的知识，让我受益匪浅，论文的完成很大部分归功于他的悉心帮助与指导。还有李伟师兄与郑少武师兄，他们曾在中心线提取算法上给予过我宝贵的意见及帮助。

感谢负责设计制造新一代华南理工大学无人赛车的各位队友们，因为有你们制造的精品赛车，才能使无人系统的各种算法有运行的保证。

感谢大学四年以来的一些挚友们，感谢你们曾在我孤独无助的时候陪伴我，与我同甘共苦，我的生活因你们而更加精彩。

感谢大学四年以来我的室友们，助我拥有一个稳定的学习与生活环境，让我在学习科研路上不孤单。

最后感谢我的父母，感谢他们多年的养育之恩，我如今收获的所有成就都离不开他们的理解与支持，唯有往后更加努力，才能无愧于他们的付出。